

Python para Análise de Dados

Apostila

Python para Análise de Dados — Apostila

Como usar esta apostila

Estrutura da apostila

Sobre os dados usados nos exemplos

Pré-requisitos

Módulo 1 — Fundamentos de Python

Tópicos

Introdução

O que é e por que importa

Como funciona (com exemplo comentado)

Erros comuns de quem está começando

Exercício prático

Solução comentada

Checklist antes de avançar

Ambiente de desenvolvimento

O que é e por que importa

Como funciona (com exemplo comentado)

Erros comuns de quem está começando

Exercício prático

Solução comentada

Checklist antes de avançar

Variáveis e print()

O que é e por que importa

Como funciona (com exemplo comentado)

Erros comuns de quem está começando

Exercício prático

Solução comentada

Checklist antes de avançar

Inteiros (int)

O que é e por que importa

Como funciona (com exemplo comentado)

Erros comuns de quem está começando

Exercício prático

Solução comentada

Checklist antes de avançar

Floats

O que é e por que importa

Como funciona (com exemplo comentado)

Erros comuns de quem está começando

Exercício prático

Solução comentada

Checklist antes de avançar

Booleanos

O que é e por que importa

Como funciona (com exemplo comentado)

Erros comuns de quem está começando

Exercício prático

Solução comentada

Checklist antes de avançar

Conversão de tipos (type casting)

O que é e por que importa

Como funciona (com exemplo comentado)

Erros comuns de quem está começando

Exercício prático

Solução comentada

- Checklist antes de avançar
- Operadores aritméticos
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Operadores de comparação
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Operadores lógicos
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Condicionais (if/elif/else)
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Loops (for/while)
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Strings (fundamentos)
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Trabalhando com strings (métodos gerais)
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- strip, replace, split
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Listas
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Tuplas

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Dicionários

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Sets

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

List comprehensions

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Funções e métodos (diferença entre os dois)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Definindo funções (def, parâmetros, return)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

*args e **kwargs

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Funções lambda

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Funções built-in (len, sum, map, filter, etc.)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Módulo 2 — Python Intermediário e Ambiente

- Tópicos
- OOP para análise de dados

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Módulo random

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Expressões regulares (re) — nível básico

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

JSON (ler e escrever)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

CSV (ler e escrever sem pandas)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

pip (instalando pacotes)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Ambientes virtuais (venv)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Conda (alternativa ao venv)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

VS Code (configuração básica para Python)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

JupyterLab

- O que é e por que importa
- Como funciona (com exemplo comentado)

- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar
- Google Colab
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Módulo 3 — NumPy
 - Tópicos
 - Introdução ao NumPy
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Criando arrays
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Indexação e slicing
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Operações vetorizadas e broadcasting
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Funções estatísticas e agregações
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Reshape e dimensões
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Boolean masking e filtragem
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Checklist do Módulo 3
- Módulo 4 — Pandas Essencial
 - Tópicos

- Introdução ao Pandas: Series e DataFrame
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Lendo e escrevendo dados (CSV, Excel, JSON)
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Explorando um DataFrame
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Seleção e indexação (loc, iloc)
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Filtragem de dados
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Criando e modificando colunas
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Ordenação
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Agrupamento com groupby
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Agregações com agg
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Merge, join e concatenação
 - O que é e por que importa

- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Introdução a valores ausentes

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Tabelas dinâmicas (pivot_table)

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar
- Checklist do Módulo 4

Módulo 5 — Limpeza de Dados

Tópicos

Estratégias para valores ausentes

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Duplicatas

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Tipos de dados e conversão

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Limpeza de texto

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Trabalhando com datas

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Outliers: detecção e tratamento

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar

Checklist de qualidade de dados

- O que é e por que importa
- Como funciona (com exemplo comentado)
- Erros comuns de quem está começando
- Exercício prático
- Solução comentada
- Checklist antes de avançar
- Checklist do Módulo 5

Módulo 6 — Estatística e EDA

- Tópicos
- Medidas de tendência central
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Medidas de dispersão
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Distribuições e histogramas
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Correlação entre variáveis
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Análise univariada com Pandas
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Análise bivariada e segmentação
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Amostragem e introdução à inferência
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
- Estruturando uma EDA completa
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar

- Checklist do Módulo 6
- Módulo 7 — Visualização
 - Tópicos
 - Introdução ao Matplotlib
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Gráficos de linha e barra
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Histogramas e boxplots
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Scatter plots e relação entre variáveis
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Customização de gráficos
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Introdução ao Seaborn
 - O que é e por que importa
 - Como funciona (com exemplo comentado)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Escolhendo o gráfico certo
 - O que é e por que importa
 - Como funciona (guia de decisão)
 - Erros comuns de quem está começando
 - Exercício prático
 - Solução comentada
 - Checklist antes de avançar
 - Checklist do Módulo 7
 - Projeto Final — Projeto Integrador
 - O que é este projeto
 - O dataset
 - Etapas do projeto
 - Checklist de conclusão
 - E depois?

Python para Análise de Dados — Apostila

Esta apostila foi criada para levar alguém sem experiência profunda em programação —

mas que já sabe o básico de Python (variáveis, tipos, condicionais, loops, listas e dicionários) — até um nível intermediário/avançado de análise de dados com Python.

O foco é **aprender fazendo**. Cada tópico tem explicação curta, exemplo comentado, erros comuns, exercício prático e solução comentada. Não é necessário decorar nada: o objetivo é entender o suficiente para resolver o exercício e seguir em frente.

Como usar esta apostila

- Siga os módulos **na ordem**. Cada um assume que você já sabe o conteúdo dos anteriores.
- Dentro de cada módulo, siga os tópicos na ordem numerada.
- Não pule o exercício prático. É nele que o conceito realmente gruda.
- Se travar em algum exercício, releia a seção “Como funciona” do tópico antes de abrir a solução.
- Ao final de cada módulo, use o checklist do último tópico para decidir se está pronto para avançar.

Estrutura da apostila

Módulo	Tema	Tempo estimado
Módulo 1	Fundamentos de Python	2–3 semanas
Módulo 2	Python Intermediário e Ambiente	1 semana
Módulo 3	NumPy	3–4 dias
Módulo 4	Pandas Essencial	2 semanas
Módulo 5	Limpeza de Dados	1 semana
Módulo 6	Estatística e EDA	1–2 semanas
Módulo 7	Visualização	1–2 semanas
Projeto Final	Projeto integrador	3–5 dias

Os tempos são só uma referência — vá no seu ritmo. O importante é sair de cada tópico conseguindo resolver o exercício sozinho.

Sobre os dados usados nos exemplos

Para dar sensação de continuidade, os módulos 1 e 2 usam como pano de fundo uma **loja fictícia** (a “Loja da Ana”), com dados de vendas simples. A partir do módulo 4, quando começamos a usar pandas, também aparecem datasets públicos reais (sempre com uma versão inline de backup, caso você não consiga baixar nada).

Pré-requisitos

- Python instalado (o Módulo 1, tópico 2, ensina isso).
- Vontade de errar bastante — é assim que se aprende a programar.

ewpage

Módulo 1 — Fundamentos de Python

Neste módulo você vai aprender a base da linguagem Python: como escrever e rodar código, os tipos de dados fundamentais, como tomar decisões e repetir tarefas com o código, e como organizar dados em listas, dicionários e outras estruturas. Tudo isso usando como pano de fundo os dados de vendas de uma pequena loja fictícia, a “Loja da Ana”.

Ao final deste módulo você será capaz de escrever pequenos programas Python do zero,

sem depender de nenhuma biblioteca externa.

Tópicos

1. Introdução
2. Ambiente de desenvolvimento
3. Variáveis e print()
4. Inteiros (int)
5. Floats
6. Booleanos
7. Conversão de tipos (casting)
8. Operadores aritméticos
9. Operadores de comparação
10. Operadores lógicos
11. Condicionais (if/elif/else)
12. Loops (for/while)
13. Strings (fundamentos)
14. Trabalhando com strings (métodos gerais)
15. strip, replace, split
16. Listas
17. Tuplas
18. Dicionários
19. Sets
20. List comprehensions
21. Funções e métodos (diferença)
22. Definindo funções (def, parâmetros, return)
23. *args e **kwargs
24. Funções lambda
25. Funções built-in (len, sum, map, filter...)

Depois de terminar todos os tópicos, siga para o Módulo 2 — Python Intermediário e Ambiente.

ewpage

Introdução

— Módulo 1 — Fundamentos de Python · Tópico 1 de 25

O que é e por que importa

Python é uma linguagem de programação criada para ser fácil de ler e escrever. Isso pode parecer um detalhe bobo, mas é o motivo principal pelo qual ela virou a linguagem mais usada em análise de dados: você passa menos tempo lutando com a sintaxe e mais tempo pensando no problema que quer resolver.

Pensa assim: se Excel é uma calculadora com superpoderes para tabelas, Python é uma calculadora ainda mais poderosa, sem limite de linhas, que você programa para fazer exatamente o que quiser — repetir uma tarefa mil vezes, cruzar dados de fontes diferentes, gerar gráficos, detectar padrões. A diferença é que, em vez de clicar em menus, você escreve instruções em texto.

“Análise de dados” é o processo de pegar um monte de números e informações soltas (vendas de um mês, respostas de uma pesquisa, temperaturas de uma cidade) e transformar isso em respostas para perguntas concretas: “qual produto vendeu mais?”, “os clientes estão satisfeitos?”, “vai chover amanhã?”. Python entra como a ferramenta que te ajuda a carregar esses dados, limpá-los, calcular coisas sobre eles e visualizar o resultado.

Ao longo desta apostila, você vai construir esse conhecimento em cima de um exemplo que se repete: os dados de vendas da “Loja da Ana”, uma lojinha fictícia de bairro. Começamos do zero absoluto e, módulo a módulo, vamos adicionando ferramentas até você conseguir

fazer uma análise completa sozinho.

Como funciona (com exemplo comentado)

Ainda não vamos escrever código de verdade neste tópico (isso começa no próximo, quando configuramos o ambiente). Mas para você já sentir o gostinho, aqui está o tipo de coisa que você será capaz de fazer ao final do Módulo 1:

```
# Uma lista com o valor de cada venda do dia, na Loja da Ana
vendas_do_dia = [45.90, 120.00, 15.50, 200.00, 33.00]

# Somando todas as vendas para saber o faturamento do dia
faturamento = sum(vendas_do_dia)

# Mostrando o resultado na tela
print("Faturamento do dia:", faturamento)
```

Repare que o código lê quase como português: “some as vendas do dia” e “mostre o faturamento”. Essa é a filosofia do Python — código que se lê como uma receita de bolo.

Erros comuns de quem está começando

- Achar que precisa “decorar” Python antes de começar a programar. Na prática, você aprende programando e errando — ninguém decora a sintaxe inteira antes de escrever a primeira linha.
- Confundir “aprender Python” com “aprender análise de dados”. Python é a ferramenta; análise de dados é o objetivo. Esta apostila ensina os dois juntos, mas é normal sentir que está demorando para “chegar no que interessa” — os fundamentos deste módulo são o alicerce de tudo depois.
- Comparar seu progresso com o de outras pessoas. Lógica de programação é uma habilidade nova para o cérebro, e é normal levar tempo para “clicar”.

Exercício prático

Não há exercício de código neste tópico (ainda não instalamos nada). Em vez disso, responda por escrito, com suas próprias palavras:

1. O que você espera conseguir fazer com dados depois de terminar esta apostila?
2. Pense em um dado do seu dia a dia (gastos do mês, horas de sono, o que for). Que pergunta você gostaria de responder sobre esse dado?

Guarde essas respostas — vamos usar ideias parecidas nos exercícios mais para frente.

Desafio bônus (opcional): pesquise rapidamente três áreas de trabalho que usam Python para análise de dados (ex: mercado financeiro, saúde, marketing) e anote uma frase sobre cada uma.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar com minhas palavras o que é Python e para que serve em análise de dados
- ☐ Entendo que vamos usar a “Loja da Ana” como exemplo contínuo
- ☐ Escrevi minhas respostas do exercício

ewpage

Ambiente de desenvolvimento

Módulo 1 — Fundamentos de Python · Tópico 2 de 25

O que é e por que importa

Para escrever e rodar código Python, você precisa de duas coisas: o **Python instalado** no seu computador (o “motor” que entende e executa o código) e um lugar para **escrever** esse código (um editor de texto). Pense no Python como o motor de um carro e o editor como o painel — você precisa dos dois para dirigir.

Neste tópico você vai instalar o Python e escrever seu primeiro script: um arquivo `.py` com instruções que o computador executa de cima para baixo, linha por linha. É a base de tudo que vem depois — inclusive das ferramentas de análise de dados (pandas, numpy) que você vai usar mais para frente, que nada mais são do que “extensões” que você adiciona a esse mesmo Python.

Mais para frente (Módulo 2) você vai conhecer ambientes mais sofisticados como JupyterLab e Google Colab, que são melhores para explorar dados interativamente. Mas para aprender lógica de programação, um editor simples e um arquivo `.py` são suficientes — e ajudam a entender o que está acontecendo por baixo dos panos antes de usar ferramentas mais “automáticas”.

Como funciona (com exemplo comentado)

Passo a passo:

1. Baixe e instale o Python em python.org/downloads (marque a opção “Add Python to PATH” durante a instalação, no Windows).
2. Instale um editor de código. O mais comum é o [VS Code](#) (gratuito). Depois de instalar, adicione a extensão “Python” dentro dele.
3. Crie uma pasta no seu computador para os exercícios desta apostila, por exemplo `loja-da-ana`.
4. Dentro dessa pasta, crie um arquivo chamado `primeiro_script.py`.
5. Escreva o código abaixo nesse arquivo e rode (no VS Code, botão “Run” ou `Ctrl+F5`; pelo terminal, comando `python primeiro_script.py`).

```
# Este é um comentário — o Python ignora tudo depois do #.  
# Comentários servem para explicar o código para humanos, não para o compu  
  
# print() é a instrução que mostra algo na tela.  
print("Bem-vindo à Loja da Ana!")  
  
# Podemos rodar quantas instruções quisermos, uma por linha.  
print("Sistema de análise de vendas iniciado.")
```

Se aparecer Bem-vindo à Loja da Ana! e Sistema de análise de vendas iniciado. na tela (no “terminal” ou “console”), seu ambiente está funcionando.

Erros comuns de quem está começando

- Instalar o Python mas esquecer de marcar “Add to PATH” no Windows — isso faz o terminal não encontrar o comando `python`. Se isso acontecer, reinstale marcando a opção.
- Salvar o arquivo com extensão errada (ex: `.txt` em vez de `.py`) — o editor não vai reconhecer como código Python.
- Confundir “rodar o arquivo” com “abrir o arquivo”. Abrir só mostra o texto; rodar é o que faz o computador executar as instruções.

Exercício prático

Crie um arquivo chamado `loja_da_ana.py` na sua pasta de exercícios. Nele, use `print()` para mostrar, em linhas separadas:

1. O nome da loja: Loja da Ana
2. Um “slogan” qualquer que você inventar
3. A frase Relatório de vendas - Dia 1

Rode o arquivo e confirme que as três linhas aparecem na tela, na ordem correta.

Desafio bônus (opcional): pesquise como comentar múltiplas linhas de uma vez (sem colocar `#` em cada uma) e adicione um bloco de comentário no topo do arquivo explicando o que o script faz.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Tenho Python instalado e consigo rodar um arquivo `.py`
- ☐ Tenho um editor de código configurado
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo a diferença entre “abrir” e “rodar” um script

ewpage

Variáveis e print()

— Módulo 1 — Fundamentos de Python · Tópico 3 de 25

O que é e por que importa

Uma variável é uma “caixa com nome” onde você guarda um valor para usar depois. Se você já usou uma célula do Excel (tipo B2), a ideia é parecida: você dá um nome a um espaço de memória e guarda algo lá dentro — só que, em vez de B2, você escolhe o nome, tipo `faturamento_do_dia`.

Em análise de dados, quase tudo começa guardando um dado em uma variável: o preço de um produto, o total de vendas, o nome de um cliente. Sem variáveis, você teria que reescrever o valor toda vez que precisasse dele — e não conseguiria fazer contas com valores que mudam.

A função `print()` é a forma mais simples de “mostrar” alguma coisa na tela. Pensa nela como o equivalente a olhar o resultado de uma célula do Excel: você calculou algo, e agora quer ver o resultado. Sem `print()`, o Python calcula tudo “por baixo dos panos”, mas você não vê nada aparecer.

Juntas, variáveis e `print()` são a base de qualquer programa: guardar dados e mostrar resultados.

Como funciona (com exemplo comentado)

```

# Guardando o nome da Loja em uma variável (texto/string)
nome_da_loja = "Loja da Ana"

# Guardando o faturamento do dia em uma variável (número)
faturamento_do_dia = 458.90

# Guardando quantas vendas aconteceram no dia
numero_de_vendas = 5

# print() mostra o valor na tela. Podemos passar vários itens separados por vírgula
print("Loja:", nome_da_loja)
print("Faturamento do dia:", faturamento_do_dia)
print("Número de vendas:", numero_de_vendas)

# Variáveis podem ser reaproveitadas em contas
ticket_medio = faturamento_do_dia / numero_de_vendas
print("Ticket médio:", ticket_medio)

# Uma variável pode ser substituída por um novo valor a qualquer momento
faturamento_do_dia = 500.00
print("Faturamento atualizado:", faturamento_do_dia)

```

Erros comuns de quem está começando

- Usar nomes de variável que não dizem nada (x, a1, coisa). Fica difícil ler o código depois — prefira nomes que descrevem o dado, como `faturamento_do_dia`.
- Esquecer que nomes de variável não podem começar com número nem ter espaço (2vendas ou total vendas dão erro). Use `_` no lugar do espaço: `total_vendas`.
- Achar que `print()` “guarda” o valor. Ele só mostra na tela — se quiser usar o valor depois, precisa estar em uma variável.

Exercício prático

Crie três variáveis para representar uma venda da Loja da Ana: `produto` (o nome do produto vendido), `preco` (o preço unitário) e `quantidade` (quantas unidades foram vendidas). Depois:

1. Calcule o valor total da venda (`preco * quantidade`) e guarde em uma variável `valor_total`.
2. Use `print()` para mostrar uma mensagem parecida com: Venda: 3x Caneta Azul - Total: R\$ 15.00.

Desafio bônus (opcional): crie uma segunda venda (outras três variáveis) e mostre o faturamento somado das duas vendas.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Consigo explicar o que é uma variável com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei usar `print()` para mostrar mais de um valor na mesma linha

ewpage

Inteiros (int)

O que é e por que importa

Inteiros são números sem casa decimal: 1, 2, 100, -5, 0. Em Python, esse tipo se chama `int` (de “integer”). Pensa neles como “coisas que você conta” — quantidade de produtos vendidos, número de clientes, dias do mês. Você nunca diz “vendi 3.5 clientes hoje”.

Em análise de dados, inteiros aparecem toda hora em contagens: quantas vendas aconteceram, quantos itens tem em um pedido, quantos clientes visitaram a loja. Saber que um valor é `int` também ajuda a evitar erros bobos, como tentar dividir uma quantidade de forma que dê resultado quebrado quando não faz sentido (não existe “meio cliente”).

Python identifica sozinho quando um número é `int`, só de você escrever sem ponto decimal. Isso é diferente de outras ferramentas onde você precisa declarar o tipo manualmente.

Como funciona (com exemplo comentado)

```
# Quantidade de produtos vendidos hoje na Loja da Ana (é uma contagem, ent
qtd_vendida = 12

# Número de clientes que passaram pela loja
qtd_clientes = 8

# A função type() mostra qual é o tipo de uma variável
print(type(qtd_vendida)) # <class 'int'>

# Inteiros aceitam contas normalmente
total_itens_por_cliente = qtd_vendida // qtd_clientes # divisão inteira,
print("Média de itens por cliente (arredondada pra baixo):", total_itens_p

# Somar dois inteiros continua sendo inteiro
qtd_vendida_ontem = 9
total_dois_dias = qtd_vendida + qtd_vendida_ontem
print("Total vendido em dois dias:", total_dois_dias)
print(type(total_dois_dias)) # continua int
```

Erros comuns de quem está começando

- Achar que qualquer número é `int`. Se tiver ponto decimal (mesmo 5.0), o tipo já é `float`, não `int` — veja 05-floats.md.
- Usar `int` para representar dinheiro. Preços quase sempre têm centavos, então o tipo certo é `float` (ou, em sistemas profissionais, tipos especiais de dinheiro — mas isso foge do escopo aqui).
- Confundir divisão normal (`/`) com divisão inteira (`//`). `//` descarta a parte decimal do resultado, o que nem sempre é o que você quer.

Exercício prático

A Loja da Ana vendeu, em uma semana, a seguinte quantidade de unidades do produto “Caneta Azul”, um valor por dia: 5, 8, 3, 10, 2, 7, 4.

1. Guarde essas quantidades em sete variáveis (uma por dia) ou some-as diretamente.
2. Calcule o total de canetas vendidas na semana.
3. Calcule quantas canetas, em média, foram vendidas por dia, usando divisão inteira (`//`).
4. Mostre os dois resultados com `print()`.

Desafio bônus (opcional): calcule quantos “pacotes de 6 canetas” dariam para montar com o total vendido na semana, e quantas canetas sobriariam (dica: `//` e o operador de

resto, que vamos formalizar em `08-operadores-aritmeticos.md` — por ora, se quiser, pesquise sobre o operador `%`).

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Consigo explicar o que é um `int` com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo a diferença entre divisão normal e divisão inteira

ewpage

Floats

| Módulo 1 — Fundamentos de Python · Tópico 5 de 25

O que é e por que importa

Float é o tipo de número que tem casa decimal: 5.90, 12.5, 0.33, -1.75. O nome vem de “ponto flutuante” (floating point), uma forma de representar números com parte fracionária. Na prática, pense em floats como “números que você mede”, em vez de “números que você conta” (isso é o `int`, visto em `04-inteiros.md`).

Em análise de dados, floats aparecem em quase todo lugar que envolve dinheiro, médias, porcentagens e medidas: preço de um produto, nota média de avaliação, temperatura, ticket médio de vendas. Se um dado pode ter “meio”, provavelmente é float.

Um detalhe importante (e que confunde muita gente): por causa de como computadores guardam números decimais, contas com float às vezes dão resultados “estranhos”, tipo `0.1 + 0.2` resultando em `0.30000000000000004` em vez de `0.3` certinho. Isso não é um bug do seu código — é uma limitação conhecida de como float funciona em praticamente toda linguagem de programação, e existem formas de contornar (como arredondar o resultado).

Como funciona (com exemplo comentado)

```
# Preço de produtos da Loja da Ana (dinheiro sempre tem casa decimal, então
preco_caneta = 5.00
preco_caderno = 12.50

# type() confirma que são floats
print(type(preco_caneta)) # <class 'float'>

# Contas entre floats resultam em float
total = preco_caneta + preco_caderno
print("Total:", total)

# Misturar int com float também vira float (Python "promove" o resultado)
quantidade = 3 # int
valor_total_canetas = preco_caneta * quantidade
print("Valor total canetas:", valor_total_canetas, "-", type(valor_total_c

# Cuidado com imprecisão de float em contas repetidas
resultado_estranho = 0.1 + 0.2
print(resultado_estranho) # 0.30000000000000004

# A função round() arredonda para um número de casas decimais, contornando
print(round(resultado_estranho, 2)) # 0.3
```

Erros comuns de quem está começando

- Comparar dois floats com `==` esperando exatidão perfeita (ex.: `0.1 + 0.2 == 0.3` dá `False` por causa da imprecisão citada acima). O jeito seguro é arredondar antes de comparar, com `round()`.
- Achar que float sempre mostra as casas decimais que você escreveu. `5.00` pode aparecer como `5.0` na tela — o valor é o mesmo, só a exibição muda.
- Usar float para IDs ou códigos (tipo “código do produto 007”). Se não faz sentido ter meio valor, provavelmente não devia ser float.

Exercício prático

A Loja da Ana vendeu os seguintes produtos em um dia, com seus preços:

- Caneta Azul: R\$ 3.50
- Caderno: R\$ 12.90
- Borracha: R\$ 2.25

1. Guarde os três preços em variáveis float.
2. Calcule o total da compra (soma dos três).
3. Arredonde o total para 2 casas decimais usando `round()` e mostre com `print()`.

Desafio bônus (opcional): calcule quanto seria o total com um desconto de 10% (multiplique o total por `0.9`) e arredonde o resultado.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre `int` e `float` com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo por que contas com float às vezes dão resultados “estranhos” e como o `round()` ajuda

ewpage

Booleanos

Módulo 1 — Fundamentos de Python · Tópico 6 de 25

O que é e por que importa

Booleano é um tipo de dado que só tem dois valores possíveis: `True` (verdadeiro) ou `False` (falso). Em Python o tipo se chama `bool`. Pensa nele como um interruptor de luz: ou está ligado, ou está desligado — não existe meio-termo.

Isso pode parecer um tipo “sem graça” perto de números e textos, mas é a base para tomar decisões no código: “essa venda foi acima de R\$ 100?”, “o cliente é novo?”, “o estoque acabou?”. Toda pergunta que se responde com sim/não vira, no Python, um `True/False`.

Booleanos ganham muita força quando combinados com comparações (`10 > 5` já resulta em um booleano) e com condicionais (`if`), que vamos ver em `09-operadores-de-comparacao.md` e `11-condicionais.md`. Por ora, o importante é entender que é um tipo de dado como qualquer outro — você pode guardar um booleano em uma variável, mostrar com `print()`, e usar em contas.

Como funciona (com exemplo comentado)

```
# Um booleano guardando se uma venda foi feita à vista
venda_a_vista = True

# Um booleano guardando se o estoque de um produto acabou
estoque_zerado = False

print(venda_a_vista)    # True
print(type(venda_a_vista)) # <class 'bool'>

# Comparações já retornam um booleano diretamente
faturamento_do_dia = 458.90
meta_do_dia = 400.00

bateu_meta = faturamento_do_dia > meta_do_dia
print("Bateu a meta?", bateu_meta) # True

# Curiosidade: internamente, True equivale a 1 e False equivale a 0
print(True + True)    # 2
print(bateu_meta * 10) # 10, porque True vale 1
```

Erros comuns de quem está começando

- Escrever `"True"` (como texto) em vez de `True` (o valor booleano de verdade). São coisas diferentes: uma é string, outra é `bool`.
- Comparar um booleano com `== True` (ex: `if bateu_meta == True:`), quando basta usar `if bateu_meta:` — é redundante e menos legível.
- Achar que booleano é só usado dentro de `if`. Ele é um valor comum, pode ser guardado em variável, em lista, retornado por uma função etc.

Exercício prático

A Loja da Ana teve, em um dia, faturamento de R\$ 620.00, e a meta do dia era R\$ 500.00. Além disso, o número de clientes atendidos foi 15, e a meta de clientes era 20.

1. Crie uma variável booleana `bateu_meta_faturamento` guardando se o faturamento passou da meta.
2. Crie uma variável booleana `bateu_meta_clientes` guardando se o número de clientes passou da meta.
3. Mostre as duas com `print()`, com uma mensagem explicando o que cada uma

representa.

Desafio bônus (opcional): conte quantas das duas metas foram batidas, somando os dois booleanos (lembre que `True` vale 1 e `False` vale 0).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que é um booleano com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo que uma comparação (como `>`) já produz um valor booleano

ewpage

Conversão de tipos (type casting)

— Módulo 1 — Fundamentos de Python · Tópico 7 de 25

O que é e por que importa

Conversão de tipos (em inglês, “type casting”) é o processo de transformar um valor de um tipo em outro — por exemplo, transformar o texto “25” no número 25. Pense nisso como converter uma medida: 1 metro e 25 centímetros escrito por extenso vira o número 1.25 quando você precisa fazer conta com ele.

Isso importa demais em análise de dados porque, na vida real, dados quase nunca chegam já no tipo certo. Um formulário preenchido por um cliente pode salvar a idade como texto (“32”), uma planilha pode exportar preços como texto (“19.90”), e você precisa converter esses valores para número antes de somar, comparar ou calcular qualquer coisa.

Em Python, você já viu os tipos `int` (04-inteiros.md), `float` (05-floats.md) e `bool` (06-booleanos.md), além de `str` (texto). As funções `int()`, `float()`, `str()` e `bool()` fazem a conversão entre eles. Sem esse passo, tentar somar um número com um texto trava o programa com um erro.

Como funciona (com exemplo comentado)

```

# Simulando um dado que "chegou" como texto (ex: de um formulário preenchido)
quantidade_texto = "10"
preco_texto = "5.50"

# Tentar somar direto daria erro, porque são strings, não números.
# Por isso convertamos primeiro:
quantidade = int(quantidade_texto) # texto -> inteiro
preco = float(preco_texto)          # texto -> float

total = quantidade * preco
print("Total da venda:", total) # 55.0

# O caminho inverso também é comum: transformar número em texto para montar
mensagem = "O total foi de R$ " + str(total)
print(mensagem)

# int() também "trunca" (corta) a parte decimal de um float, sem arredonda
avaliacao_media = 4.8
avaliacao_inteira = int(avaliacao_media)
print(avaliacao_inteira) # 4, não 5 -- int() corta, não arredonda

# bool() converte outros tipos considerando "vazio/zero" como False
print(bool(0))          # False
print(bool(150))        # True
print(bool(""))         # False (texto vazio)
print(bool("Ana"))      # True (texto não vazio)

```

Erros comuns de quem está começando

- Tentar fazer conta direto com números que na verdade são texto (ex: "10" + "5" resulta em "105", concatenação de texto, não soma). É preciso converter com `int()` ou `float()` antes.
- Usar `int()` em um texto que não é um número puro, como `int("10 unidades")` — isso gera erro. `int()` só converte textos que sejam exatamente um número.
- Achar que `int()` arredonda floats. Na verdade ele trunca (corta a casa decimal), então `int(4.9)` vira 4, não 5. Para arredondar de verdade, use `round()`.

Exercício prático

Imagine que a Loja da Ana recebeu os seguintes dados de uma venda, digitados por um sistema que salva tudo como texto:

```

quantidade_texto = "4"
preco_unitario_texto = "7.25"

```

1. Converta as duas variáveis para os tipos numéricos corretos.
2. Calcule o valor total da venda.
3. Monte e mostre uma mensagem de texto, tipo: "Total da venda: R\$ 29.0", usando `str()` para juntar texto com número.

Desafio bônus (opcional): o sistema também informou `em_promocao_texto = "1"` (1 significa sim, 0 significa não). Converta esse valor para `bool` usando o que você aprendeu, e mostre se a venda foi de um produto em promoção. Dica: primeiro converta para `int`, depois para `bool`.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que é conversão de tipos com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo a diferença entre `int()` truncar e `round()` arredondar

ewpage

Operadores aritméticos

Módulo 1 — Fundamentos de Python · Tópico 8 de 25

O que é e por que importa

Operadores aritméticos são os símbolos que fazem contas: + (soma), - (subtração), * (multiplicação), / (divisão), // (divisão inteira), % (resto da divisão) e ** (potência). Se você já usou fórmulas no Excel, é exatamente a mesma ideia, só que escrita direto no código em vez de dentro de uma célula.

Em análise de dados, esses operadores são o motor de praticamente todo cálculo que você vai fazer: somar vendas, calcular o total de uma compra (preço vezes quantidade), achar a média (soma dividida pela quantidade de itens), calcular crescimento percentual. Você já usou alguns deles nos tópicos anteriores sem nomeá-los formalmente — agora vamos consolidar todos.

Dois operadores merecem atenção especial por serem menos intuitivos: // (divisão inteira, que descarta a parte decimal) e % (o resto que sobra de uma divisão). Eles parecem estranhos no começo, mas são extremamente úteis para tarefas como “descobrir se um número é par” ou “dividir um total em grupos iguais e saber quanto sobra”.

Como funciona (com exemplo comentado)

```
faturamento_semana = 1250.00
dias_da_semana = 7

# Soma, subtração e multiplicação são diretas
meta_semanal = 1000.00
diferenca_para_meta = faturamento_semana - meta_semanal
print("Quanto passou da meta:", diferenca_para_meta)

# Divisão normal (/) sempre retorna float, mesmo dividindo dois inteiros
media_diaria = faturamento_semana / dias_da_semana
print("Média diária:", round(media_diaria, 2))

# Divisão inteira (//) descarta a parte decimal do resultado
qtd_vendida = 50
tamanho_da_caixa = 8
caixas_completas = qtd_vendida // tamanho_da_caixa
print("Caixas completas:", caixas_completas) # 6

# % (resto/módulo) mostra o que sobra depois da divisão inteira
itens_avulsos = qtd_vendida % tamanho_da_caixa
print("Itens que sobram fora de caixa:", itens_avulsos) # 2

# ** eleva a um expoente -- útil para juros compostos, crescimento etc.
crescimento_anual = 1.10 # 10% de crescimento ao ano
faturamento_projetado_2_anos = faturamento_semana * (crescimento_anual ** 2)
print("Faturamento projetado em 2 anos:", round(faturamento_projetado_2_anos, 2))

# % também é o truque clássico para saber se um número é par
numero_de_vendas = 15
if numero_de_vendas % 2 == 0:
    print("Número par de vendas")
else:
    print("Número ímpar de vendas")
```

Erros comuns de quem está começando

- Confundir / com //. / sempre dá o resultado “certo” com decimais; // corta a parte decimal, mesmo quando isso não faz sentido para o problema.
- Esquecer a ordem das operações (precedência): multiplicação e divisão são calculadas antes de soma e subtração, igual na matemática da escola. Use parênteses () para deixar claro, mesmo quando não é obrigatório — ajuda a evitar erro de leitura.
- Achar que % é “porcentagem”. Em Python, % dentro de uma expressão numérica é o operador de resto da divisão, não tem relação direta com porcentagem (para porcentagem, você multiplica por 0.10, por exemplo, como no exemplo acima).

Exercício prático

A Loja da Ana tem 37 unidades de um produto em estoque e quer organizá-las em caixas de 5 unidades cada, para transporte.

1. Calcule quantas caixas completas dá para montar.
2. Calcule quantas unidades sobram, fora de caixa.
3. Mostre os dois resultados com `print()`.

Desafio bônus (opcional): o faturamento da loja neste mês foi R\$ 8400.00 e o objetivo é crescer 15% ao mês. Calcule qual seria o faturamento projetado daqui a 3 meses, usando o operador `**`.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre /, // e % com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei usar ** para elevar um número a uma potência

ewpage

Operadores de comparação

Módulo 1 — Fundamentos de Python · Tópico 9 de 25

O que é e por que importa

Operadores de comparação são os símbolos que comparam dois valores e respondem com um booleano (True ou False, vistos em 06-booleans.md): == (igual), != (diferente), > (maior), < (menor), >= (maior ou igual), <= (menor ou igual). É a forma de o código responder perguntas do tipo “esse valor é maior que aquele?”.

Em análise de dados, comparações são a base para filtrar e classificar informação: “quais vendas foram acima de R\$ 100?”, “esse cliente é o mesmo que já comprou antes?”, “o estoque está abaixo do mínimo?”. Sem comparação, não dá para separar dados “bons” de dados “ruins”, nem tomar decisão nenhuma no código.

Um erro clássico de quem está começando (em quase todas as linguagens, não só Python) é confundir = com ==. O = (um sinal só) guarda um valor em uma variável, ensinado em 03-variaveis-e-print.md. O == (dois sinais) compara dois valores e diz se são iguais. São operações completamente diferentes, apesar de parecidas visualmente.

Como funciona (com exemplo comentado)

```
preco_produto_a = 25.90
preco_produto_b = 18.50
meta_de_venda = 1000.00
faturamento_do_dia = 950.00

# == compara igualdade (não confundir com = de atribuição)
mesmo_preco = preco_produto_a == preco_produto_b
print("Produtos com mesmo preço?", mesmo_preco) # False

# != verifica diferença
precos_diferentes = preco_produto_a != preco_produto_b
print("Preços diferentes?", precos_diferentes) # True

# > e < comparam grandeza
produto_a_mais_caro = preco_produto_a > preco_produto_b
print("Produto A é mais caro?", produto_a_mais_caro) # True

# >= e <= incluem o caso de igualdade
bateu_a_meta = faturamento_do_dia >= meta_de_venda
print("Bateu a meta (ou empatou)?", bateu_a_meta) # False

# Comparações também funcionam com texto (ordem alfabética) e são muito us
cliente_1 = "Ana"
cliente_2 = "Bruno"
print(cliente_1 < cliente_2) # True, "Ana" vem antes de "Bruno" no alfabe
```

Erros comuns de quem está começando

- Usar = (atribuição) quando o objetivo era == (comparação). `if faturamento = 100:` dá erro de sintaxe — o certo é `if faturamento == 100:`.
- Comparar float com == esperando exatidão perfeita, como visto em `05-floats.md` (`0.1 + 0.2 == 0.3` dá `False`). Para floats, é mais seguro comparar com uma margem de tolerância ou arredondar antes.
- Esquecer que comparação de texto ("`Ana`" < "`Bruno`") é sensível a maiúsculas/minúsculas — "`ana`" < "`Bruno`" pode não se comportar como esperado, porque letras maiúsculas e minúsculas têm códigos diferentes internamente.

Exercício prático

A Loja da Ana define que uma venda é “grande” quando o valor total é maior ou igual a R\$ 150.00. Considere as vendas: 89.90, 150.00, 210.50, 45.00.

1. Para cada valor, use um operador de comparação para verificar se a venda é “grande” (maior ou igual a 150.00) e guarde o resultado em uma variável booleana.
2. Mostre, para cada venda, o valor e se ela é grande ou não, usando `print()`.

Desafio bônus (opcional): compare o total de vendas do dia (some as quatro) com a meta diária de R\$ 400.00, e mostre se a meta foi batida.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre = e == com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei usar >= e <= corretamente

ewpage

Operadores lógicos

| Módulo 1 — Fundamentos de Python · Tópico 10 de 25

O que é e por que importa

Operadores lógicos combinam duas ou mais condições booleanas (vistas em `06-booleanos.md` e `09-operadores-de-comparacao.md`) em uma única resposta verdadeiro/falso. São três: `and` (e — todas as condições precisam ser verdadeiras), `or` (ou — basta uma condição ser verdadeira) e `not` (nega o valor: transforma `True` em `False` e vice-versa).

Pensa em `and` como uma exigência de lista: “só aprovo o empréstimo se a pessoa tiver renda comprovada E não tiver nome sujo” — as duas coisas precisam ser verdade. Já o `or` é uma alternativa: “aceito pagamento em dinheiro OU cartão” — qualquer um dos dois já resolve.

Em análise de dados, operadores lógicos são essenciais para filtros compostos: “vendas acima de R\$ 100 E feitas em dinheiro”, “clientes que compraram no último mês OU que gastaram mais de R\$ 500 no total”. Praticamente todo filtro “de verdade” em dados combina mais de uma condição, e é aí que `and`, `or` e `not` entram.

Como funciona (com exemplo comentado)

```

valor_venda = 180.00
forma_pagamento = "dinheiro"
cliente_fidelidade = True

# and: as duas condições precisam ser verdadeiras
venda_grande_em_dinheiro = valor_venda > 150 and forma_pagamento == "dinheiro"
print("Venda grande paga em dinheiro?", venda_grande_em_dinheiro) # True

# or: basta uma condição ser verdadeira
merece_desconto = valor_venda > 200 or cliente_fidelidade
print("Merece desconto?", merece_desconto) # True, porque cliente_fidelid

# not: inverte o valor booleano
nao_e_fidelidade = not cliente_fidelidade
print("Cliente não é fidelidade?", nao_e_fidelidade) # False

# Combinando os três operadores (parênteses ajudam a deixar a ordem clara)
promocao_valida = (valor_venda >= 100 and forma_pagamento == "dinheiro") or
print("Promoção válida?", promocao_valida)

# and e or "curto-circuitam": param assim que já sabem a resposta.
# Em "False and ...", o Python nem avalia o resto, porque já sabe que é Fa
estoque = 0
tem_estoque_e_preco_valido = estoque > 0 and (100 / estoque > 5) # não qu
print("Tem estoque e preço válido?", tem_estoque_e_preco_valido)

```

Erros comuns de quem está começando

- Usar & e | no lugar de and e or. Esses símbolos existem em Python, mas têm outro significado (operações bit a bit) e não devem ser usados aqui neste contexto — o correto para lógica booleana comum é escrever a palavra and/or.
- Esquecer parênteses em expressões com and e or misturados, o que pode gerar um resultado diferente do esperado. Quando tiver dúvida, agrupe com () para deixar a intenção clara.
- Escrever not (x == y) quando dava para simplificar com x != y, deixando o código mais difícil de ler sem necessidade.

Exercício prático

A Loja da Ana quer identificar vendas que merecem um cupom de desconto na próxima compra. A regra é: o cupom é dado se a venda foi maior que R\$ 200 e o pagamento foi em cartão, **ou** se o cliente já é fidelidade (independente do valor).

Considere os dados de uma venda:

```

valor_venda = 250.00
forma_pagamento = "cartao"
cliente_fidelidade = False

```

1. Escreva uma expressão lógica que combine as três informações seguindo a regra acima.
2. Guarde o resultado em uma variável merece_cupom e mostre com print().
3. Repita o exercício com valor_venda = 90.00, forma_pagamento = "dinheiro", cliente_fidelidade = True.

Desafio bônus (opcional): adicione a regra “e não é a primeira compra da pessoa” (primeira_compra = False para valer o desconto) combinando com not.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre `and` e `or` com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei usar `not` para inverter uma condição

ewpage

Condicionais (if/elif/else)

| Módulo 1 — Fundamentos de Python · Tópico 11 de 25

O que é e por que importa

Condicional é a estrutura que faz o código tomar decisões: “se isso for verdade, faça aquilo; senão, faça outra coisa”. Em Python, isso se escreve com `if` (se), `elif` (senão-se, contração de “else if”) e `else` (senão). Se você já usou a função `SE()` do Excel, a ideia é idêntica — só que em Python dá para encadear quantas condições quiser, de forma mais clara.

Todos os tópicos anteriores — tipos de dado, operadores de comparação (`09-operadores-de-comparacao.md`) e operadores lógicos (`10-operadores-logicos.md`) — existiam, em parte, para chegar até aqui: eles produzem os `True/False` que o `if` usa para decidir qual caminho o código deve seguir.

Em análise de dados, condicionais aparecem para classificar informação: “essa venda é grande ou pequena?”, “esse cliente é novo ou recorrente?”, “essa nota é aprovação ou reprovação?”. É a ferramenta que transforma uma regra de negócio (escrita em português) em uma decisão automática que o código toma sozinho para cada dado que ele processa.

Como funciona (com exemplo comentado)

```

valor_venda = 175.00

# if sozinho: só executa o bloco de baixo se a condição for True
if valor_venda > 100:
    print("Venda considerada grande")

# if/else: um caminho para True, outro para False
if valor_venda > 100:
    print("Venda grande")
else:
    print("Venda pequena")

# if/elif/else: várias faixas de classificação, verificadas em ordem
if valor_venda >= 200:
    categoria = "grande"
elif valor_venda >= 100:
    categoria = "media"
else:
    categoria = "pequena"

print("Categoria da venda:", categoria)

# Repare que a indentação (espaços no início da linha) define o que pertencem
# Isso não é estético -- em Python, indentação é parte da sintaxe da linguagem
forma_pagamento = "cartao"
cliente_fidelidade = True

if valor_venda > 150 and forma_pagamento == "cartao":
    print("Cliente ganha 5% de desconto")
elif cliente_fidelidade:
    print("Cliente ganha 3% de desconto por fidelidade")
else:
    print("Sem desconto")

```

Erros comuns de quem está começando

- Esquecer os dois-pontos (:) no final da linha do if/elif/else. Python exige isso para saber onde o bloco começa.
- Bagunçar a indentação (misturar espaços de forma inconsistente). Cada bloco dentro de um if precisa estar alinhado da mesma forma — o recomendado é sempre 4 espaços por nível.
- Usar vários if separados quando o certo seria elif. Com if separados, o Python testa todas as condições, mesmo que a primeira já tenha sido True — com elif, ele para no primeiro caminho verdadeiro, o que costuma ser o comportamento esperado e mais eficiente.

Exercício prático

A Loja da Ana quer classificar suas vendas em três categorias:

- “grande”: valor maior ou igual a R\$ 200
- “média”: valor entre R\$ 50 (inclusive) e R\$ 200 (exclusive)
- “pequena”: valor menor que R\$ 50

Para as vendas 35.00, 89.90 e 300.00:

1. Para cada uma, use if/elif/else para descobrir a categoria.
2. Mostre com print() o valor da venda e sua categoria.

Desafio bônus (opcional): adicione a regra de que, se a venda for “grande” e o pagamento for em dinheiro, a loja dá 5% de desconto — mostre também se o desconto se aplica.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre `if`, `elif` e `else` com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo por que a indentação é obrigatória em Python

ewpage

Loops (for/while)

| Módulo 1 — Fundamentos de Python · Tópico 12 de 25

O que é e por que importa

Loop é uma estrutura que repete um bloco de código várias vezes, sem você precisar copiar e colar a mesma instrução. Pensa em quando você arrasta uma fórmula do Excel para baixo, aplicando o mesmo cálculo em cada linha da planilha — um loop faz isso, só que de forma explícita no código.

Python tem dois tipos principais: `for`, que repete um número definido de vezes (por exemplo, “uma vez para cada venda na lista de vendas”), e `while`, que repete enquanto uma condição continuar sendo verdadeira (por exemplo, “enquanto o estoque não tiver acabado”).

Loops são um dos motivos pelos quais programação é tão poderosa para análise de dados: em vez de calcular manualmente o total de 500 vendas uma por uma, você escreve a regra de cálculo uma vez e deixa o `for` aplicar essa regra a cada item automaticamente. Neste tópico, ainda vamos usar loops com dados simples (sequências de números); em `16-listas.md` você vai ver como combinar `for` com listas de verdade, o que é onde loops realmente brilham.

Como funciona (com exemplo comentado)

```

# for com range(): repete um número definido de vezes
# range(5) gera os números 0, 1, 2, 3, 4 (começa em 0, não inclui o 5)
for dia in range(5):
    print("Processando dados do dia", dia)

# range(inicio, fim) permite escolher onde começar
for dia in range(1, 6): # 1, 2, 3, 4, 5
    print("Dia", dia, "de vendas")

# Loops são ótimos para acumular um resultado, como somar vendas
faturamento_total = 0
vendas_do_dia = 150 # imaginando o mesmo valor todo dia, por simplicidade
for dia in range(1, 8):
    faturamento_total = faturamento_total + vendas_do_dia
print("Faturamento estimado da semana:", faturamento_total)

# while: repete enquanto a condição for True
estoque = 5
vendas_feitas = 0
while estoque > 0:
    print("Vendendo 1 unidade. Estoque restante:", estoque - 1)
    estoque = estoque - 1
    vendas_feitas = vendas_feitas + 1

print("Total de vendas até esgotar o estoque:", vendas_feitas)

# Cuidado: while precisa de uma condição que eventualmente vire False,
# senão o loop nunca para (loop infinito) -- por isso sempre atualizamos
# a variável "estoque" dentro do próprio loop.

```

Erros comuns de quem está começando

- Esquecer de atualizar a variável de controle dentro do while, criando um loop infinito que nunca termina (o programa “trava”).
- Achar que range(5) começa em 1. Ele começa em 0 e não inclui o número final — range(5) gera 0, 1, 2, 3, 4 (cinco números no total).
- Usar while quando for resolveria de forma mais simples. Se você já sabe quantas vezes quer repetir, for costuma ser a escolha mais clara; while é melhor quando a repetição depende de uma condição que só se resolve durante a execução.

Exercício prático

A Loja da Ana quer saber quanto teria faturado ao longo de 6 dias, sabendo que o faturamento cresce R\$ 20,00 por dia a partir de uma base de R\$ 100,00 no primeiro dia (dia 1: R\$ 100, dia 2: R\$ 120, dia 3: R\$ 140, e assim por diante).

1. Use um for com range() para simular os 6 dias.
2. A cada dia, calcule o faturamento daquele dia e some em uma variável faturamento_total.
3. Ao final, mostre o faturamento total da semana.

Desafio bônus (opcional): usando while, simule quantos dias são necessários para o faturamento acumulado ultrapassar R\$ 900,00, mantendo a mesma regra de crescimento.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre for e while com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução

❑ Entendo por que um `while` mal escrito pode virar um loop infinito

ewpage

Strings (fundamentos)

Módulo 1 — Fundamentos de Python · Tópico 13 de 25

O que é e por que importa

String é o tipo de dado usado para texto: nomes, endereços, descrições de produtos, categorias. Em Python, o tipo se chama `str`, e você já usou strings desde o primeiro tópico, sempre que escreveu algo entre aspas, como "Loja da Ana".

Em análise de dados, uma parte enorme do trabalho é com texto: nome de clientes, nome de produtos, categorias de venda, respostas abertas de pesquisas. Diferente de números, texto tem particularidades próprias — você pode juntar duas strings, pegar um pedaço específico dela, contar quantos caracteres tem, e muito mais (isso será aprofundado em 14-metodos-de-string.md e 15-strip-replace-split.md).

Uma ideia importante deste tópico é a de “índice”: cada caractere de uma string tem uma posição numerada, começando em 0 (igual ao `range()` visto em 12-loops.md). Isso permite pegar pedaços específicos do texto, como as três primeiras letras de um nome.

Como funciona (com exemplo comentado)

```
nome_produto = "Caneta Azul"

# Strings podem usar aspas simples ou duplas -- tanto faz, desde que combi
categoria = 'Papelaria'

# Concatenação: juntar strings com +
descricao = nome_produto + " - " + categoria
print(descricao) # Caneta Azul - Papelaria

# Len() mede o comprimento de uma string (quantos caracteres tem)
tamanho_nome = len(nome_produto)
print("Tamanho do nome do produto:", tamanho_nome)

# Indexação: cada caractere tem uma posição, começando em 0
primeira_letra = nome_produto[0]
print("Primeira letra:", primeira_letra) # C

# Índices negativos contam a partir do final (-1 é o último caractere)
ultima_letra = nome_produto[-1]
print("Última letra:", ultima_letra) # L

# Fatiamento (slicing): pegar um pedaço da string, do índice inicial (incl
# até o final (excluso)
primeiras_6_letras = nome_produto[0:6]
print("Primeiras 6 letras:", primeiras_6_letras) # Caneta

# f-strings: forma moderna e prática de montar mensagens com valores dentr
preco = 5.90
mensagem = f"O produto {nome_produto} custa R$ {preco}"
print(mensagem)
```

Erros comuns de quem está começando

- Esquecer que índices começam em 0, não em 1. `nome_produto[1]` pega o segundo caractere, não o primeiro.

- Tentar concatenar string com número diretamente (`"Total: " + 10`), o que gera erro. É preciso converter o número para string primeiro com `str()` (visto em `07-conversao-de-tipos.md`), ou usar f-string, que faz essa conversão automaticamente.
- Achar que fatiamento (`[0:6]`) inclui o índice final. `nome_produto[0:6]` pega os caracteres nas posições 0 a 5 — a posição 6 fica de fora.

Exercício prático

A Loja da Ana tem um produto chamado "Caderno Universitário 96 folhas".

1. Descubra e mostre quantos caracteres tem esse nome, usando `len()`.
2. Mostre apenas a palavra "Caderno" usando fatiamento (`[0:7]`).
3. Monte e mostre uma f-string com a frase: "O produto Caderno Universitário 96 folhas tem 32 caracteres" (usando os valores calculados, não digitando o número na mão).

Desafio bônus (opcional): mostre o último caractere do nome do produto usando índice negativo, e explique (num comentário) por que ele é um espaço em branco ou uma letra, dependendo de como você escreveu a string.

Solução comentada

- Clique para ver a solução

Checklist antes de avançar

- ☐ Consigo explicar o que é uma string e um índice com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei usar fatiamento (`[inicio:fim]`) para pegar um pedaço de um texto

ewpage

Trabalhando com strings (métodos gerais)

| Módulo 1 — Fundamentos de Python · Tópico 14 de 25

O que é e por que importa

Método é uma “ação” que um valor sabe executar em si mesmo. A sintaxe é `valor.metodo()` — por exemplo, `"ana".upper()` transforma o texto em maiúsculas. Pensa em método como um “botão” que já vem embutido no próprio dado: toda string em Python já nasce com um conjunto de botões prontos para maiúsculas, minúsculas, busca de palavras etc. (a diferença formal entre “função” e “método” será explicada com calma em `21-funcoes-e-metodos.md` — por ora, use método como “uma função que pertence a um valor”).

Em análise de dados, texto quase nunca chega “limpo”: nomes de clientes com maiúsculas trocadas, categorias de produto escritas de formas diferentes (“papellaria”, “Papellaria”, “PAPELLARIA”), espaços a mais. Métodos de string são a ferramenta para padronizar e investigar esse texto antes de usá-lo em qualquer análise.

Este tópico cobre os métodos mais gerais de string. Os métodos de “limpeza e separação” mais usados em dados (`strip`, `replace`, `split`) ganham um tópico próprio, `15-strip-replace-split.md`, porque merecem atenção especial.

Como funciona (com exemplo comentado)

```

nome_cliente = "  ana Paula Da Silva  "

# upper() deixa tudo maiúsculo, lower() deixa tudo minúsculo
print(nome_cliente.upper()) # "  ANA PAULA DA SILVA  "
print(nome_cliente.lower()) # "  ana paula da silva  "

# title() deixa a primeira letra de cada palavra maiúscula (bom para nomes)
print(nome_cliente.title()) # "  Ana Paula Da Silva  "

# Métodos não alteram a string original -- eles retornam uma nova string
print(nome_cliente) # continua com espaços e caixa original

# Padronizar texto para comparação é um uso comum em análise de dados
categoria_1 = "Papeleria"
categoria_2 = "PAPELARIA"
sao_a_mesma_categoria = categoria_1.lower() == categoria_2.lower()
print("São a mesma categoria?", sao_a_mesma_categoria) # True

# in verifica se um texto contém outro pedaço de texto
descricao_produto = "Caneta esferográfica azul, ponta fina"
tem_palavra_azul = "azul" in descricao_produto
print("Descrição menciona 'azul'?", tem_palavra_azul) # True

# startswith() e endswith() checam início e fim do texto
codigo_produto = "PAP-001"
eh_da_papelaria = codigo_produto.startswith("PAP")
print("Código é da categoria papelaria?", eh_da_papelaria) # True

# count() conta quantas vezes um pedaço aparece no texto
qtd_letra_a = nome_cliente.lower().count("a")
print("Quantidade de 'a' no nome:", qtd_letra_a)

```

Erros comuns de quem está começando

- Achar que `.upper()` (ou qualquer método de string) muda a variável original. Strings em Python são “imutáveis” — o método sempre devolve uma string nova, então é preciso guardar o resultado em uma variável (`nome = nome.upper()`) se quiser manter a mudança.
- Comparar textos sem padronizar maiúsculas/minúsculas antes, e concluir erroneamente que “Papeleria” e “papelaria” são categorias diferentes.
- Esquecer o parêntese ao chamar um método, escrevendo `nome_cliente.upper` em vez de `nome_cliente.upper()`. Sem os parênteses, o Python não executa o método, apenas referencia ele.

Exercício prático

A Loja da Ana recebeu os seguintes nomes de clientes de um cadastro malfeito, com maiúsculas inconsistentes:

```
clientes = ["ANA SILVA", "bruno costa", "Carla Dias"]
```

(Vamos usar listas de verdade em breve, em 16-listas.md — por ora, trate cada nome como uma string separada.)

1. Usando os três nomes acima como variáveis separadas (`cliente_1`, `cliente_2`, `cliente_3`), padronize todos para o formato “Nome Sobrenome” (primeira letra maiúscula) usando `.title()`.
2. Verifique, para cada nome padronizado, se ele contém a palavra “Silva” usando o operador `in`.

Desafio bônus (opcional): conte quantas vezes a letra “a” aparece em cada nome (ignorando maiúsculas/minúsculas).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que é um método (`valor.metodo()`) com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo que métodos de string retornam um valor novo, sem alterar o original

ewpage

strip, replace, split

| Módulo 1 — Fundamentos de Python · Tópico 15 de 25

O que é e por que importa

Continuando de `14-metodos-de-string.md`, este tópico foca em três métodos de string que você vai usar o tempo todo ao trabalhar com dados reais: `strip()` (remove espaços em branco do início/fim), `replace()` (troca um pedaço de texto por outro) e `split()` (quebra um texto em pedaços, formando uma lista).

Dados de vendas, formulários e planilhas raramente chegam “limpos”. É comum ter espaços a mais digitados sem querer ("Ana Silva "), separadores misturados ("produto;preco;quantidade") ou textos que precisam de pequenas correções (trocar "R\$" por nada, para sobrar só o número). Esses três métodos são o kit básico de limpeza de texto — o primeiro passo de qualquer análise de dados de verdade é “limpar” os dados antes de calcular qualquer coisa.

`split()` merece destaque especial porque devolve uma lista (que veremos formalmente em `16-listas.md`) — é o método que transforma uma linha de texto separada por vírgulas (como um arquivo CSV) em pedaços individuais que dá para processar um a um.

Como funciona (com exemplo comentado)

```

# strip(): remove espaços (ou quebras de linha) do início e do fim do texto
nome_com_espacos = "   Ana Silva   "
nome_limpo = nome_com_espacos.strip()
print(f'{nome_limpo}') # 'Ana Silva', sem os espaços nas pontas

# strip() não mexe em espaços no meio do texto
print(f'{nome_limpo}') # o espaço entre "Ana" e "Silva" continua lá

# replace(): troca todas as ocorrências de um pedaço de texto por outro
preco_texto = "R$ 25,90"
preco_sem_simbolo = preco_texto.replace("R$ ", "")
print(preco_sem_simbolo) # "25,90"

# Um caso muito comum em dados brasileiros: trocar vírgula decimal por ponto
# antes de converter para float (visto em 07-conversao-de-tipos.md)
preco_com_ponto = preco_sem_simbolo.replace(",", ".")
preco_numero = float(preco_com_ponto)
print(preco_numero) # 25.9

# split(): quebra uma string em uma lista de pedaços, usando um separador
linha_csv = "Caneta Azul,5.90,10"
partes = linha_csv.split(",")
print(partes) # ['Caneta Azul', '5.90', '10']

# Cada pedaço pode ser acessado por índice (igual visto em 13-strings-fund)
nome_produto = partes[0]
preco = float(partes[1])
quantidade = int(partes[2])
print(nome_produto, preco, quantidade)

# split() sem argumento quebra por espaços em branco (útil para separar pa
frase = "Caneta esferográfica azul ponta fina"
palavras = frase.split()
print(palavras) # ['Caneta', 'esferográfica', 'azul', 'ponta', 'fina']

```

Erros comuns de quem está começando

- Esquecer que `strip()` só remove espaços das pontas, não do meio do texto. Para remover espaços do meio, seria preciso usar `replace(" ", "")`, mas isso raramente é o que você quer (destruiria a separação entre palavras).
- Tentar converter direto para `float` um texto com vírgula decimal (`float("25,90")`), o que gera erro em Python — antes é preciso trocar a vírgula por ponto com `replace(",", ".")`.
- Esquecer que `split()` devolve uma lista, e tentar tratar o resultado como se fosse ainda uma string única.

Exercício prático

A Loja da Ana recebeu a seguinte linha de um sistema antigo, representando uma venda:

```
linha = "   Caderno Universitário , 12,90 , 3   "
```

1. Use `split(",")` para separar a linha em três partes: nome do produto, preço e quantidade.
2. Para cada parte, use `strip()` para remover espaços indesejados.
3. Converta o preço (trocando vírgula por ponto com `replace()`) e a quantidade para os tipos numéricos corretos.
4. Calcule e mostre o valor total da venda (preço $\times$ quantidade).

Desafio bônus (opcional): monte uma mensagem final usando f-string, do tipo: "Venda de 3x Caderno Universitário - Total: R\$ 38.7".

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre `strip()`, `replace()` e `split()` com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo por que separador de campo e vírgula decimal podem conflitar em dados brasileiros

ewpage

Listas

| Módulo 1 — Fundamentos de Python · Tópico 16 de 25

O que é e por que importa

Lista é uma coleção ordenada de valores, guardada em uma única variável. Pensa nela como uma coluna inteira de uma planilha, em vez de uma célula só: em vez de ter `venda_1`, `venda_2`, `venda_3` como variáveis separadas (como fizemos em tópicos anteriores), você guarda tudo em uma única lista, `vendas = [45.90, 120.00, 15.50]`.

Isso muda tudo na prática: com uma lista, você pode usar um `for` (visto em `12-loops.md`) para processar cada valor automaticamente, não importa se são 3 vendas ou 3 mil. É exatamente assim que análise de dados funciona de verdade — você quase nunca sabe de antemão quantos registros vai ter, então precisa de uma estrutura que cresça e seja percorrida de forma genérica.

Listas em Python podem guardar qualquer tipo de dado (números, textos, booleanos, até outras listas), e são “mutáveis” — ou seja, dá para adicionar, remover e alterar itens depois de criadas.

Como funciona (com exemplo comentado)

```

# Lista com o valor de cada venda do dia, na Loja da Ana
vendas_do_dia = [45.90, 120.00, 15.50, 200.00, 33.00]

# Len() mede quantos itens tem na lista
qtd_vendas = len(vendas_do_dia)
print("Número de vendas:", qtd_vendas)

# Acesso por índice, igual em strings (começa em 0)
primeira_venda = vendas_do_dia[0]
ultima_venda = vendas_do_dia[-1]
print("Primeira venda:", primeira_venda, "- Última venda:", ultima_venda)

# Percorrendo a lista inteira com for
for venda in vendas_do_dia:
    print("Processando venda de R$", venda)

# Funções prontas que funcionam direto em listas de números
print("Total do dia:", sum(vendas_do_dia))
print("Maior venda:", max(vendas_do_dia))
print("Menor venda:", min(vendas_do_dia))

# Adicionando um novo item ao final da lista
vendas_do_dia.append(89.00)
print("Depois de adicionar uma venda:", vendas_do_dia)

# Removendo um item específico
vendas_do_dia.remove(15.50)
print("Depois de remover a venda de 15.50:", vendas_do_dia)

# Fatiamento funciona igual em strings: pega um pedaço da lista
tres_primeiras = vendas_do_dia[0:3]
print("Três primeiras vendas:", tres_primeiras)

# Listas podem guardar qualquer tipo, inclusive misturado
produto = ["Caneta Azul", 5.90, 10, True] # nome, preço, quantidade, ativ
print(produto)

```

Erros comuns de quem está começando

- Tentar acessar um índice que não existe (`vendas_do_dia[10]` em uma lista de 5 itens), o que gera erro `IndexError`. Sempre confira o tamanho com `len()` quando tiver dúvida.
- Confundir `append()` (adiciona um item ao final) com tentar simplesmente somar (`lista + valor`), que não funciona da forma esperada para adicionar um único item.
- Esquecer que listas são “mutáveis”: se duas variáveis apontam para a mesma lista, alterar uma afeta a outra. Isso é sutil e será revisitado com mais detalhe adiante — por ora, saiba que existe.

Exercício prático

A Loja da Ana registrou as seguintes vendas em um dia: `[45.90, 120.00, 15.50, 200.00, 33.00, 89.90]`.

1. Guarde essas vendas em uma lista chamada `vendas_do_dia`.
2. Use `sum()` para calcular o faturamento total do dia.
3. Use `max()` e `min()` para descobrir a maior e a menor venda.
4. Use um `for` para mostrar cada venda, indicando se ela é maior que R\$ 100 ou não.

Desafio bônus (opcional): adicione uma nova venda de R\$ 75.00 à lista com `append()` e recalcule o faturamento total.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que é uma lista com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei usar `append()`, `sum()`, `max()` e `min()` em uma lista

ewpage

Tuplas

— Módulo 1 — Fundamentos de Python · Tópico 17 de 25

O que é e por que importa

Tupla é uma coleção ordenada de valores, muito parecida com uma lista (`16-listas.md`), com uma diferença fundamental: depois de criada, ela não pode ser alterada — não dá para adicionar, remover ou trocar itens. Por isso dizemos que tupla é “imutável”, enquanto lista é “mutável”.

Pensa em tupla como um “pacote fechado e lacrado”: uma vez que você define, por exemplo, as coordenadas de um endereço (`latitude`, `longitude`) ou os dados fixos de um produto (`"Caneta Azul"`, `"Papelaria"`), não faz sentido que esses valores mudem no meio do programa. Usar tupla nesses casos é uma forma de dizer “isso aqui é fixo” tanto para quem lê o código quanto para o próprio Python, que pode até otimizar o uso de memória sabendo disso.

Em análise de dados, tuplas aparecem com frequência para representar um “registro” com poucos campos fixos (como um par nome-valor) ou quando uma função (veremos em `22-definindo-funcoes.md`) precisa devolver mais de um resultado de uma vez.

Como funciona (com exemplo comentado)

```

# Tupla criada com parênteses (embora, tecnicamente, o que define é a vírgula)
produto = ("Caneta Azul", 5.90, "Papeleria")

# Acesso por índice, igual em listas
nome = produto[0]
preco = produto[1]
categoria = produto[2]
print(nome, preco, categoria)

# Len() também funciona em tuplas
print("Quantidade de campos:", len(produto))

# for também percorre tuplas normalmente
for informacao in produto:
    print(informacao)

# Tentar alterar um item de uma tupla gera erro
try:
    produto[1] = 6.50
except TypeError as erro:
    print("Não é possível alterar uma tupla:", erro)

# "Desempacotamento": jeito prático de separar os valores de uma tupla em
nome, preco, categoria = produto
print(f"{nome} custa R$ {preco} e é da categoria {categoria}")

# Uso comum: uma lista de tuplas representando várias vendas (produto, valor)
vendas = [("Caneta Azul", 5.90), ("Caderno", 12.50), ("Borracha", 2.25)]

for venda in vendas:
    nome_produto, valor = venda
    print(f"Vendido: {nome_produto} por R$ {valor}")

```

Erros comuns de quem está começando

- Tentar alterar um item de tupla como se fosse lista (`produto[1] = 6.50`), o que gera `TypeError`. Se você precisa alterar os valores depois, a estrutura certa é lista, não tupla.
- Esquecer a vírgula ao criar uma tupla de um item só: `(5.90)` não é uma tupla, é só o número entre parênteses. O correto é `(5.90,)`, com vírgula no final.
- Confundir quando usar tupla e quando usar lista. Regra prática: se os dados podem crescer, encolher ou mudar durante o programa, use lista; se são um conjunto fixo de valores relacionados, tupla é mais apropriada.

Exercício prático

A Loja da Ana quer guardar informações fixas de um produto: nome, preço e categoria.

1. Crie uma tupla `produto` com os valores `("Mochila Escolar", 89.90, "Acessórios")`.
2. Use desempacotamento para separar os três valores em variáveis (`nome, preco, categoria`).
3. Mostre uma mensagem formatada com os três valores.

Desafio bônus (opcional): crie uma lista chamada `catalogo` com três tuplas de produtos diferentes (`nome, preco, categoria`) e use um `for` para mostrar todos, um por linha.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre lista e tupla com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei fazer desempacotamento de tupla em variáveis

ewpage

Dicionários

| Módulo 1 — Fundamentos de Python · Tópico 18 de 25

O que é e por que importa

Dicionário é uma coleção de dados organizada em pares de “chave” e “valor” — em vez de acessar um item pela posição (como em listas, `16-listas.md`), você acessa pelo nome. Pensa em um dicionário de verdade: você não procura a palavra “maçã” pela posição 4523 na página, você procura pela própria palavra. Em Python, `dict` funciona assim: `produto["nome"]` te dá o valor associado à chave `"nome"`.

Isso resolve um problema real de listas simples: se você guarda `["Caneta Azul", 5.90, 10]`, precisa lembrar de cabeça que a posição 0 é o nome, a 1 é o preço e a 2 é a quantidade. Com dicionário, você escreve `{"nome": "Caneta Azul", "preco": 5.90, "quantidade": 10}` e acessa cada campo pelo nome, o que deixa o código muito mais legível e menos sujeito a erro.

Em análise de dados, dicionários são a forma mais natural de representar um “registro” com vários campos nomeados — muito parecido com uma linha de planilha, onde cada coluna tem um nome. É também a estrutura de dados por trás de formatos populares como JSON, super comuns quando dados vêm de sites ou de outros sistemas.

Como funciona (com exemplo comentado)

```

# Um dicionário representando um produto da Loja da Ana
produto = {
    "nome": "Caneta Azul",
    "preco": 5.90,
    "quantidade_estoque": 10,
    "categoria": "Papeleria",
}

# Acessando um valor pela chave
print(produto["nome"])      # Caneta Azul
print(produto["preco"])     # 5.90

# Alterando um valor existente
produto["quantidade_estoque"] = 8
print(produto["quantidade_estoque"]) # 8

# Adicionando uma nova chave
produto["em_promocao"] = False
print(produto)

# .get() acessa um valor sem quebrar o programa se a chave não existir
fornecedor = produto.get("fornecedor", "Não informado")
print(fornecedor) # "Não informado", pois a chave "fornecedor" não existe

# .keys(), .values() e .items() percorrem o dicionário de formas diferente
for chave in produto.keys():
    print("Chave:", chave)

for valor in produto.values():
    print("Valor:", valor)

for chave, valor in produto.items():
    print(f"{chave}: {valor}")

# Uso muito comum: uma lista de dicionários, representando várias vendas
vendas = [
    {"produto": "Caneta Azul", "valor": 5.90, "quantidade": 3},
    {"produto": "Caderno", "valor": 12.50, "quantidade": 1},
]

for venda in vendas:
    total_venda = venda["valor"] * venda["quantidade"]
    print(f"{venda['produto']}: total de R$ {total_venda}")

```

Erros comuns de quem está começando

- Acessar uma chave que não existe com `[]` (ex: `produto["fornecedor"]`), o que gera erro `KeyError`. Quando não tem certeza se a chave existe, use `.get()`, que permite definir um valor padrão.
- Confundir chave de dicionário com índice de lista. Dicionários não têm “posição 0” — a ordem de inserção até é preservada em Python moderno, mas o acesso correto é sempre pela chave.
- Esquecer as aspas nas chaves de texto (`produto[nome]` em vez de `produto["nome"]`), o que faz o Python procurar por uma variável chamada `nome`, gerando erro se ela não existir.

Exercício prático

A Loja da Ana quer representar uma venda como um dicionário.

1. Crie um dicionário `venda` com as chaves `"produto"` (texto), `"preco_unitario"` (float), `"quantidade"` (int) e `"forma_pagamento"` (texto).
2. Calcule o valor total da venda (`preco_unitario * quantidade`) e guarde numa nova chave `"total"` dentro do próprio dicionário.
3. Mostre uma mensagem com todos os dados da venda, usando as chaves do dicionário.

Desafio bônus (opcional): crie uma lista `vendas_do_dia` com três dicionários de vendas diferentes, use um `for` para somar o `"total"` de todas e mostre o faturamento do dia.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que é um dicionário (chave/valor) com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei usar `.get()` para acessar uma chave com segurança

ewpage

Sets

| Módulo 1 — Fundamentos de Python · Tópico 19 de 25

O que é e por que importa

Set (conjunto) é uma coleção de valores parecida com lista, mas com duas diferenças importantes: não guarda itens duplicados, e não mantém uma ordem garantida. Pensa em set como uma “lista de presença” onde cada nome só pode aparecer uma vez, mesmo que a pessoa tente se cadastrar de novo.

Em análise de dados, sets resolvem um problema muito comum: descobrir os valores únicos de uma coluna. Por exemplo, se você tem uma lista com o produto de cada venda do mês, e quer saber “quais produtos diferentes foram vendidos” (sem repetição), transformar essa lista em set já te dá a resposta na hora, sem precisar escrever um loop manual para eliminar duplicatas.

Sets também são ótimos para operações de comparação entre grupos: “quais clientes compraram em janeiro E em fevereiro” (interseção), “quais produtos existem na loja A OU na loja B” (união), “quais clientes compraram em janeiro mas NÃO em fevereiro” (diferença). Essas operações, feitas na mão com listas e `for`, dariam bem mais trabalho.

Como funciona (com exemplo comentado)

```

# Lista de produtos vendidos ao longo do dia (com repetição)
produtos_vendidos = ["Caneta Azul", "Caderno", "Caneta Azul", "Borracha",

# set() remove as repetições automaticamente
produtos_unicos = set(produtos_vendidos)
print(produtos_unicos) # {'Caneta Azul', 'Caderno', 'Borracha'} -- ordem

print("Quantidade de produtos diferentes vendidos:", len(produtos_unicos))

# in funciona em sets, e costuma ser mais rápido do que em listas grandes
print("Vendemos Borracha hoje?", "Borracha" in produtos_unicos)

# Sets também podem ser criados diretamente com chaves {}
clientes_janeiro = {"Ana", "Bruno", "Carla"}
clientes_fevereiro = {"Bruno", "Carla", "Diego"}

# Interseção: quem comprou nos dois meses
clientes_fieis = clientes_janeiro & clientes_fevereiro
print("Clientes que compraram nos dois meses:", clientes_fieis)

# União: todos os clientes distintos dos dois meses juntos
todos_os_clientes = clientes_janeiro | clientes_fevereiro
print("Todos os clientes (sem repetição):", todos_os_clientes)

# Diferença: quem comprou em janeiro mas não voltou em fevereiro
clientes_perdidos = clientes_janeiro - clientes_fevereiro
print("Clientes que sumiram em fevereiro:", clientes_perdidos)

# Adicionando um item a um set (não gera erro mesmo se já existir)
clientes_janeiro.add("Ana") # já existe, não faz diferença
clientes_janeiro.add("Elias") # novo cliente
print(clientes_janeiro)

```

Erros comuns de quem está começando

- Tentar acessar um set por índice (`produtos_unicos[0]`), o que gera erro — sets não têm ordem garantida, então não existe “primeiro item” de forma confiável.
- Esquecer que transformar uma lista em set perde a ordem original e qualquer repetição intencional. Se a ordem ou a contagem de repetições importa para a análise, set não é a estrutura certa.
- Confundir set vazio: `{}` cria um dicionário vazio, não um set vazio. Para criar um set vazio, é preciso usar `set()`.

Exercício prático

A Loja da Ana registrou os produtos vendidos ao longo da semana:

```
produtos_vendidos = ["Caneta Azul", "Caderno", "Caneta Azul", "Borracha",
```

1. Use `set()` para descobrir quantos produtos diferentes foram vendidos na semana.
2. Mostre a lista de produtos únicos.
3. Verifique se "Lápis" foi vendido nessa semana, usando `in`.

Desafio bônus (opcional): a loja também tem uma lista de produtos em promoção: `{"Caderno", "Mochila", "Lápis"}`. Descubra quais produtos vendidos na semana também estavam em promoção, usando interseção (`&`).

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que é um set e para que serve com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei usar interseção (&), união (|) e diferença (-) entre sets

ewpage

List comprehensions

Módulo 1 — Fundamentos de Python · Tópico 20 de 25

O que é e por que importa

List comprehension é uma forma compacta de criar uma lista nova a partir de outra, aplicando uma transformação e/ou um filtro, tudo em uma linha só. É basicamente um `for` (visto em `12-loops.md`) que constrói uma lista, só que escrito de forma mais enxuta.

Pensa no problema: você tem uma lista de preços sem desconto e quer uma lista nova com todos os preços já com 10% de desconto aplicado. Com `for` tradicional, você criaria uma lista vazia, faria um loop, e usaria `.append()` a cada volta (como fizemos em `16-listas.md`). List comprehension faz a mesma coisa em uma única linha, e é a forma que a maioria dos programadores Python prefere escrever quando a transformação é simples.

Em análise de dados, esse padrão — “para cada item de uma lista, calcule algo e monte uma lista nova” (ou “para cada item, mantenha só os que passam em um filtro”) — é extremamente comum: aplicar desconto em todos os preços, filtrar só as vendas acima de um valor, extrair só os nomes de uma lista de dicionários.

Como funciona (com exemplo comentado)

```

precos = [10.00, 25.90, 5.50, 100.00]

# Jeito tradicional, com for e append (o que você já sabe fazer)
precos_com_desconto_tradicional = []
for preco in precos:
    precos_com_desconto_tradicional.append(preco * 0.9)
print(precos_com_desconto_tradicional)

# O mesmo resultado, usando List comprehension
precos_com_desconto = [preco * 0.9 for preco in precos]
print(precos_com_desconto)

# List comprehension também pode filtrar itens, usando if no final
vendas = [45.90, 120.00, 15.50, 200.00, 33.00]
vendas_grandes = [venda for venda in vendas if venda > 100]
print("Vendas grandes:", vendas_grandes)

# Combinando transformação e filtro ao mesmo tempo
vendas_grandes_com_taxa = [venda * 1.05 for venda in vendas if venda > 100]
print("Vendas grandes com taxa de 5%:", vendas_grandes_com_taxa)

# Também funciona para extrair um campo de uma lista de dicionários
catalogo = [
    {"nome": "Caneta Azul", "preco": 5.90},
    {"nome": "Caderno", "preco": 12.50},
    {"nome": "Mochila", "preco": 89.90},
]

nomes_produtos = [produto["nome"] for produto in catalogo]
print(nomes_produtos)

produtos_caros = [produto["nome"] for produto in catalogo if produto["preco"] > 10]
print("Produtos acima de R$ 10:", produtos_caros)

```

Erros comuns de quem está começando

- Tentar fazer uma list comprehension complexa demais (com vários if/else e lógica pesada) e acabar com uma linha ilegível. Se está difícil de ler, é sinal de que um for tradicional é a escolha melhor.
- Esquecer os colchetes [] envolvendo a expressão. Sem eles, o Python interpreta a expressão de outra forma (gerador), o que foge do escopo deste tópico.
- Confundir a ordem: em list comprehension, a expressão de transformação vem **antes** do for ([preco * 0.9 for preco in precos]), o que é o oposto da ordem “natural” de um for tradicional.

Exercício prático

A Loja da Ana tem a seguinte lista de vendas do dia: [45.90, 120.00, 15.50, 200.00, 33.00, 89.90].

1. Usando list comprehension, crie uma nova lista vendas_com_taxa_cartao aplicando uma taxa de 3% (multiplicando por 1.03) em cada venda.
2. Usando list comprehension com filtro, crie uma lista vendas_acima_de_50 contendo só as vendas maiores que R\$ 50.
3. Mostre as duas listas resultantes.

Desafio bônus (opcional): usando list comprehension, crie uma lista de textos formatados, tipo "R\$ 45.9", a partir da lista original de vendas (dica: use str() dentro da comprehension).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que é uma list comprehension com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei quando é melhor usar `for` tradicional em vez de list comprehension

ewpage

Funções e métodos (diferença entre os dois)

Módulo 1 — Fundamentos de Python · Tópico 21 de 25

O que é e por que importa

Você já usou os dois conceitos deste tópico sem uma explicação formal: funções, como `print()`, `len()`, `sum()`, `int()`; e métodos, como `.upper()`, `.strip()`, `.append()` (vistos em `14-metodos-de-string.md` e `16-listas.md`). Chegou a hora de formalizar a diferença entre eles, porque isso ajuda a entender qualquer código Python que você ler daqui para frente.

Uma função é independente: ela existe “solta” e você a chama passando o que ela precisa entre parênteses, como `len(nome_produto)` — aqui, `len` é a função, e `nome_produto` é o dado que ela processa. Já um método é uma função que “pertence” a um valor específico e só pode ser chamada a partir dele, com a sintaxe `valor.metodo()` — como `nome_produto.upper()`, onde `upper()` só existe porque `nome_produto` é uma string, e strings sabem se transformar em maiúsculas.

Pensa assim: função é uma ferramenta genérica na caixa de ferramentas (qualquer coisa compatível pode usar); método é uma habilidade que só um tipo específico de objeto tem (só string sabe fazer `.upper()`, só lista sabe fazer `.append()`). Entender essa diferença evita a confusão comum de tentar chamar `upper(nome_produto)` (função, errado) ou `len.nome_produto()` (método, errado) — e também prepara terreno para `22-definindo-funcoes.md`, onde você vai aprender a criar suas próprias funções.

Como funciona (com exemplo comentado)

```

nome_produto = "   caneta azul   "

# Função: chamada como algo_independente(valor)
tamanho = len(nome_produto)          # len() é função, funciona com string
print("Tamanho (com espaços):", tamanho)

tipo_do_dado = type(nome_produto)     # type() também é função
print("Tipo do dado:", tipo_do_dado)

# Método: chamado como valor.algo_do_valor()
nome_limpo = nome_produto.strip()     # strip() é método, só existe porque
nome_maiusculo = nome_limpo.upper()   # upper() também é método de string
print(nome_maiusculo)

# Um mesmo nome pode existir como função OU como método, com comportamento
# Por exemplo, não existe "sort" como função solta para ordenar qualquer c
# mas listas têm o método .sort():
vendas = [45.90, 120.00, 15.50, 200.00]
vendas.sort()                         # método -- ordena a própria lista
print(vendas)

# Já a função sorted() (função, não método) funciona em várias coleções e
vendas_originais = [45.90, 120.00, 15.50, 200.00]
vendas_ordenadas = sorted(vendas_originais) # função -- não altera a list
print("Original:", vendas_originais)
print("Ordenada (nova lista):", vendas_ordenadas)

# Métodos podem ser "encadeados" um atrás do outro, já que cada um devolve
nome_formatado = "   ana silva   ".strip().title()
print(nome_formatado)                 # "Ana Silva"

```

Erros comuns de quem está começando

- Tentar chamar um método como se fosse função, escrevendo `upper(nome_produto)` em vez de `nome_produto.upper()`. Métodos sempre precisam do valor “dono” antes do ponto.
- Achar que todo tipo de dado tem os mesmos métodos. `.upper()` só existe em string; tentar usar em uma lista ou número gera erro `AttributeError`.
- Confundir método que altera o valor original (como `.append()` e `.sort()` em listas) com método que devolve um valor novo sem alterar o original (como `.upper()` em strings, ou `sorted()`, que é função). Vale sempre checar a documentação ou testar quando tiver dúvida.

Exercício prático

Considere a string " Loja Da ANA " e a lista `[89.90, 45.00, 200.00, 15.50]`.

1. Use uma função para descobrir quantos caracteres tem a string original (com espaços).
2. Use métodos encadeados para limpar os espaços e formatar a string no padrão “Título” (primeira letra de cada palavra maiúscula).
3. Use uma função (não método) para criar uma nova lista ordenada a partir da lista de vendas, sem alterar a lista original.
4. Mostre a lista original e a lista ordenada lado a lado, para confirmar que a original não mudou.

Desafio bônus (opcional): explique, em um comentário no código, por que `sorted(lista)` é uma função e `lista.sort()` é um método, e qual a diferença prática de efeito colateral entre os dois.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre função e método com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei identificar, olhando o código, se algo é uma função ou um método

ewpage

Definindo funções (def, parâmetros, return)

| Módulo 1 — Fundamentos de Python · Tópico 22 de 25

O que é e por que importa

Até agora você usou funções prontas do Python, como `print()`, `len()` e `sum()` (formalizadas em `21-funcoes-e-metodos.md`). Chegou a hora de criar as suas próprias. Uma função definida por você é um pedaço de código nomeado e reutilizável — você escreve a lógica uma vez, e depois só “chama” a função sempre que precisar daquele cálculo, sem copiar e colar o código de novo.

Pensa em uma função como uma receita de bolo escrita uma vez: a receita recebe ingredientes (os “parâmetros”, os dados de entrada) e produz um bolo (o “retorno”, o resultado). Você não reescreve a receita toda vez que quer fazer um bolo — só segue os passos de novo, trocando os ingredientes se quiser.

Em análise de dados, funções evitam repetição e reduzem erro: se você precisa calcular “valor total com desconto” em quinze lugares diferentes do seu código, é melhor escrever essa regra uma vez, em uma função, do que repetir a fórmula quinze vezes (e correr o risco de errar em uma delas, ou esquecer de atualizar todas se a regra mudar).

Como funciona (com exemplo comentado)

```

# def define uma função. Os nomes entre parênteses são os "parâmetros" --
# os dados que a função espera receber para funcionar
def calcular_total_venda(preco_unitario, quantidade):
    total = preco_unitario * quantidade
    return total # return devolve o resultado para quem chamou a função

# Chamando a função, passando os "argumentos" (os valores reais para os pa
total_venda_1 = calcular_total_venda(5.90, 3)
print("Total da venda 1:", total_venda_1)

total_venda_2 = calcular_total_venda(12.50, 2)
print("Total da venda 2:", total_venda_2)

# Uma função pode ter um valor padrão para um parâmetro, usado quando não
def calcular_total_com_desconto(preco_unitario, quantidade, desconto=0):
    total = preco_unitario * quantidade
    total_com_desconto = total * (1 - desconto)
    return total_com_desconto

print(calcular_total_com_desconto(10.00, 2)) # sem desconto, usa
print(calcular_total_com_desconto(10.00, 2, 0.10)) # com 10% de descon

# Uma função pode ter várias linhas de lógica antes do return, inclusive i
def classificar_venda(valor):
    if valor >= 200:
        return "grande"
    elif valor >= 50:
        return "media"
    else:
        return "pequena"

print(classificar_venda(35.00)) # pequena
print(classificar_venda(89.90)) # media

# Uma função sem return explícito devolve None (o "vazio" do Python)
def mostrar_recibo(produto, total):
    print(f"Recibo: {produto} - Total: R$ {total}")
    # sem return aqui -- essa função só imprime, não devolve valor útil

resultado = mostrar_recibo("Caneta Azul", 17.70)
print(resultado) # None

```

Erros comuns de quem está começando

- Esquecer o return, e depois se surpreender que a função “não funciona” quando na verdade ela só não devolve nada (None) para ser usado depois. print() dentro da função mostra algo na tela, mas não é o mesmo que devolver um valor com return.
- Confundir “parâmetro” (o nome usado dentro da definição da função, como preco_unitario) com “argumento” (o valor real passado na hora de chamar, como 5.90). Na prática os termos são usados de forma intercambiável no dia a dia, mas vale saber a distinção.
- Esquecer que uma variável criada dentro da função (como total no primeiro exemplo) só existe dentro dela — tentar usar total fora da função gera erro, porque o “escopo” da variável é local à função.

Exercício prático

Crie uma função chamada calcular_ticket_medio que recebe dois parâmetros: faturamento_total e numero_de_vendas, e devolve o ticket médio (faturamento dividido pelo número de vendas).

1. Defina a função com def, usando return para devolver o resultado.
2. Chame a função com os dados: faturamento de R\$ 850.00 e 10 vendas.
3. Mostre o resultado com print().

Desafio bônus (opcional): crie uma função `classificar_ticket_medio` que recebe o ticket médio calculado e devolve "alto" se for maior que R\$ 100, "médio" se estiver entre R\$ 50 e R\$ 100, e "baixo" caso contrário. Use a saída da primeira função como entrada da segunda.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que são parâmetros e `return` com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo que uma variável criada dentro de uma função não existe fora dela

ewpage

*args e **kwargs

Módulo 1 — Fundamentos de Python · Tópico 23 de 25

O que é e por que importa

Em `22-definindo-funcoes.md` você aprendeu a criar funções com um número fixo de parâmetros. Mas e se você não souber, de antemão, quantos valores a função vai receber? Por exemplo, uma função que soma o total de vendas, que às vezes recebe 3 valores, às vezes 10. É para isso que existem `*args` e `**kwargs` — duas formas de uma função aceitar uma quantidade variável de argumentos.

`*args` (o nome “args” é só uma convenção, o `*` é o que importa) permite que a função receba qualquer quantidade de argumentos “soltos” (posicionais), que chegam dentro da função organizados automaticamente como uma tupla (vista em `17-tuplas.md`). Já `**kwargs` permite receber qualquer quantidade de argumentos “nomeados” (`chave=valor`), organizados como um dicionário (visto em `18-dicionarios.md`) dentro da função.

Pensa em `*args` como uma caixa que aceita quantos itens quiser, sem rótulo (só a ordem importa); e `**kwargs` como uma caixa que aceita quantos itens quiser, mas cada um com uma etiqueta escrita (o nome do argumento). Você provavelmente vai usar isso com menos frequência do que funções normais no dia a dia, mas é comum encontrar em bibliotecas de análise de dados (como `pandas`), então vale entender a ideia.

Como funciona (com exemplo comentado)

```
# *args: recebe qualquer quantidade de valores "soltos", vira uma tupla de
def somar_vendas(*valores):
    print("Valores recebidos (como tupla):", valores)
    return sum(valores)

print(somar_vendas(45.90, 120.00, 15.50))      # funciona com 3 valores
print(somar_vendas(10.00, 20.00, 30.00, 40.00)) # funciona com 4 valores

# **kwargs: recebe qualquer quantidade de argumentos nomeados, vira um dic
def cadastrar_produto(**dados):
    print("Dados recebidos (como dicionário):", dados)
    nome = dados.get("nome", "Produto sem nome")
    preco = dados.get("preco", 0)
    print(f"Cadastrando {nome} por R$ {preco}")

cadastrar_produto(nome="Caneta Azul", preco=5.90, categoria="Papeleria")
cadastrar_produto(nome="Caderno", preco=12.50) # funciona mesmo sem "cate

# É possível combinar parâmetros fixos com *args e **kwargs na mesma função
def registrar_venda(vendedor, *itens, **detalhes):
    print("Vendedor:", vendedor)
    print("Itens vendidos:", itens)
    print("Detalhes extras:", detalhes)

registrar_venda("Ana", "Caneta Azul", "Caderno", forma_pagamento="cartao",
```

Erros comuns de quem está começando

- Achar que `*args` e `**kwargs` são nomes obrigatórios. O que importa é o `*` e o `**` — você poderia escrever `*valores` ou `**dados`, como no exemplo acima; `args` e `kwargs` são só o costume mais comum entre programadores Python.
- Tentar misturar a ordem errada: parâmetros fixos vêm primeiro, depois `*args`, depois `**kwargs`. Escrever na ordem errada gera erro de sintaxe.
- Usar `*args/**kwargs` quando uma função com parâmetros fixos resolveria de forma mais clara. Se você sabe exatamente quantos e quais valores a função vai receber, prefira parâmetros nomeados normais — são mais fáceis de entender e de usar corretamente.

Exercício prático

Crie uma função `calcular_faturamento_total` que aceita uma quantidade variável de valores de venda (usando `*args`) e devolve a soma de todos.

1. Defina a função usando `*args`.
2. Chame-a com três valores de vendas da Loja da Ana: 45.90, 120.00, 15.50.
3. Chame-a novamente com cinco valores diferentes, para provar que funciona com quantidades variadas.

Desafio bônus (opcional): crie uma função `resumo_produto` que aceita `**kwargs` e monta uma frase descrevendo o produto usando os pares chave-valor recebidos (ex: `resumo_produto(nome="Mochila", preco=89.90, cor="azul")`).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre `*args` e `**kwargs` com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução

☐ Entendo que `*args` vira tupla e `**kwargs` vira dicionário dentro da função

ewpage

Funções lambda

Módulo 1 — Fundamentos de Python · Tópico 24 de 25

O que é e por que importa

Função lambda é uma forma de criar uma função pequena, sem nome, em uma única linha — usada quando você precisa de uma lógica simples e rápida, geralmente para usar uma única vez ou passar como argumento para outra função. É uma alternativa mais curta ao `def` (visto em 22-definindo-funcoes.md), reservada para casos bem simples.

Pensa em lambda como um “post-it” comparado a um “manual de instruções”: se a lógica é tão simples que caberia em uma frase (como “multiplique por 2” ou “pegue o preço”), às vezes não vale a pena escrever uma função `def` completa com nome — um lambda resolve rapidinho, sem burocracia.

Em análise de dados, lambdas aparecem com muita frequência dentro de funções que esperam receber “uma regra” como argumento — por exemplo, dizer para o Python “ordene esta lista de produtos, mas pelo preço, não pelo nome” ou “aplique esta transformação em cada valor”. Isso conecta diretamente com 25-funcoes-built-in.md, onde `map()`, `filter()` e `sorted()` costumam usar lambda lado a lado.

Como funciona (com exemplo comentado)

```
# Uma função def tradicional, simples
def dobro(numero):
    return numero * 2

print(dobro(5)) # 10

# A mesma lógica, como lambda -- sem nome, sem "def", sem "return" explícito
dobro_lambda = lambda numero: numero * 2
print(dobro_lambda(5)) # 10

# Lambda pode receber mais de um parâmetro, separados por vírgula
calcular_total = lambda preco, quantidade: preco * quantidade
print(calcular_total(5.90, 3)) # 17.7

# O uso mais comum de lambda é como argumento de outra função, sem precisar nomear a função separadamente. Exemplo: ordenar uma lista de produtos por preço
produtos = [
    {"nome": "Mochila", "preco": 89.90},
    {"nome": "Caneta Azul", "preco": 5.90},
    {"nome": "Caderno", "preco": 12.50},
]

# sorted() aceita um parâmetro "key" -- uma função que diz "pelo que ordenar"
produtos_por_preco = sorted(produtos, key=lambda produto: produto["preco"])
for produto in produtos_por_preco:
    print(produto["nome"], "-", produto["preco"])

# Sem lambda, seria preciso definir uma função separada só para isso:
def pegar_preco(produto):
    return produto["preco"]

produtos_por_preco_v2 = sorted(produtos, key=pegar_preco) # mesmo resultado
```

Erros comuns de quem está começando

- Tentar escrever lógica complexa dentro de um lambda (com `if/elif/else` de várias linhas, laços, etc.). Lambda só aceita uma única expressão — se a lógica é complicada, use `def` normal.
- Esquecer que lambda não usa a palavra `return`: o valor depois dos dois-pontos já é devolvido automaticamente.
- Usar lambda “porque parece mais profissional”, mesmo quando uma função `def` nomeada deixaria o código mais legível. Lambda é uma ferramenta de conveniência, não uma obrigação — clareza vem sempre primeiro.

Exercício prático

A Loja da Ana tem a seguinte lista de produtos:

```
produtos = [  
    {"nome": "Mochila", "preco": 89.90},  
    {"nome": "Caneta Azul", "preco": 5.90},  
    {"nome": "Caderno", "preco": 12.50},  
    {"nome": "Borracha", "preco": 2.25},  
]
```

1. Use `sorted()` com uma lambda para ordenar os produtos do mais barato para o mais caro.
2. Use `sorted()` novamente, mas com `reverse=True`, para ordenar do mais caro para o mais barato.
3. Mostre os nomes dos produtos, na ordem resultante, em cada um dos dois casos.

Desafio bônus (opcional): crie uma lambda que recebe um produto e devolve `True` se o preço for maior que R\$ 10 (útil para usar depois com `filter()`, que veremos em 25-funcoes-built-in.md).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que é uma função lambda e quando usá-la com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei usar lambda junto com `sorted()` e o parâmetro `key`

ewpage

Funções built-in (len, sum, map, filter, etc.)

| Módulo 1 — Fundamentos de Python · Tópico 25 de 25

O que é e por que importa

Funções built-in (“embutidas”) são funções que já vêm prontas no Python, sem você precisar defini-las nem importar nada. Você já usou várias ao longo desta apostila — `print()`, `len()`, `type()`, `int()`, `float()`, `sum()`, `max()`, `min()`, `sorted()` — mas ainda não tinha esse nome formal para o grupo. Este é o último tópico do Módulo 1, e serve para consolidar essas ferramentas e apresentar duas que faltavam: `map()` e `filter()`.

Pensa nas built-ins como os “eletrodomésticos de fábrica” da cozinha: já vêm instalados,

prontos para usar, e resolvem tarefas tão comuns que não faria sentido cada pessoa reinventar a sua própria versão. `map()` aplica uma função a cada item de uma coleção (parecido com o que list comprehensions fazem, visto em 20-list-comprehensions.md); `filter()` seleciona só os itens que passam em uma condição (parecido com list comprehension com `if`).

Dominar essas funções builtin, combinadas com o que você aprendeu sobre lambda (24-funcoes-lambda.md) e funções próprias (22-definindo-funcoes.md), fecha o ciclo de fundamentos: a partir do Módulo 2, você vai aplicar tudo isso em bibliotecas como pandas, que são construídas em cima dessas mesmas ideias.

Como funciona (com exemplo comentado)

```
vendas = [45.90, 120.00, 15.50, 200.00, 33.00, 89.90]

# Revisão rápida das built-ins mais usadas até aqui
print(len(vendas))    # 6 -- quantidade de itens
print(sum(vendas))     # soma de todos os valores
print(max(vendas))     # maior valor
print(min(vendas))     # menor valor
print(sorted(vendas))  # nova lista ordenada, sem alterar a original

# map(): aplica uma função a cada item de uma coleção, devolvendo um "map"
# que normalmente convertemos para List() para poder ver/usar
vendas_com_taxa = list(map(lambda venda: venda * 1.05, vendas))
print("Vendas com taxa de 5%:", vendas_com_taxa)

# O mesmo resultado poderia ser feito com List comprehension (visto em 20-
vendas_com_taxa_v2 = [venda * 1.05 for venda in vendas]
print(vendas_com_taxa_v2 == vendas_com_taxa) # True, mesmo resultado, for

# filter(): mantém só os itens que passam em uma condição (função que devo
vendas_grandes = list(filter(lambda venda: venda > 100, vendas))
print("Vendas grandes:", vendas_grandes)

# round() arredonda, já visto em 05-floats.md -- reforçando aqui como buil
print(round(45.678, 1)) # 45.7

# any() e all() são úteis para checagens em lote
tem_venda_acima_de_150 = any(venda > 150 for venda in vendas)
print("Alguma venda acima de R$ 150?", tem_venda_acima_de_150)

todas_vendas_positivas = all(venda > 0 for venda in vendas)
print("Todas as vendas são positivas?", todas_vendas_positivas)
```

Erros comuns de quem está começando

- Esquecer de converter o resultado de `map()` e `filter()` para `list()` antes de usar. Sem essa conversão, você tem um objeto especial que ainda não “virou” a lista de valores propriamente dita para imprimir ou percorrer normalmente.
- Achar que `map()/filter()` são obrigatoriamente melhores que list comprehension. Na prática, muitos programadores Python preferem list comprehension por ser mais legível — vale conhecer as duas formas e escolher a que ficar mais clara para o seu caso.
- Usar `sum()` em uma lista que contém texto misturado com número, o que gera erro (`sum()` só funciona em coleções totalmente numéricas).

Exercício prático

A Loja da Ana tem a seguinte lista de vendas do mês: [45.90, 120.00, 15.50, 200.00, 33.00, 89.90, 310.00].

1. Use `sum()` e `len()` para calcular o ticket médio das vendas (faturamento total dividido

- pela quantidade de vendas).
2. Use `filter()` com uma `lambda` para obter apenas as vendas acima do ticket médio calculado.
 3. Use `map()` com uma `lambda` para criar uma lista com 5% de comissão sobre cada venda acima do ticket médio.
 4. Mostre o ticket médio, as vendas acima da média e os valores de comissão.

Desafio bônus (opcional): use `any()` para checar se existe alguma venda acima de R\$ 300, e `all()` para checar se todas as vendas são maiores que R\$ 10.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que `map()` e `filter()` fazem com minhas próprias palavras
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Terminei o Módulo 1 e me sinto pronto para seguir para o Módulo 2

ewpage

Módulo 2 — Python Intermediário e Ambiente

Com os fundamentos da linguagem em mãos (Módulo 1), este módulo fecha o conhecimento de Python “puro” que falta antes de entrarmos em bibliotecas de dados (NumPy e Pandas), e te apresenta as ferramentas de ambiente que profissionais de análise de dados usam no dia a dia: como instalar pacotes, isolar projetos e trabalhar em notebooks interativos.

Continuamos usando a “Loja da Ana” como fio condutor, mas agora ela vai guardar dados em arquivos (JSON, CSV) e ganhar um pouco mais de estrutura com classes.

Tópicos

1. OOP para análise de dados
2. Módulo `random`
3. Expressões regulares (re) — nível básico
4. JSON (ler e escrever)
5. CSV (ler e escrever sem pandas)
6. `pip` (instalando pacotes)
7. Ambientes virtuais (`venv`)
8. `Conda` (alternativa ao `venv`)
9. VS Code (configuração básica para Python)
10. JupyterLab
11. Google Colab

Depois de terminar todos os tópicos, siga para o Módulo 3 — NumPy.

ewpage

OOP para análise de dados

O que é e por que importa

Até agora, os dados da Loja da Ana viviam em variáveis soltas ou em dicionários: um dicionário para cada produto, uma lista de dicionários para as vendas. Isso funciona, mas em algum momento fica repetitivo — toda venda tem “produto”, “quantidade” e “preço”, e toda vez que você quer calcular o total daquela venda, reescreve a mesma fórmula.

OOP (Programação Orientada a Objetos, do inglês “Object-Oriented Programming”) é uma forma de organizar código em torno de “coisas” do mundo real. Em vez de espalhar dados e funções separadamente, você cria um **objeto** que já sabe quais informações carrega e o que sabe fazer com elas. Pensa numa ficha de cadastro: uma ficha de produto tem campos fixos (nome, preço, categoria) e ações associadas (calcular imposto, aplicar desconto). Uma **classe** é o molde dessa ficha; um **objeto** é uma ficha preenchida a partir desse molde.

Isso importa em análise de dados porque, conforme seus scripts crescem, você começa a lidar com “coisas” que se repetem — produtos, vendas, clientes — cada uma com os mesmos campos e os mesmos cálculos. Uma classe evita que você repita a lógica de cálculo em vários lugares e deixa o código mais parecido com a forma como você pensa no problema (“pegue essa venda e calcule o total dela”, em vez de “pegue esse dicionário, ache a chave ‘preço’, multiplique pela chave ‘quantidade’”).

Não se preocupe em virar um “programador orientado a objetos” agora — aqui vamos só o suficiente para reconhecer classes quando aparecerem (inclusive dentro de bibliotecas como pandas, que usa objetos por baixo dos panos) e para criar as suas próprias quando ajudar a organizar os dados da Loja da Ana.

Como funciona (com exemplo comentado)

```

# "class" define o molde. Por convenção, nomes de classes começam com Letra Maiúscula
class Produto:
    # __init__ é um método especial, chamado automaticamente quando você cria um objeto
    # "self" representa o próprio objeto sendo criado -- é assim que o objeto acessa seus próprios dados
    def __init__(self, nome, preco, quantidade_em_estoque):
        self.nome = nome
        self.preco = preco
        self.quantidade_em_estoque = quantidade_em_estoque

    # Um "método" é uma função que pertence à classe -- ela sempre recebe o objeto como primeiro parâmetro, para poder acessar os dados do objeto
    def valor_total_em_estoque(self):
        return self.preco * self.quantidade_em_estoque

    def aplicar_reajuste(self, percentual):
        self.preco = self.preco * (1 + percentual)

# Criando objetos a partir do molde "Produto" -- cada um é uma "ficha" individual
caneta = Produto("Caneta Azul", 2.50, 100)
caderno = Produto("Caderno Capa Dura", 15.90, 40)

# Acessando os atributos (os dados guardados no objeto)
print(caneta.nome, caneta.preco)          # Caneta Azul 2.5
print(caderno.quantidade_em_estoque)      # 40

# Chamando os métodos (as ações que o objeto sabe fazer)
print("Valor em estoque da caneta:", caneta.valor_total_em_estoque())  # 250
print("Valor em estoque do caderno:", caderno.valor_total_em_estoque()) # 636

caneta.aplicar_reajuste(0.10) # reajuste de 10%
print("Novo preço da caneta:", caneta.preco) # 2.75

# Uma classe para representar uma venda, reaproveitando um objeto Produto
class Venda:
    def __init__(self, produto, quantidade):
        self.produto = produto
        self.quantidade = quantidade

    def total(self):
        return self.produto.preco * self.quantidade

venda_1 = Venda(caneta, 3)
print("Total da venda:", venda_1.total()) # 8.25

```

Repare que caneta e caderno foram criados a partir do **mesmo** molde (Produto), mas cada um guarda seus próprios valores — mudar o preço da caneta não afeta o caderno. Isso é o que chamamos de “instância”: cada objeto é uma instância independente da classe.

Erros comuns de quem está começando

- Esquecer o `self` como primeiro parâmetro de um método (inclusive do `__init__`). Sem ele, o Python não sabe a quem pertencem os dados, e você recebe um erro de argumentos.
- Confundir uma classe (o molde, ex: `Produto`) com o objeto (a ficha preenchida, ex: `caneta`). Você define a classe uma vez, mas pode criar quantos objetos quiser a partir dela.
- Tentar acessar um atributo antes de criar o objeto, ou esquecer o `self`. na frente do nome do atributo dentro dos métodos (escrever `preco` em vez de `self.preco`) — isso faz o Python procurar uma variável solta que não existe.

Exercício prático

Crie uma classe `Cliente` com:

1. `__init__` recebendo nome e `total_gasto` (valor já gasto na loja até agora).
2. Um método `e_cliente_vip()` que devolve `True` se `total_gasto` for maior que R\$ 500, e `False` caso contrário.
3. Um método `registrar_compra(valor)` que soma valor ao `total_gasto` do cliente.

Crie um objeto `Cliente("Marcos", 320.00)`, registre uma nova compra de R\$ 250.00, e depois imprima se ele é VIP.

Desafio bônus (opcional): crie uma lista com três objetos `Cliente` diferentes e use um `for` (visto em 12-loops.md do Módulo 1) para imprimir o nome de todos que são VIP.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar com minhas próprias palavras a diferença entre classe e objeto
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo para que serve o `self` dentro de uma classe

ewpage

Módulo random

— Módulo 2 — Python Intermediário e Ambiente · Tópico 2 de 11

O que é e por que importa

O Python vem com uma biblioteca embutida chamada `random`, que gera valores aleatórios: números sorteados, escolhas aleatórias dentro de uma lista, embaralhamento de itens. Um “módulo”, nesse contexto, é só um arquivo com funções prontas que você importa e passa a usar — pensa nele como uma “caixa de ferramentas extra” que você abre quando precisa.

Em análise de dados, `random` é extremamente útil para **simular dados** quando você ainda não tem uma base real (ou quer testar seu código antes de plugar dados de verdade), e também para tarefas como sortear uma amostra de clientes para uma pesquisa, embaralhar uma lista antes de dividir em grupos, ou simular cenários (“e se as vendas variassem entre R\$ 10 e R\$ 500 por dia?”).

Vamos usar `random` bastante daqui para frente para gerar bases de dados fictícias da Loja da Ana maiores do que seria prático digitar à mão.

Como funciona (com exemplo comentado)

```

import random # "import" carrega o módulo para você poder usar suas funções

produtos = ["Caneta Azul", "Caderno Capa Dura", "Lapis HB", "Borracha", "M

# random.choice escolhe UM item aleatório de uma sequência (lista, tupla e
produto_sorteado = random.choice(produtos)
print("Produto sorteado:", produto_sorteado)

# random.randint(a, b) sorteia um número inteiro entre a e b, incluindo os
quantidade_vendida = random.randint(1, 10)
print("Quantidade vendida:", quantidade_vendida)

# random.sample escolhe VÁRIOS itens diferentes, sem repetir nenhum
clientes = ["Ana", "Beto", "Carla", "Diego", "Elisa", "Fabio"]
sorteados_para_pesquisa = random.sample(clientes, 3)
print("Clientes sorteados para a pesquisa:", sorteados_para_pesquisa)

# random.seed "trava" a sequência de sorteios -- útil para que o resultado
# aleatório seja sempre o mesmo quando você roda o código de novo (bom par
# testar e para reproduzir um exemplo em uma aula ou apostila)
random.seed(42)
print(random.randint(1, 100)) # sempre vai dar o mesmo número, pois a see
print(random.randint(1, 100)) # o segundo sorteio também é sempre igual,

# Simulando uma pequena base de vendas da Loja da Ana
random.seed(7)
vendas_simuladas = []
for _ in range(5):
    venda = {
        "produto": random.choice(produtos),
        "quantidade": random.randint(1, 5),
        "preco_unitario": round(random.uniform(2.0, 50.0), 2), # uniform
    }
    vendas_simuladas.append(venda)

for venda in vendas_simuladas:
    print(venda)

```

Erros comuns de quem está começando

- Confundir `random.choice` (escolhe um item, pode repetir se você chamar de novo) com `random.sample` (escolhe vários itens diferentes de uma vez, sem repetição entre eles).
- Esquecer que `random.randint(a, b)` inclui o `b` no sorteio (diferente de outras funções em Python que costumam excluir o último valor, como o `range`).
- Achar que `random.seed()` faz o número “parar de ser aleatório” para sempre. Na verdade, ela só garante que, a partir daquele ponto, a sequência de sorteios será sempre a mesma — se você mudar a seed ou tirar a linha, os valores voltam a variar a cada execução.

Exercício prático

Usando `random.seed(10)` no início do script (para o resultado ser sempre igual), simule 8 vendas da Loja da Ana. Cada venda deve ser um dicionário com "produto" (sorteado de uma lista com pelo menos 4 produtos que você escolher) e "quantidade" (um inteiro entre 1 e 6). Guarde tudo em uma lista chamada `vendas` e imprima cada venda com um `for`.

Desafio bônus (opcional): depois de gerar a lista `vendas`, use `random.sample` para sortear 2 vendas dessa lista como se fossem selecionadas para uma “auditoria” e imprima só essas duas.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar com minhas próprias palavras a diferença entre `choice` e `sample`
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo para que serve `random.seed`

ewpage

Expressões regulares (re) — nível básico

— Módulo 2 — Python Intermediário e Ambiente · Tópico 3 de 11

O que é e por que importa

Expressão regular (ou “regex”, abreviação de *regular expression*) é uma forma de descrever um **padrão de texto** para o computador procurar. Em vez de procurar uma palavra exata, você descreve uma “forma”: “um número de telefone tem esse formato”, “um e-mail sempre tem um @ no meio”, “um código de produto começa com três letras seguidas de quatro números”.

Pensa em regex como uma busca do tipo “encontre arquivos” no seu computador, mas para pedaços de texto dentro de uma string: em vez de `*.pdf` para achar arquivos PDF, você escreve um padrão tipo `\d+` para achar “um ou mais números seguidos” dentro de um texto qualquer.

Em análise de dados, regex aparece o tempo todo na etapa de **limpeza de dados**: validar se um campo de telefone está no formato certo, extrair só a parte numérica de um preço escrito como “R\$ 45,90”, checar se um e-mail de cliente parece válido antes de usá-lo, ou separar um código de pedido do resto de uma descrição. O módulo embutido do Python para isso se chama `re`.

Regex tem fama de assustadora por causa dos símbolos, mas neste tópico vamos ver só o essencial — o suficiente para resolver 80% dos casos do dia a dia em análise de dados.

Como funciona (com exemplo comentado)

```

import re # módulo embutido de expressões regulares

# re.search procura o padrão em QUALQUER parte do texto e devolve
# um "objeto de match" se encontrar, ou None se não encontrar
descricao = "Pedido #4521 - Caneta Azul - R$ 2,50"

resultado = re.search(r"\d+", descricao)
# \d significa "um dígito" (0-9). 0+ significa "um ou mais seguidos".
# O "r" antes da string (string "raw") evita que o Python interprete a bar
# invertida de forma estranha -- é praticamente obrigatório usar r"" em re

if resultado:
    print("Encontrado:", resultado.group()) # .group() pega o texto que d
else:
    print("Nenhum número encontrado")

# re.findall devolve TODOS os trechos que combinam com o padrão, em uma li
numeros = re.findall(r"\d+", descricao)
print(numeros) # ['4521', '2', '50']

# re.match só verifica o INÍCIO do texto (diferente de re.search, que proc
codigo_produto = "CAN001"
if re.match(r"[A-Z]{3}", codigo_produto):
    # [A-Z] significa "uma letra maiúscula de A a Z". {3} significa "exata
    print("Código começa com 3 letras maiúsculas -- formato válido")
else:
    print("Formato de código inválido")

# Validando um formato de e-mail de cliente (padrão simples, não cobre tod
email_cliente = "ana.compras@lojadaana.com"
padrao_email = r"^[\\w.]+@[\\w.]+\\.\\w+$"
# ^ marca o início do texto, $ marca o fim (garante que o texto INTEIRO pr
# \\w significa "letra, número ou underline". 0+ de novo é "um ou mais"
if re.match(padrao_email, email_cliente):
    print("E-mail parece válido")
else:
    print("E-mail parece inválido")

# Extraindo só o valor numérico de um preço escrito como texto
preco_texto = "R$ 45,90"
valor = re.findall(r"\d+,\d+", preco_texto)[0] # pega o primeiro (e único
valor_convertido = float(valor.replace(",", ".")) # troca vírgula por pon
print(valor_convertido) # 45.9

```

Erros comuns de quem está começando

- Esquecer o `r` antes da string do padrão (ex: usar `"\d+"` em vez de `r"\d+"`). Na maioria dos casos simples funciona igual, mas em padrões mais complexos a barra invertida pode ser interpretada errado — o hábito de sempre usar `r""` evita dor de cabeça.
- Confundir `re.match` (só olha o começo do texto) com `re.search` (procura em qualquer parte). Usar `match` esperando que ele ache um padrão no meio do texto é um erro comum.
- Esquecer de checar se o resultado de `re.search` ou `re.match` é `None` antes de chamar `.group()` nele. Se o padrão não for encontrado, o resultado é `None`, e chamar `.group()` em `None` gera erro.

Exercício prático

Você recebeu uma lista de descrições de pedidos da Loja da Ana, cada uma no formato "Pedido #NUMERO - NOME_DO_PRODUTO", por exemplo "Pedido #101 - Caderno". Usando `re.findall(r"\d+", ...)` (visto acima), extraia apenas o número do pedido de cada descrição da lista abaixo e imprima todos:

```
pedidos = [  
    "Pedido #101 - Caderno",  
    "Pedido #204 - Caneta Azul",  
    "Pedido #305 - Mochila",  
]
```

Desafio bônus (opcional): usando `re.match`, valide se cada número de pedido extraído tem exatamente 3 dígitos (padrão `r"^\d{3}$"`), e imprima uma mensagem dizendo se o formato está correto para cada um.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar com minhas próprias palavras a diferença entre `re.search`, `re.match` e `re.findall`
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo o que fazem os símbolos `\d`, `+` e `^/$` usados neste tópico

[ewpage](#)

JSON (ler e escrever)

| Módulo 2 — Python Intermediário e Ambiente · Tópico 4 de 11

O que é e por que importa

JSON (*JavaScript Object Notation*) é um formato de texto para guardar dados estruturados em arquivo — parecido com os dicionários e listas que você já conhece do Módulo 1, só que salvo como um arquivo `.json` que pode ser lido por praticamente qualquer linguagem de programação, não só Python. É um dos formatos mais usados hoje para troca de dados entre sistemas (uma API de e-commerce, por exemplo, costuma devolver informações em JSON).

Pensa em JSON como uma “foto” de um dicionário Python guardada em disco: se você tem `{"nome": "Caneta Azul", "preco": 2.50}` na memória do seu programa, o JSON é a versão desse mesmo conteúdo salva em um arquivo de texto, para você poder fechar o programa hoje e abrir o mesmo dado amanhã (ou mandar esse arquivo para outra pessoa).

Até agora, os dados da Loja da Ana existiam só enquanto o script rodava — ao fechar o programa, tudo se perdia. A partir de agora, vamos guardar os dados de vendas, produtos e clientes em arquivos, para simular como isso funciona de verdade no mercado de trabalho. O módulo `json`, embutido no Python, converte dicionários e listas Python em texto JSON (e vice-versa) automaticamente.

Como funciona (com exemplo comentado)

```

import json # módulo embutido, não precisa instalar nada

produto = {
    "nome": "Caneta Azul",
    "preco": 2.50,
    "quantidade_em_estoque": 100,
    "categoria": "Papelaria",
}

# json.dumps converte um dicionário/Lista Python em uma STRING no formato
# ("dumps" = "dump string", ou seja, "despeje como string")
texto_json = json.dumps(produto, indent=2) # indent=2 deixa o texto forma
print(texto_json)

# json.dump faz a mesma coisa, mas escreve DIRETO em um arquivo
with open("produto.json", "w", encoding="utf-8") as arquivo:
    # "w" significa "write" (escrever) -- se o arquivo já existir, ele é s
    json.dump(produto, arquivo, indent=2, ensure_ascii=False)
    # ensure_ascii=False permite salvar acentos e caracteres especiais cor

# json.Load lê um arquivo .json e devolve um dicionário/Lista Python pront
with open("produto.json", "r", encoding="utf-8") as arquivo:
    # "r" significa "read" (ler)
    produto_carregado = json.load(arquivo)

print(produto_carregado["nome"]) # Caneta Azul
print(type(produto_carregado))   # <class 'dict'>

# json.loads (com "s" no final) faz o mesmo que json.load, mas a partir de
# que já está na memória, não de um arquivo aberto
texto_recebido = '{"nome": "Mochila", "preco": 89.90}'
produto_2 = json.loads(texto_recebido)
print(produto_2["preco"]) # 89.9

# JSON também guarda listas de dicionários -- útil para uma base inteira d
vendas = [
    {"produto": "Caneta Azul", "quantidade": 3, "total": 7.50},
    {"produto": "Caderno", "quantidade": 1, "total": 15.90},
]

with open("vendas.json", "w", encoding="utf-8") as arquivo:
    json.dump(vendas, arquivo, indent=2, ensure_ascii=False)

with open("vendas.json", "r", encoding="utf-8") as arquivo:
    vendas_carregadas = json.load(arquivo)

for venda in vendas_carregadas:
    print(venda["produto"], "-", venda["total"])

```

Repare no `with open(...)` `as arquivo:` — essa construção abre o arquivo, executa o bloco indentado, e fecha o arquivo automaticamente no final (mesmo se der erro no meio do caminho). É a forma recomendada de trabalhar com arquivos em Python.

Erros comuns de quem está começando

- Usar "w" (write) quando queria "r" (read), ou vice-versa. Abrir para escrita ("w") em um arquivo que já tem dados importantes **apaga o conteúdo anterior**.
- Confundir `json.dump/json.load` (trabalham com **arquivos** abertos) com `json.dumps/json.loads` (trabalham com **strings** na memória). O "s" extra no nome é a pista: "s" de string.
- Esquecer `encoding="utf-8"` ao abrir arquivos com acentos (nomes de produtos, cidades etc.) — em alguns sistemas isso causa erro ou caracteres corrompidos ao salvar/ler.

Exercício prático

Crie um dicionário representando um cliente da Loja da Ana, com as chaves "nome", "email" e "total_gasto". Salve esse dicionário em um arquivo chamado `cliente.json` usando `json.dump`. Depois, em outra parte do código, leia o arquivo de volta com `json.load` e imprima o nome do cliente carregado.

Desafio bônus (opcional): crie uma lista com três clientes (dicionários) e salve todos em um único arquivo `clientes.json`. Depois carregue o arquivo e, usando um `for`, imprima só os nomes dos clientes que têm `total_gasto` maior que R\$ 300.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre `json.dump/load` e `json.dumps/loads`
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo por que "w" sobrescreve o conteúdo de um arquivo

ewpage

CSV (ler e escrever sem pandas)

| Módulo 2 — Python Intermediário e Ambiente · Tópico 5 de 11

O que é e por que importa

CSV (*Comma-Separated Values*, “valores separados por vírgula”) é o formato de arquivo mais comum para guardar dados em forma de tabela — é basicamente o que você vê quando exporta uma planilha do Excel ou Google Sheets como “CSV”. Cada linha do arquivo é uma linha da tabela, e cada valor dentro da linha é separado por vírgula (ou outro caractere, como ponto e vírgula).

Pensa num CSV como uma planilha “crua”, sem formatação, cores ou fórmulas — só os dados em texto puro, um valor depois do outro. Por ser simples, é o formato mais universal para trocar dados de tabela entre sistemas diferentes: praticamente qualquer ferramenta de dados (Excel, Google Sheets, bancos de dados, pandas) consegue abrir um CSV.

Você vai usar pandas para trabalhar com CSVs de forma mais poderosa a partir do Módulo 4, mas entender como o módulo `csv` embutido do Python funciona por baixo é importante: ajuda a entender o que está acontecendo quando o pandas “mastiga” o arquivo para você, e serve para casos simples onde instalar uma biblioteca extra seria exagero. Vamos guardar as vendas da Loja da Ana em CSV, exatamente como aconteceria numa exportação real de um sistema de vendas.

Como funciona (com exemplo comentado)

```

import csv # módulo embutido, não precisa instalar

vendas = [
    ["produto", "quantidade", "preco_unitario"], # cabeçalho (nomes das
    ["Caneta Azul", 3, 2.50],
    ["Caderno", 1, 15.90],
    ["Mochila", 2, 89.90],
]

# csv.writer escreve linha por linha em um arquivo, a partir de listas
with open("vendas.csv", "w", newline="", encoding="utf-8") as arquivo:
    # newline="" evita linhas em branco extras no Windows -- é praticamente
    # obrigatório usar esse parâmetro ao escrever CSV
    escritor = csv.writer(arquivo)
    for linha in vendas:
        escritor.writerow(linha)
    # ou, em uma linha só, para todas as linhas de uma vez:
    # escritor.writerows(vendas)

# csv.reader lê o arquivo de volta, linha por linha, cada linha vira uma L
with open("vendas.csv", "r", encoding="utf-8") as arquivo:
    leitor = csv.reader(arquivo)
    for linha in leitor:
        print(linha)
    # Saída:
    # ['produto', 'quantidade', 'preco_unitario']
    # ['Caneta Azul', '3', '2.5']
    # ['Caderno', '1', '15.9']
    # ['Mochila', '2', '89.9']
    # Repare que TUDO vira string -- csv.reader não sabe que "3" era um nú

# DictReader lê o CSV e já transforma cada linha em um dicionário,
# usando o cabeçalho como chaves -- muito mais prático que csv.reader puro
with open("vendas.csv", "r", encoding="utf-8") as arquivo:
    leitor_dict = csv.DictReader(arquivo)
    for linha in leitor_dict:
        print(linha["produto"], "-", linha["quantidade"])
        # ainda são strings! para calcular, é preciso converter:
        total = float(linha["quantidade"]) * float(linha["preco_unitario"])
        print("Total:", total)

# DictWriter escreve dicionários direto, sem precisar montar listas na mão
clientes = [
    {"nome": "Marcos", "email": "marcos@email.com"},
    {"nome": "Julia", "email": "julia@email.com"},
]

with open("clientes.csv", "w", newline="", encoding="utf-8") as arquivo:
    campos = ["nome", "email"] # precisa informar os nomes das colunas
    escritor_dict = csv.DictWriter(arquivo, fieldnames=campos)
    escritor_dict.writeheader() # escreve a linha de cabeçalho
    for cliente in clientes:
        escritor_dict.writerow(cliente)

```

Erros comuns de quem está começando

- Esquecer que **tudo que vem de um CSV é string**, mesmo números. Somar "3" com "5" sem converter para int/float primeiro (como visto em 07-conversao-de-tipos.md do Módulo 1) gera erro ou concatenação de texto em vez de soma.
- Esquecer o `newline=""` ao abrir o arquivo para escrita no Windows, o que pode gerar linhas em branco extras entre cada linha do CSV.
- No `DictWriter`, esquecer de chamar `.writeheader()` antes de escrever as linhas — sem isso, o arquivo não terá a linha de cabeçalho com os nomes das colunas.

Exercício prático

Crie uma lista de listas representando vendas da Loja da Ana, com cabeçalho ["produto", "quantidade", "preco_unitario"] e pelo menos 4 vendas. Salve em um arquivo `vendas_loja.csv` usando `csv.writer`. Depois, leia o arquivo de volta com `csv.DictReader` e calcule o total de cada venda (`quantidade * preco_unitario`, lembrando de converter os valores para número), imprimindo o produto e o total.

Desafio bônus (opcional): some o total de todas as vendas lidas do CSV e imprima o faturamento total.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar por que os valores lidos de um CSV vêm sempre como string
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Entendo a diferença entre `csv.reader/writer` e `csv.DictReader/DictWriter`

ewpage

pip (instalando pacotes)

| Módulo 2 — Python Intermediário e Ambiente · Tópico 6 de 11

O que é e por que importa

Até agora você só usou o que já vem “de fábrica” no Python (`random`, `re`, `json`, `csv` — módulos embutidos que não precisam de instalação). Mas o que faz do Python uma linguagem tão forte em análise de dados são bibliotecas externas como `pandas`, `numpy` e `matplotlib`, que **não** vêm instaladas junto com o Python — você precisa buscá-las e instalá-las separadamente.

`pip` é o “gerenciador de pacotes” do Python: um programa de linha de comando que baixa e instala bibliotecas de um repositório público chamado PyPI (*Python Package Index*). Pensa nele como uma loja de aplicativos para o seu Python — em vez de procurar o instalador de cada biblioteca manualmente na internet, você digita um comando e o `pip` baixa, instala e organiza tudo para você.

Esse tópico é a ponte entre “Python puro” (o que você aprendeu até aqui) e as ferramentas de dados que vêm a partir do Módulo 3. Antes de instalar `pandas` e `numpy`, você precisa saber como o `pip` funciona — inclusive porque, no mercado de trabalho, é comum um projeto especificar exatamente quais bibliotecas (e quais versões delas) ele precisa, para garantir que o código funcione igual na sua máquina e na de outra pessoa.

Como funciona (com exemplo comentado)

`pip` é usado no terminal (a tela de linha de comando), não dentro de um arquivo `.py`. Abra o terminal do seu computador (ou o terminal integrado do VS Code, que veremos em `09-vs-code.md`) e experimente:

```
# Verifica se o pip está instalado e qual versão
pip --version

# Instala a versão mais recente disponível de uma biblioteca
pip install pandas

# Instala uma versão ESPECÍFICA de uma biblioteca (útil quando um projeto
# exige uma versão exata, para evitar incompatibilidades)
pip install pandas==2.1.0

# Instala uma versão MÍNIMA (qualquer uma igual ou mais nova que a indicada)
pip install pandas>=2.0.0

# Lista todas as bibliotecas instaladas no ambiente atual, com suas versões
pip list

# Mostra detalhes de uma biblioteca já instalada (versão, local, dependências)
pip show pandas

# Desinstala uma biblioteca
pip uninstall pandas

# Gera um arquivo requirements.txt com todas as bibliotecas instaladas e suas
# versões -- um "resumo" de tudo que o projeto precisa para funcionar
pip freeze > requirements.txt

# Instala TODAS as bibliotecas listadas em um requirements.txt de uma vez
# muito comum ao pegar um projeto de outra pessoa ou configurar uma máquina
pip install -r requirements.txt
```

Um requirements.txt de exemplo, como o que a Loja da Ana usaria em um projeto real de análise de dados:

```
pandas==2.1.0
numpy==1.26.0
matplotlib==3.8.0
```

Esse arquivo funciona como uma “lista de compras”: qualquer pessoa (ou você mesmo, em outro computador) roda `pip install -r requirements.txt` e recria exatamente o mesmo ambiente, com as mesmas bibliotecas e versões.

Erros comuns de quem está começando

- Rodar `pip install` dentro do código Python (ex: escrever `pip install pandas` num arquivo `.py`) — isso é um comando de terminal, não uma instrução Python, e vai gerar erro de sintaxe se colado num script.
- Instalar uma biblioteca em um projeto e ela “não aparecer” no outro — geralmente é sinal de que os dois projetos estão usando ambientes Python diferentes (veremos como isolar isso em `07-ambientes-virtuais-venv.md`).
- Ignorar mensagens de erro do `pip install` achando que “deu certo mesmo assim”. Erros de instalação costumam indicar problema de versão do Python, permissão, ou conexão com a internet — vale sempre ler a mensagem antes de tentar de novo.

Exercício prático

Sem precisar rodar nada (ou, se quiser, rode de verdade no seu terminal):

1. Escreva o comando que instalaria a versão 1.5.3 exata da biblioteca `requests`.
2. Escreva o comando que lista todas as bibliotecas instaladas no seu ambiente atual.
3. Escreva o conteúdo de um `requirements.txt` fictício para um projeto que usa `pandas` versão 2.1.0 e `matplotlib` versão 3.8.0.

Desafio bônus (opcional): se você já tem Python instalado, rode `pip list` de verdade no seu terminal e identifique se `pip` já veio instalado junto (em instalações modernas do Python, geralmente vem).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar o que é o pip e para que serve um `requirements.txt`
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei a diferença entre `pip install nome`, `pip install nome==versao` e `pip install nome>=versao`

ewpage

Ambientes virtuais (venv)

— Módulo 2 — Python Intermediário e Ambiente · Tópico 7 de 11

O que é e por que importa

Imagine que você tem dois projetos no seu computador: o da Loja da Ana, que usa `pandas` versão 2.1.0, e outro projeto antigo de um cliente, que só funciona com `pandas` versão 1.3.0. Se você instalar as bibliotecas com `pip install` “no computador todo” (o que chamamos de Python “global”), só é possível ter **uma** versão de cada biblioteca instalada por vez — instalar a versão nova para um projeto quebra o outro.

Um **ambiente virtual** resolve isso criando uma “caixa” isolada de Python e bibliotecas para cada projeto. Pensa nele como uma gaveta separada para cada projeto: dentro da gaveta da Loja da Ana, você guarda a versão de `pandas` que aquele projeto precisa; em outra gaveta, outro projeto guarda a versão que ele precisa — e as duas gavetas nunca se misturam. `venv` é o módulo embutido do Python que cria essas “gavetas”.

Isolar dependências por projeto é uma das práticas mais básicas (e mais importantes) no dia a dia de quem trabalha com dados: evita o clássico problema de “no meu computador funciona” quando alguém tenta rodar seu código em outra máquina, e evita que atualizar uma biblioteca para um projeto novo quebre silenciosamente um projeto antigo.

Como funciona (com exemplo comentado)

Todos os comandos abaixo rodam no terminal, dentro da pasta do seu projeto:

```
# Cria um ambiente virtual chamado "venv" (o nome é uma convenção, pode se
# qualquer nome, mas "venv" é o mais usado) dentro da pasta atual do proje
python -m venv venv

# No Windows, ativa o ambiente virtual (PowerShell)
venv\Scripts\Activate.ps1

# No Windows, ativa o ambiente virtual (cmd.exe)
venv\Scripts\activate.bat

# No Mac/Linux, ativa o ambiente virtual
source venv/bin/activate

# Depois de ativado, o nome do ambiente aparece entre parênteses no início
# Linha do terminal, algo como:
# (venv) C:\Users\voce\Loja-da-ana>

# Com o ambiente ativado, qualquer "pip install" instala SÓ dentro dessa g
# sem afetar o Python do resto do computador nem outros projetos
pip install pandas==2.1.0

# Para sair do ambiente virtual e voltar ao Python "global"
deactivate
```

Passo a passo típico para começar um projeto novo:

1. Crie a pasta do projeto (ex: loja-da-ana-analise).
2. Dentro dela, rode `python -m venv venv` para criar o ambiente.
3. Ative o ambiente (`venv\Scripts\Activate.ps1` no Windows/PowerShell).
4. Instale as bibliotecas que o projeto precisa (`pip install pandas`).
5. Trabalhe normalmente — sempre que voltar a esse projeto em outro dia, ative o ambiente de novo antes de rodar ou instalar algo.
6. Ao terminar a sessão de trabalho, rode `deactivate` (opcional — fechar o terminal também “desativa” o ambiente).

Erros comuns de quem está começando

- Esquecer de ativar o ambiente virtual antes de instalar uma biblioteca — nesse caso, o `pip install` vai direto para o Python global, e não para a “gaveta” do projeto.
- Colocar a pasta `venv` dentro do controle de versão do projeto (ex: subir ela para o Git). Ela é pesada e específica da sua máquina — o correto é listar as bibliotecas em um `requirements.txt` (visto em 06-pip.md) e deixar cada pessoa criar seu próprio ambiente virtual.
- Achar que criar o ambiente virtual (`python -m venv venv`) já é suficiente. É preciso **ativar** o ambiente (`Activate.ps1` ou equivalente) toda vez que for usá-lo — criar só prepara a gaveta, ativar é “entrar” nela.

Exercício prático

No seu computador, dentro de uma pasta de teste:

1. Crie um ambiente virtual chamado `venv` com `python -m venv venv`.
2. Ative o ambiente (use o comando correspondente ao seu sistema operacional).
3. Confirme que o ambiente está ativo (o nome deve aparecer entre parênteses no terminal).
4. Instale uma biblioteca qualquer, por exemplo `pip install requests`.
5. Rode `pip list` e confirme que `requests` aparece na lista.
6. Desative o ambiente com `deactivate`.

Desafio bônus (opcional): com o ambiente ativado, rode `pip freeze > requirements.txt` (visto em 06-pip.md) e abra o arquivo gerado para ver o que foi salvo.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar com minhas próprias palavras por que isolar dependências por projeto é importante
- ☐ Conseguir criar e ativar um ambiente virtual no meu computador
- ☐ Entendo a diferença entre “criar” e “ativar” um ambiente virtual

ewpage

Conda (alternativa ao venv)

— Módulo 2 — Python Intermediário e Ambiente · Tópico 8 de 11

O que é e por que importa

No tópico anterior ([07-ambientes-virtuais-venv.md](#)) você viu que venv cria “gavetas” isoladas de bibliotecas Python por projeto. conda resolve o mesmo problema — isolar dependências por projeto — mas com uma diferença importante: conda não gerencia só bibliotecas Python, ele gerencia **pacotes de qualquer tipo**, incluindo o próprio Python (diferentes versões dele) e bibliotecas que dependem de código escrito em outras linguagens (como C ou Fortran), muito comuns em bibliotecas científicas e de dados.

Pensa assim: venv é como uma gaveta dentro de um guarda-roupa que já é seu (o Python que você já tem instalado). conda é como um guarda-roupa completo e independente, que você pode montar do zero, escolhendo até que “modelo” de Python vai morar ali dentro. Isso é útil especialmente em bibliotecas de dados e ciência (numpy, pandas, scipy) que às vezes dependem de componentes que não são Python puro — o conda sabe instalar essas peças extras sozinho, enquanto o pip/venv às vezes exige que você instale essas peças manualmente no sistema operacional.

conda vem geralmente através da distribuição **Anaconda** (ou sua versão mais enxuta, **Miniconda**) — um pacote de instalação que já traz Python, conda e várias bibliotecas de dados populares pré-instaladas, muito usado por quem trabalha com ciência de dados e machine learning.

Como funciona (com exemplo comentado)

Depois de instalar Anaconda ou Miniconda, os comandos rodam no terminal (no Windows, geralmente em um terminal específico chamado “Anaconda Prompt”):

```
# Cria um ambiente conda chamado "loja-da-ana", já especificando a versão
conda create --name loja-da-ana python=3.11

# Ativa o ambiente -- note que o comando é diferente do venv
conda activate loja-da-ana

# Com o ambiente ativado, instala pacotes -- pode usar "conda install" ou
conda install pandas numpy matplotlib

# Lista todos os ambientes conda já criados no seu computador
conda env list

# Lista os pacotes instalados no ambiente ativo no momento
conda list

# Desativa o ambiente atual, voltando ao ambiente "base"
conda deactivate

# Remove um ambiente que não é mais necessário
conda env remove --name loja-da-ana
```

Quando escolher um ou outro, na prática:

- Use **venv** quando seu projeto usa só bibliotecas Python “normais” (a maioria dos casos) e você já tem Python instalado — é mais leve e já vem embutido, sem precisar instalar nada extra.
- Use **conda** quando trabalha bastante com bibliotecas científicas pesadas (numpy, scipy, bibliotecas de machine learning), quando precisa alternar entre várias versões diferentes do próprio Python facilmente, ou quando alguma biblioteca exige componentes que não são Python puro e o pip sozinho tem dificuldade de instalar.

Para o restante desta apostila, venv é suficiente — mas é comum encontrar conda no mercado de trabalho, especialmente em times de ciência de dados, então vale reconhecer os comandos.

Erros comuns de quem está começando

- Achar que precisa escolher entre venv OU conda para sempre. É perfeitamente normal usar conda em um projeto e venv em outro, dependendo da necessidade.
- Misturar `conda install` e `pip install` sem entender a ordem — em geral, é mais seguro instalar tudo que der com `conda install` primeiro, e só usar `pip install` para o que não estiver disponível via conda.
- Esquecer de ativar o ambiente conda certo antes de instalar algo (igual ao erro equivalente no venv) — os comandos `conda install` sem um ambiente ativado adequado vão parar no ambiente errado.

Exercício prático

Sem precisar instalar nada (a menos que já tenha Anaconda/Miniconda e queira testar de verdade), escreva os comandos que você usaria para:

1. Criar um ambiente conda chamado `analise-vendas` com Python 3.10.
2. Ativar esse ambiente.
3. Instalar `pandas` e `matplotlib` dentro dele.
4. Listar todos os ambientes conda existentes no computador.

Desafio bônus (opcional): pesquise a diferença entre Anaconda e Miniconda e escreva, em uma frase, qual delas você instalaria em um computador com pouco espaço em disco e por quê.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Consigo explicar a principal diferença entre conda e venv
- ☐ Resolvi o exercício sem olhar a solução
- ☐ Sei em que situação faria mais sentido escolher conda em vez de venv

ewpage

VS Code (configuração básica para Python)

Módulo 2 — Python Intermediário e Ambiente · Tópico 9 de 11

O que é e por que importa

Você já instalou o VS Code no Módulo 1 (02-ambiente-de-desenvolvimento.md) para escrever seus primeiros scripts. Agora que você vai lidar com bibliotecas externas, ambientes virtuais e arquivos de dados, vale configurar o VS Code direito para que ele “converse” bem com tudo isso — em vez de só abrir arquivos de texto, ele pode entender seu ambiente Python, sugerir código, apontar erros antes de você rodar, e até rodar notebooks Jupyter (visto no próximo tópico) sem sair do editor.

Pensa no VS Code como um painel de controle: sozinho, ele é só um editor de texto genérico. As “extensões” são como aplicativos que você instala dentro dele para ganhar funcionalidades específicas — a extensão “Python” ensina o VS Code a entender arquivos .py, mostrar erros de sintaxe, sugerir autocompletar, e rodar código diretamente.

Um conceito chave aqui é o **interpretador**: é a instalação específica do Python que o VS Code vai usar para rodar seu código (pode ser o Python global do computador, ou um ambiente virtual específico de um projeto, como visto em 07-ambientes-virtuais-venv.md). Selecionar o interpretador errado é uma das causas mais comuns de “por que essa biblioteca que eu instalei não aparece no VS Code?”.

Como funciona (com exemplo comentado)

Passo a passo de configuração:

1. Abra o VS Code.
2. Vá até a aba de Extensões (ícone de blocos quadrados na barra lateral, ou atalho Ctrl+Shift+X).
3. Busque por "Python" (da Microsoft) e clique em "Install".
4. Enquanto estiver ali, busque também por "Jupyter" (da Microsoft) e instale -- ela permite abrir e rodar notebooks (.ipynb) direto dentro do VS Code, sem precisar abrir o JupyterLab separadamente (veremos no próximo tópico).
5. Abra a pasta do seu projeto (Menu > File > Open Folder), por exemplo a pasta "loja-da-ana" onde você já tem seu ambiente virtual criado.
6. Abra um arquivo .py qualquer dentro dessa pasta.
7. No canto inferior direito do VS Code, deve aparecer o nome de um interpretador Python (ex: "Python 3.11.4"). Clique nele.
8. Uma lista aparece com os interpretadores disponíveis -- escolha o que corresponde ao ambiente virtual do seu projeto (geralmente aparece como algo como ".\venv\bin\python" ou "venv\Scripts\python.exe").
9. Se o ambiente virtual do projeto não aparecer na lista, clique em "Enter interpreter path..." e navegue até o arquivo python.exe dentro da pasta venv do projeto.

10. Para rodar um arquivo `.py`: com o arquivo aberto, clique no botão de "play" (triângulo) no canto superior direito, ou use o atalho `Ctrl+F5`.
11. O resultado aparece no "terminal integrado" -- uma aba de terminal que já vem embutida no próprio VS Code (Menu > Terminal > New Terminal, ou atalho `Ctrl+``). Esse terminal já roda dentro da pasta do seu projeto e, se o interpretador certo estiver selecionado, já usa o ambiente virtual ativado automaticamente.

Depois de configurado, seu fluxo de trabalho fica assim: abrir a pasta do projeto no VS Code, conferir que o interpretador certo está selecionado (canto inferior direito), escrever o código, e rodar com `Ctrl+F5` ou pelo terminal integrado.

Erros comuns de quem está começando

- Instalar uma biblioteca com `pip install` no terminal integrado, mas o VS Code estar configurado para usar um interpretador Python diferente do que o terminal está usando — resultado: `ModuleNotFoundError` mesmo depois de instalar. A solução é sempre conferir, no canto inferior direito, se o interpretador selecionado é o mesmo ambiente onde você instalou a biblioteca.
- Abrir um arquivo `.py` solto (sem abrir a pasta inteira do projeto com "Open Folder") — isso faz o VS Code não enxergar corretamente o ambiente virtual e outros arquivos relacionados do projeto.
- Ignorar os sublinhados coloridos que a extensão Python desenha no código (avisos e erros antes mesmo de rodar) — vale passar o mouse por cima para entender o que está sendo sinalizado, em vez de só rodar e ver o erro acontecer.

Exercício prático

No seu computador:

1. Instale as extensões "Python" e "Jupyter" no VS Code.
2. Abra (ou crie) a pasta do projeto onde você tem um ambiente virtual criado (do exercício de 07-ambientes-virtuais-venv.md).
3. Abra um arquivo `.py` de teste e verifique, no canto inferior direito, qual interpretador está selecionado.
4. Troque para o interpretador do seu ambiente virtual, se ainda não estiver selecionado.
5. Rode o arquivo com `Ctrl+F5` e confirme que o resultado aparece no terminal integrado.

Desafio bônus (opcional): abra o terminal integrado (`Ctrl+``) e `rodepip list`` nele — confirme que a lista de bibliotecas bate com o ambiente virtual que você selecionou como interpretador.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Instalei as extensões Python e Jupyter no VS Code
- ☐ Consigo selecionar corretamente o interpretador de um ambiente virtual
- ☐ Consigo rodar um arquivo `.py` e ver o resultado no terminal integrado

ewpage

JupyterLab

O que é e por que importa

Até agora, você escreveu código em arquivos `.py` que rodam de cima para baixo, do início ao fim, toda vez. JupyterLab propõe outra forma de trabalhar: um **notebook** (arquivo `.ipynb`), que é dividido em pequenos blocos chamados **células**, cada uma podendo ser executada de forma independente e na ordem que você quiser.

Pensa num notebook como um caderno de anotações de laboratório: em vez de escrever um relatório inteiro de uma vez e só ver o resultado no final, você testa um pedaço, vê o resultado ali mesmo, ajusta, testa o próximo pedaço — sem precisar rodar tudo de novo desde o início cada vez que muda algo. Isso é exatamente o que análise de dados exige na prática: carregar os dados uma vez (processo que pode demorar), e depois ir testando cálculos e gráficos diferentes em cima deles, sem recarregar tudo a cada tentativa.

Existem dois tipos de célula: **células de código**, que rodam Python de verdade (e mostram o resultado logo abaixo, incluindo tabelas e gráficos), e **células de markdown**, que mostram texto formatado (títulos, listas, explicações) — úteis para documentar o raciocínio da análise junto com o código, como se fosse um relatório interativo.

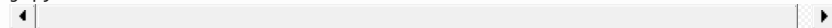
JupyterLab é ótimo para **explorar** dados — testar hipóteses, visualizar rapidamente, ajustar um filtro e ver o efeito na hora. Mas é ruim para **produção** (código que vai rodar automaticamente todo dia, por exemplo): notebooks não são feitos para isso, são mais difíceis de testar automaticamente, de versionar bem, e de rodar de forma confiável sem intervenção manual. Scripts `.py` (o que você já aprendeu) continuam sendo a escolha certa para automatizar processos.

Como funciona (com exemplo comentado)

Passo a passo para instalar e abrir:

```
# Com um ambiente virtual ativado (visto em 07-ambientes-virtuais-venv.md)
# instale o JupyterLab
pip install jupyterlab
```

```
# Rode o comando abaixo dentro da pasta do seu projeto para abrir o
# JupyterLab no navegador
jupyter lab
```

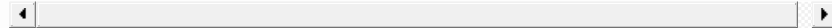


1. O comando "jupyter lab" abre uma aba no seu navegador com a interface do JupyterLab.
2. Clique em "Notebook" (sob Python 3) para criar um notebook novo (`.ipynb`).
3. Uma célula vazia aparece -- por padrão, é uma célula de código.
4. Escreva código Python nela e rode com Shift+Enter (roda a célula atual e já cria/pula para a próxima).
5. Para transformar uma célula em markdown, selecione a célula e mude o tipo no menu suspenso da barra de ferramentas (de "Code" para "Markdown").

Um exemplo do que aconteceria célula por célula, num notebook de análise da Loja da Ana:

```
# --- Célula 1 (código) ---
vendas_do_dia = [45.90, 120.00, 15.50, 200.00, 33.00]

# --- Célula 2 (código) ---
# Essa célula só funciona se a Célula 1 já tiver rodado antes --
# ela depende da variável "vendas_do_dia" já existir na memória do notebook
faturamento = sum(vendas_do_dia)
print("Faturamento do dia:", faturamento)
```



```
--- Célula 3 (markdown) ---  
## Análise do faturamento
```

O faturamento do dia foi calculado somando todas as vendas registradas.

A **ordem de execução** é o ponto mais importante de entender: o notebook não roda “de cima para baixo” automaticamente como um script — ele roda a célula que **você** mandar rodar, na ordem que você mandar. Ao lado de cada célula de código aparece um número entre colchetes, como [1], [2], [3], mostrando em que ordem aquela célula foi executada de verdade. Isso é poderoso (permite testar coisas fora de ordem, refazer só um pedaço), mas também é a maior fonte de confusão de quem está começando.

Erros comuns de quem está começando

- Rodar células fora de ordem e se confundir com o resultado. Se você rodar a Célula 2 antes da Célula 1, ou rodar a Célula 1 de novo depois de mudar uma variável em outro lugar, o estado da memória pode não bater com o que você vê na tela, escrito de cima para baixo.
- Achar que “salvar o notebook” garante que ele vai rodar do mesmo jeito depois. Um notebook salvo guarda o **resultado** da última execução de cada célula, mas não garante que rodar tudo de novo, na ordem das células, dê o mesmo resultado (variáveis podem ter sido alteradas fora de ordem).
- Usar notebooks para código que precisa rodar sozinho, todo dia, sem supervisão (como um relatório automático) — para isso, o ideal continua sendo um script `.py`, não um notebook.

Exercício prático

Instale o JupyterLab no seu ambiente virtual e crie um notebook novo. Nele:

1. Em uma célula de markdown, escreva um título tipo `## Análise de vendas – Loja da Ana`.
2. Em uma célula de código, crie uma lista `vendas_do_dia` com pelo menos 5 valores.
3. Em outra célula de código, calcule e imprima o faturamento total (soma da lista).
4. Em uma terceira célula de código, calcule e imprima a venda média (faturamento dividido pelo número de vendas).

Desafio bônus (opcional): rode a célula do faturamento total, depois volte e mude um valor da lista `vendas_do_dia` na primeira célula, sem rodar essa célula de novo. Rode a célula da venda média e observe: ela usa o valor antigo ou o novo? Explique por quê, com suas próprias palavras.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre célula de código e célula de markdown
- ☐ Entendo por que a ordem de execução pode ser diferente da ordem visual das células
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Google Colab

O que é e por que importa

Google Colab (ou “Colaboratory”) é um serviço do Google que roda notebooks Jupyter (o mesmo formato de células de código e markdown visto em `10-jupyterlab.md`) inteiramente na nuvem, direto no navegador — sem precisar instalar Python, JupyterLab, nem nenhuma biblioteca no seu computador.

Pensa no Colab como o Google Docs dos notebooks: assim como você abre o Google Docs no navegador e escreve um documento sem instalar o Word, você abre o Colab no navegador e escreve/roda código Python sem instalar nada. O Google mantém, “do outro lado”, um computador (o servidor) rodando o Python e as bibliotecas para você — inclusive já vem com pandas, numpy, matplotlib e várias outras bibliotecas de dados pré-instaladas, prontas para usar.

Isso torna o Colab uma porta de entrada excelente para quem está aprendendo (ou para testar uma ideia rapidamente): não existe barreira de instalação, funciona em qualquer computador com navegador e internet, é gratuito para uso básico, e os notebooks ficam salvos no seu Google Drive, acessíveis de qualquer lugar. A limitação principal é justamente a dependência da nuvem: sem internet, não tem Colab, e as sessões gratuitas **expiram** depois de um tempo de inatividade (ou depois de várias horas seguidas rodando) — quando isso acontece, você perde tudo que estava na memória (as variáveis calculadas), embora o texto do notebook em si continue salvo.

Como funciona (com exemplo comentado)

Passo a passo para começar a usar:

1. Acesse `colab.research.google.com` no navegador (é necessário ter uma conta Google).
2. Clique em “New Notebook” (Novo notebook) para criar um notebook em branco.
3. O notebook já abre com uma célula de código vazia -- funciona exatamente como no JupyterLab: Shift+Enter roda a célula atual.
4. Para adicionar uma célula de markdown, use o botão “+ Texto” na barra superior (ou “+ Code” para adicionar mais uma célula de código).
5. O notebook salva automaticamente no seu Google Drive, na pasta “Colab Notebooks”, conforme você trabalha -- não é preciso “salvar” manualmente com frequência, mas vale conferir de vez em quando.

O código dentro das células funciona igual ao que você já viu:

```
# Célula de código -- já roda direto, sem precisar instalar pandas antes,  
# pois o Colab já vem com as principais bibliotecas de dados pré-instalada  
import random
```

```
produtos = ["Caneta Azul", "Caderno", "Mochila", "Lapis HB"]  
random.seed(1)
```

```
vendas_simuladas = []  
for _ in range(5):  
    venda = {  
        "produto": random.choice(produtos),  
        "quantidade": random.randint(1, 5),  
    }  
    vendas_simuladas.append(venda)
```

```
for venda in vendas_simuladas:  
    print(venda)
```



```
# Se, mesmo assim, precisar de uma biblioteca que NÃO vem pré-instalada,  
# funciona igual ao pip normal -- só rodar num célula de código com um "!"  
na frente,  
# indicando que aquela linha é um comando de terminal, não Python:  
!pip install nome-da-biblioteca
```

Vantagens principais: gratuito (com limites de uso), zero instalação, acesso de qualquer computador com navegador, ótimo para compartilhar um notebook com outra pessoa (parecido com compartilhar um link do Google Docs) e ótimo para quem está começando e não quer lidar com configuração de ambiente ainda.

Limitações principais: precisa de internet o tempo todo, sessões gratuitas têm limite de tempo de uso contínuo e expiram por inatividade (perdendo o que estava na memória, precisando rodar as células de novo), e o poder de processamento das máquinas gratuitas é limitado comparado a rodar localmente em um computador potente.

Erros comuns de quem está começando

- Achar que, depois que a sessão expira, o notebook “perdeu tudo”. Só a **memória** (variáveis calculadas) se perde — o texto e código do notebook continuam salvos no Drive. Basta rodar as células de novo, na ordem certa, para recalculá-las.
- Fazer upload de arquivos de dados (CSV, JSON) direto na sessão do Colab e depois perder esses arquivos quando a sessão expira — arquivos enviados diretamente para a sessão não ficam salvos permanentemente. Para dados que você quer manter entre sessões, é melhor guardá-los no Google Drive e conectar o Drive ao notebook.
- Confundir Colab com JupyterLab achando que são coisas totalmente diferentes. Na prática, o formato do notebook (.ipynb, células de código e markdown, ordem de execução) é o mesmo — a diferença é só **onde** ele roda: no seu computador (JupyterLab) ou na nuvem do Google (Colab).

Exercício prático

Acesse o Google Colab (colab.research.google.com) e crie um notebook novo. Nele:

1. Em uma célula de markdown, escreva um título `## Simulação de vendas – Loja da Ana`.
2. Em uma célula de código, importe o módulo `random`, defina uma seed com `random.seed(5)`, e crie uma lista de 6 vendas simuladas (cada uma um dicionário com "produto" e "quantidade", reaproveitando a lógica de `02-modulo-random.md`).
3. Em outra célula, calcule a quantidade total vendida somando o campo "quantidade" de todas as vendas simuladas.

Desafio bônus (opcional): feche a aba do navegador, espere alguns minutos, e reabra o notebook pelo seu Google Drive. Rode as células de novo, na ordem, e confirme que o resultado é o mesmo de antes (graças à seed fixa).

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Consigo explicar a diferença entre Colab e JupyterLab
- ☐ Entendo o que se perde (e o que não se perde) quando uma sessão do Colab expira
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Módulo 3 — NumPy

Com Python “puro” e as ferramentas de ambiente dominadas (Módulos 1 e 2), chegou a hora da primeira biblioteca de dados de verdade: **NumPy**. Ela é a base sobre a qual o Pandas (Módulo 4) e praticamente todo o ecossistema científico de Python são construídos — entender bem NumPy torna tudo que vem depois muito mais fácil.

Deixamos a “Loja da Ana” de lado por um momento: NumPy trabalha com números em massa (preços, quantidades, medições), então os exemplos aqui usam dados numéricos mais diretos — nos módulos seguintes voltamos a aplicá-los a contextos de negócio.

Tópicos

1. Introdução ao NumPy
2. Criando arrays
3. Indexação e slicing
4. Operações vetorizadas e broadcasting
5. Funções estatísticas e agregações
6. Reshape e dimensões
7. Boolean masking e filtragem

Depois de terminar todos os tópicos, siga para o Módulo 4 — Pandas Essencial.

ewpage

Introdução ao NumPy

| Módulo 3 — NumPy · Tópico 1 de 7

O que é e por que importa

NumPy (*Numerical Python*) é uma biblioteca para trabalhar com **arrays**: coleções de números organizadas em linhas e colunas (ou mais dimensões), com operações matemáticas muito rápidas. É a fundação de quase tudo em ciência de dados com Python — Pandas, Matplotlib, scikit-learn e várias outras bibliotecas usam NumPy por baixo dos panos.

A pergunta óbvia é: por que não usar listas do Python, que você já conhece? Duas razões:

- **Velocidade.** Uma lista Python guarda referências para objetos espalhados na memória; um array NumPy guarda números “colados” uns nos outros, em blocos contíguos. Isso permite que o NumPy delegue as operações para código C otimizado, muito mais rápido que um loop Python percorrendo uma lista.
- **Conveniência.** Com listas, somar dois “vetores” elemento a elemento exige um loop ou uma list comprehension. Com arrays NumPy, você escreve `array1 + array2` e pronto — a operação é aplicada a todos os elementos de uma vez (isso se chama **vetorização**, e volta com mais detalhes no Tópico 4).

Instalação (caso ainda não tenha, veja `06-pip.md` do Módulo 2 para lembrar):

```
pip install numpy
```

A convenção universal é importar NumPy com o apelido `np`:

```
import numpy as np
```

Praticamente todo código NumPy que você vai ver (em tutoriais, documentação, Stack Overflow) usa esse apelido — vale a pena adotar desde já.

Como funciona (com exemplo comentado)

```

import numpy as np

# Uma Lista Python comum
precos_lista = [10.0, 25.5, 8.75, 42.0]

# Um array NumPy criado a partir da lista
precos_array = np.array(precos_lista)

print(precos_array)      # [10.  25.5  8.75 42.  ]
print(type(precos_array)) # <class 'numpy.ndarray'> -- "ndarray" = N-dime

# Diferença prática: aplicar um aumento de 10% em todos os preços

# Com lista, precisa de loop ou list comprehension:
precos_lista_novos = [p * 1.10 for p in precos_lista]

# Com array, a operação é aplicada direto, sem loop explícito:
precos_array_novos = precos_array * 1.10

print(precos_lista_novos) # [11.0, 28.05, 9.625, 46.2]
print(precos_array_novos) # [11.  28.05  9.625 46.2  ]

# Arrays NumPy também têm atributos úteis
print(precos_array.shape) # (4,) -- formato do array: 4 elementos, uma di
print(precos_array.dtype) # float64 -- tipo dos dados guardados no array
print(precos_array.ndim)  # 1 -- número de dimensões (1D, como uma lista

```

Um ponto importante: **um array NumPy só guarda um tipo de dado por vez** (todos int, todos float, etc.), diferente de uma lista Python, que pode misturar tipos livremente. Se você criar um array com valores mistos, o NumPy converte tudo para um tipo comum:

```

misto = np.array([1, 2.5, 3])
print(misto)      # [1.  2.5  3.  ] -- o inteiro 1 virou 1.0
print(misto.dtype) # float64

```

Erros comuns de quem está começando

- Confundir array NumPy com lista Python e tentar usar métodos de lista (como `.append()`) que não existem (ou funcionam diferente) em arrays — arrays têm tamanho fixo por padrão; “crescer” um array repetidamente é caro e normalmente sinal de que uma lista comum seria mais apropriada até o momento de converter para array.
- Esquecer o `import numpy as np` e tentar usar `numpy.array(...)` direto, ou usar um apelido diferente do padrão `np` — funciona, mas quebra a convenção que todo mundo espera ao ler seu código.
- Achar que `np.array([1, 2, 3])` e `[1, 2, 3]` se comportam igual em operações matemáticas. `[1, 2, 3] * 2` duplica a lista (`[1, 2, 3, 1, 2, 3]`); `np.array([1, 2, 3]) * 2` multiplica cada elemento por 2 (`[2, 4, 6]`) — comportamentos completamente diferentes para o mesmo operador.

Exercício prático

Crie uma lista Python com as temperaturas (em Celsius) de 5 dias: `[21.5, 19.0, 23.2, 18.7, 25.1]`. Converta essa lista para um array NumPy. Depois:

1. Imprima o `shape`, o `dtype` e o `ndim` do array.
2. Converta todas as temperaturas para Fahrenheit usando a fórmula $F = C * 9/5 + 32$, aplicando a operação direto no array (sem loop).
3. Imprima o resultado.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Entendo por que arrays NumPy são mais rápidos que listas para cálculos numéricos
- ☐ Sei importar NumPy com o apelido convencional `np`
- ☐ Consigo explicar a diferença entre `lista * 2` e `array * 2`
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Criando arrays

| Módulo 3 — NumPy · Tópico 2 de 7

O que é e por que importa

No tópico anterior você criou um array a partir de uma lista já pronta com `np.array()`. Mas na prática, muitas vezes você precisa **gerar** dados — sequências de números, arrays cheios de zeros para inicializar algo, valores igualmente espaçados para um gráfico. NumPy tem funções prontas para cada uma dessas situações, e usá-las é bem mais rápido (e menos propenso a erro) do que escrever loops para montar as listas na mão.

Também é comum trabalhar com arrays de mais de uma dimensão — pense numa planilha, com linhas e colunas: isso é um array **2D** em NumPy. Uma imagem em preto e branco, uma tabela de vendas por mês e por produto, uma matriz de distâncias entre cidades: tudo isso é naturalmente representado como array 2D (ou até 3D, 4D...).

Como funciona (com exemplo comentado)

```

import numpy as np

# np.arange: como o range() do Python, mas retorna um array
sequencia = np.arange(0, 10)          # [0 1 2 3 4 5 6 7 8 9]
sequencia_pares = np.arange(0, 20, 2) # [0 2 4 6 8 10 12 14 16 18] -- (iní

# np.zeros: array cheio de zeros -- útil para "reservar espaço" antes de p
zeros_1d = np.zeros(5)                # [0. 0. 0. 0. 0.]
zeros_2d = np.zeros((3, 4))          # matriz 3 linhas x 4 colunas, tudo zero

# np.ones: mesma ideia, mas cheio de 1
uns = np.ones((2, 3))                # matriz 2x3 cheia de 1.0

# np.full: array cheio de um valor qualquer escolhido por você
cheio = np.full((2, 2), 7)           # matriz 2x2 cheia de 7

# np.linspace: N valores igualmente espaçados entre um início e um fim (in
# muito usado para gerar eixos de gráficos
espaçados = np.linspace(0, 1, 5)     # [0.    0.25 0.5   0.75 1.   ] -- 5 valore

# np.random: números aleatórios (veremos mais no contexto de amostragem/si
aleatorios = np.random.rand(3)        # 3 números aleatórios entre 0 e 1
inteiros_aleatorios = np.random.randint(1, 100, size=5) # 5 inteiros entr

# Criando um array 2D direto de uma lista de listas
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6],
])
print(matriz.shape) # (2, 3) -- 2 Linhas, 3 colunas
print(matriz.ndim)  # 2

# Especificando o dtype explicitamente (útil para controlar memória/precis
inteiros = np.array([1, 2, 3], dtype=np.int32)
print(inteiros.dtype) # int32

```

Repare no padrão do shape para arrays 2D: (linhas, colunas). É fácil confundir a ordem no começo — sempre “linhas primeiro, colunas depois”, igual a como lemos uma tabela (primeiro em qual linha, depois em qual coluna).

Erros comuns de quem está começando

- Confundir a ordem dos argumentos em `np.arange(inicio, fim, passo)` com `np.linspace(inicio, fim, quantidade)` — no `arange` o terceiro argumento é o “pulo” entre os números; no `linspace` é **quantos números** você quer no total, não o espaçamento entre eles.
- Esquecer que `np.arange` (assim como `range()`) **não inclui** o valor final, enquanto `np.linspace` **inclui** por padrão. `np.arange(0, 5)` vai até 4; `np.linspace(0, 5, 6)` vai até 5.
- Passar o shape de zeros/ones/full sem parênteses quando é mais de uma dimensão: `np.zeros(3, 4)` dá erro — o correto é `np.zeros((3, 4))`, com o formato como uma única tupla.

Exercício prático

1. Crie um array com os números pares de 0 a 30 usando `np.arange`.
2. Crie uma matriz 3x3 cheia de zeros usando `np.zeros`.
3. Crie um array com 6 valores igualmente espaçados entre 10 e 20 (incluindo os dois extremos) usando `np.linspace`.
4. Para cada array criado, imprima o shape.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei a diferença entre `np.arange` e `np.linspace` (e qual usar em cada caso)
- ☐ Consigo criar arrays 2D e sei ler o `shape` como (linhas, colunas)
- ☐ Sei criar arrays preenchidos com zeros, uns ou um valor fixo
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Indexação e slicing

— Módulo 3 — NumPy · Tópico 3 de 7

O que é e por que importa

Você já sabe indexar e fatiar listas Python (`lista[0]`, `lista[1:3]`) — veja `16-listas.md` do Módulo 1 se precisar relembrar. Arrays NumPy usam a mesma ideia, mas estendida para múltiplas dimensões, com uma sintaxe que evita loops aninhados para acessar linhas, colunas ou sub-regiões de uma matriz.

Entender bem indexação é essencial: é como você vai “recortar” pedaços dos seus dados — pegar só uma coluna, só as primeiras 10 linhas, só os valores que atendem a uma condição (isso último vem com mais detalhes no Tópico 7).

Como funciona (com exemplo comentado)

```
import numpy as np

# Indexação em array 1D -- igual a Lista Python
notas = np.array([7.5, 8.0, 6.5, 9.0, 5.5])
print(notas[0])      # 7.5 -- primeiro elemento
print(notas[-1])     # 5.5 -- último elemento
print(notas[1:3])    # [8.  6.5] -- fatia do índice 1 até 3 (exclusivo)

# Indexação em array 2D -- use [linha, coluna], separado por vírgula
# (diferente de lista de listas em Python puro, onde seria lista[linha][co
matriz = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90],
])

print(matriz[0, 0])   # 10 -- linha 0, coluna 0
print(matriz[1, 2])   # 60 -- linha 1, coluna 2
print(matriz[2])      # [70 80 90] -- linha inteira (índice 2)

# Fatiamento também funciona com vírgula: [linhas, colunas]
print(matriz[:, 0])    # [10 40 70] -- todas as linhas, só a coluna 0
print(matriz[0, :])    # [10 20 30] -- só a linha 0, todas as colunas
print(matriz[0:2, 1:3]) # [[20 30] [50 60]] -- linhas 0-1, colunas 1-2

# Atenção: slicing em NumPy retorna uma "view" (referência), não uma cópia
fatia = notas[1:3]
fatia[0] = 999
print(notas) # [7.5 999. 6.5 9. 5.5] -- o array ORIGINAL também mudou!

# Para evitar isso, use .copy() explicitamente quando precisar de uma cópi
fatia_segura = notas[1:3].copy()
fatia_segura[0] = -1
print(notas) # não foi afetado dessa vez
```

Esse comportamento de “view” é diferente de listas Python (onde `lista[1:3]` sempre cria uma lista nova). É uma decisão de design do NumPy para performance — copiar dados o tempo todo seria caro para arrays grandes — mas é uma armadilha comum para quem vem de listas puras.

Erros comuns de quem está começando

- Usar `matriz[linha][coluna]` em vez de `matriz[linha, coluna]`. O primeiro até funciona (primeiro pega a linha, depois indexa nela), mas é mais lento e não é a forma idiomática em NumPy — prefira sempre a vírgula.
- Esquecer que slices são “views” e alterar uma fatia achando que o array original não seria afetado, causando bugs difíceis de rastrear.
- Trocar a ordem de linha e coluna: `matriz[:, 0]` (coluna 0 inteira) não é o mesmo que `matriz[0, :]` (linha 0 inteira) — sempre “linha primeiro, coluna depois”, igual ao `shape`.

Exercício prático

Crie a matriz abaixo com NumPy:

```
[[ 1,  2,  3,  4],
 [ 5,  6,  7,  8],
 [ 9, 10, 11, 12]]
```

1. Imprima o elemento da linha 2, coluna 3 (deve ser 11).
2. Imprima a segunda coluna inteira (deve ser `[2 6 10]`).
3. Imprima a submatriz formada pelas duas primeiras linhas e as duas últimas colunas.
4. Faça uma cópia da primeira linha, altere o primeiro valor da cópia para 100, e confirme (imprimindo a matriz original) que ela não foi afetada.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei indexar e fatiar arrays 1D e 2D usando a sintaxe `[linha, coluna]`
- ☐ Entendo a diferença entre “view” e cópia, e sei quando usar `.copy()`
- ☐ Consigo pegar uma linha, uma coluna ou uma submatriz específica de uma matriz
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Operações vetorizadas e broadcasting

— Módulo 3 — NumPy · Tópico 4 de 7

O que é e por que importa

No Tópico 1 você já viu um exemplo de **vetorização**: multiplicar um array inteiro por um número (`precos_array * 1.10`) sem precisar de loop. Esse é o recurso mais poderoso do NumPy, e vale entender com mais profundidade — é o que faz o código de análise de dados ser curto, legível e rápido.

Broadcasting é a regra que o NumPy usa para decidir como aplicar uma operação entre arrays de formatos (shapes) diferentes, “esticando” mentalmente o menor para combinar com o maior, sem realmente duplicar dados na memória. Entender broadcasting evita muita confusão (e muitos `ValueError`) ao operar com arrays de tamanhos diferentes.

Como funciona (com exemplo comentado)

```
import numpy as np

# Operações entre um array e um número (escalar) -- o número é aplicado a
precos = np.array([10.0, 20.0, 30.0])
print(precos + 5)      # [15. 25. 35.]
print(precos * 2)      # [20. 40. 60.]
print(precos ** 2)     # [100. 400. 900.]

# Operações entre dois arrays de MESMO shape -- elemento a elemento (posiç
quantidades = np.array([3, 1, 2])
print(precos * quantidades) # [30. 20. 60.] -- preco[0]*qtd[0], preco[1]*

# Funções matemáticas do NumPy também são vetorizadas
notas = np.array([7.2, 8.9, 5.5, 9.8])
print(np.round(notas))    # [7. 9. 6. 10.] -- arredonda cada elemento
print(np.sqrt(np.array([4, 9, 16]))) # [2. 3. 4.]

# Broadcasting: operação entre array 2D e array 1D de shape compatível
# Matriz de vendas: 3 produtos (linhas) x 4 meses (colunas)
vendas = np.array([
    [100, 120, 90, 130],
    [200, 210, 195, 220],
    [50, 60, 55, 58],
])

# Um array 1D com 4 valores (um por mês) é "esticado" para combinar com ca
ajuste_mensal = np.array([1.0, 1.05, 0.95, 1.10]) # shape (4,) -- combina
vendas_ajustadas = vendas * ajuste_mensal
print(vendas_ajustadas)
# cada COLUNA da matriz foi multiplicada pelo valor correspondente em ajus

# Comparação (operadores de comparação também são vetorizados)
print(precos > 15) # [False True True] -- array de booleanos, usado no
```

A regra de broadcasting, resumida: o NumPy compara os shapes dos dois arrays “de trás para frente” (da última dimensão para a primeira). Duas dimensões são compatíveis se forem iguais, ou se uma delas for 1 (nesse caso, o NumPy “repete” o valor). Se nenhuma condição valer, dá erro.

```
a = np.zeros((3, 4)) # shape (3, 4)
b = np.array([1, 2, 3, 4]) # shape (4,)
c = a + b # funciona: (4,) combina com a última dimensão de (3, 4)

d = np.array([1, 2, 3]) # shape (3,)
# e = a + d # ERRO: (3,) não combina com a última dimensão (4) de (3, 4)
```

Erros comuns de quem está começando

- Tentar somar/multiplicar dois arrays de shapes incompatíveis e não entender a mensagem de erro (ValueError: operands could not be broadcast together) — o primeiro passo para debugar é sempre imprimir o `.shape` dos dois arrays envolvidos.
- Usar loop for para aplicar uma operação matemática simples a um array, quando a versão vetorizada (`array * 2`, `np.sqrt(array)`, etc.) já resolve — além de mais lento, é um sinal de que “pensamento NumPy” ainda não aconteceu. Se você se pegar escrevendo `for i in range(len(array))` para fazer conta, pare e pergunte: “dá pra vetorizar isso?”.
- Achar que `array1 * array2` faz multiplicação de matrizes (álgebra linear). Em NumPy, `*` é sempre elemento a elemento; multiplicação de matrizes usa `@` ou `np.matmul()` — fora do escopo desta apostila, mas bom saber que existe essa diferença.

Exercício prático

Uma loja tem os preços de 4 produtos: [50.0, 120.0, 15.5, 300.0]. Ela vai aplicar um reajuste diferente para cada produto: [1.10, 1.05, 1.20, 0.95] (10% de aumento no primeiro, 5% no segundo, 20% no terceiro, 5% de desconto no quarto).

1. Calcule os novos preços com uma operação vetorizada (sem loop).
2. Calcule quanto cada produto aumentou ou diminuiu em valor absoluto (novo preço menos preço antigo).
3. Descubra quais produtos ficaram mais caros que R\$ 100,00 usando um operador de comparação (deve retornar um array de booleanos).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei aplicar operações matemáticas a um array inteiro sem usar loop
- ☐ Entendo a regra básica de broadcasting (comparar shapes de trás para frente)
- ☐ Sei que * entre arrays é elemento a elemento, não multiplicação de matrizes
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Funções estatísticas e agregações

Módulo 3 — NumPy · Tópico 5 de 7

O que é e por que importa

Agregação é reduzir um array a um único número (ou a um array menor) que resume seus dados: a soma, a média, o máximo, o desvio padrão. É uma das operações mais usadas em análise de dados — quase toda pergunta de negócio (“qual o faturamento total?”, “qual o produto mais vendido em média?”) vira, no código, uma chamada de função de agregação.

NumPy traz essas funções prontas e otimizadas, evitando que você reimplente soma ou média com loops. E quando os dados são 2D (matriz), você escolhe se quer agregar a matriz inteira, só as linhas, ou só as colunas — usando o parâmetro `axis`, que costuma confundir no início mas é simples uma vez que “clica”.

Como funciona (com exemplo comentado)

```

import numpy as np

notas = np.array([7.5, 8.0, 6.5, 9.0, 5.5])

print(np.sum(notas))      # 36.5 -- soma de tudo
print(np.mean(notas))     # 7.3 -- média
print(np.min(notas))      # 5.5
print(np.max(notas))      # 9.0
print(np.std(notas))      # ~1.19 -- desvio padrão (dispersão dos dados)
print(np.median(notas))   # 7.5 -- mediana

# Essas funções também existem como MÉTODO do array, forma equivalente e c
print(notas.sum())        # 36.5
print(notas.mean())       # 7.3

# argmin/argmax: retornam o ÍNDICE do menor/maior valor, não o valor em si
print(np.argmax(notas))   # 3 -- índice do maior valor (9.0 está na posição
print(np.argmin(notas))   # 4 -- índice do menor valor (5.5 está na posição

# Agregações em matrizes 2D -- aqui entra o parâmetro axis
vendas = np.array([
    [100, 120, 90, 130], # produto A, vendas em 4 meses
    [200, 210, 195, 220], # produto B
    [50, 60, 55, 58],     # produto C
])

print(np.sum(vendas))      # 1488 -- soma TUDO, vira um único número
print(np.sum(vendas, axis=0)) # [350 390 340 408] -- soma por COLUNA (som
print(np.sum(vendas, axis=1)) # [440 825 223] -- soma por LINHA (soma cad

# Truque para Lembrar o axis: axis=0 "colapsa" as linhas (anda na vertical
# resultando em um valor por coluna. axis=1 "colapsa" as colunas (anda na
# horizontal), resultando em um valor por linha.

print(np.mean(vendas, axis=1)) # [110. 206.25 55.75] -- média de vendas p
print(np.max(vendas, axis=0))  # [200 210 195 220] -- maior venda em cada

```

Erros comuns de quem está começando

- Trocar `axis=0` com `axis=1` e obter um resultado com o número errado de elementos, sem entender por quê. Se a dúvida bater, teste com uma matriz pequena e confira o shape do resultado: agregação com `axis=0` numa matriz (3, 4) retorna algo de shape (4,); com `axis=1`, shape (3,).
- Confundir `np.argmax/np.argmin` (que retornam o **índice**) com `np.max/np.min` (que retornam o **valor**) — usar um no lugar do outro é um erro sutil que não quebra o código, só dá resultado errado.
- Esquecer que `np.std` (desvio padrão) e `np.median` (mediana) já existem prontas e tentar calcular na mão — vale sempre checar se o NumPy já resolve antes de implementar algo do zero.

Exercício prático

Uma matriz representa notas de 3 alunos (linhas) em 4 provas (colunas):

```

notas = np.array([
    [7.0, 8.5, 6.0, 9.0],
    [5.5, 6.0, 7.5, 8.0],
    [9.0, 9.5, 8.5, 10.0],
])

```

1. Calcule a média geral de todas as notas (um único número).
2. Calcule a média de cada aluno (média por linha).
3. Calcule a média de cada prova (média por coluna).
4. Descubra o índice do aluno com a maior média (dica: primeiro calcule as médias por

aluno, depois use `np.argmax` nesse resultado).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Conheço as principais funções de agregação: `sum`, `mean`, `min`, `max`, `std`, `median`
- ☐ Entendo a diferença entre `argmax/argmin` e `max/min`
- ☐ Sei usar `axis=0` e `axis=1` para agregar por coluna ou por linha em matrizes
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Reshape e dimensões

| Módulo 3 — NumPy · Tópico 6 de 7

O que é e por que importa

Às vezes os dados chegam num formato (`shape`) diferente do que você precisa para trabalhar: uma lista “achatada” de 12 números que na verdade representa uma tabela de 3 linhas por 4 colunas, ou uma matriz que você precisa transpor (trocar linhas por colunas) para combinar com outra. **Reshape** é a operação de reorganizar os mesmos dados em um novo formato, sem alterar os valores nem precisar recriá-los.

Isso é puramente sobre **organização** dos dados na memória — nenhum número é criado ou perdido, só a “forma” como eles são agrupados muda.

Como funciona (com exemplo comentado)

```

import numpy as np

# Um array "achatado" (1D) com 12 elementos
dados = np.arange(12)
print(dados)          # [ 0  1  2  3  4  5  6  7  8  9 10 11]
print(dados.shape)    # (12,)

# reshape reorganiza os mesmos 12 valores em outro formato
matriz_3x4 = dados.reshape(3, 4)
print(matriz_3x4)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

matriz_4x3 = dados.reshape(4, 3)
print(matriz_4x3)
# [[ 0  1  2]
#  [ 3  4  5]
#  [ 6  7  8]
#  [ 9 10 11]]

# O total de elementos precisa "fechar a conta": 3*4 = 12, 4*3 = 12, ok.
# dados.reshape(3, 5) # ERRO: 3*5 = 15, não bate com os 12 elementos orig

# Truque útil: usar -1 em uma das dimensões para o NumPy calcular automati
auto = dados.reshape(3, -1) # "3 Linhas, quantas colunas precisar" -> vir
print(auto.shape) # (3, 4)

# flatten / ravel: o caminho inverso, transforma um array de várias dimens
de_volta = matriz_3x4.flatten()
print(de_volta) # [ 0  1  2  3  4  5  6  7  8  9 10 11]

# .T (transposta): troca linhas por colunas
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6],
])
print(matriz.shape) # (2, 3)
print(matriz.T)
# [[1 4]
#  [2 5]
#  [3 6]]
print(matriz.T.shape) # (3, 2)

# Assim como slicing, reshape geralmente retorna uma VIEW (referência aos
# mesmos dados), não uma cópia -- alterar o resultado pode afetar o origin
vista = dados.reshape(3, 4)
vista[0, 0] = 999
print(dados[0]) # 999 -- o array original também mudou

```

Erros comuns de quem está começando

- Tentar dar reshape para um formato cujo total de elementos não bate com o original ($3 \times 4 \neq 3 \times 5$), gerando `ValueError`. Antes de chamar `.reshape()`, vale conferir: `linhas * colunas` precisa ser igual ao número de elementos do array original (`.size`).
- Confundir reshape (reorganiza os dados existentes) com criar um array novo do zero com `np.zeros/np.array` — reshape não adiciona nem remove dados, só rearranja o que já existe.
- Esquecer que reshape (como slicing) costuma retornar uma view, e se surpreender quando alterar o array “remodelado” afeta o original. Use `.copy()` quando precisar de independência total, igual visto no Tópico 3.

Exercício prático

1. Crie um array 1D com os números de 1 a 20 usando `np.arange`.
2. Transforme esse array em uma matriz 4x5 usando `.reshape()`.
3. Transponha essa matriz (deve virar 5x4) e imprima o resultado.
4. Volte a matriz original (4x5) para o formato 1D usando `.flatten()` e confirme que os valores estão na ordem original (1 a 20).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei calcular se um reshape é possível (total de elementos precisa bater)
- ☐ Sei usar `-1` para deixar o NumPy calcular uma dimensão automaticamente
- ☐ Entendo o que `.T` (transposta) faz e sei quando reshape retorna uma view
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Boolean masking e filtragem

| Módulo 3 — NumPy · Tópico 7 de 7 (último do módulo)

O que é e por que importa

No Tópico 4 você viu que comparar um array com um valor (`precos > 15`) retorna um array de `True/False` — um para cada elemento. Esse array de booleanos é chamado de **máscara booleana** (*boolean mask*), e é a ferramenta central para **filtrar** dados em NumPy: “me dê só os elementos que atendem a essa condição”.

Essa técnica é extremamente importante porque é exatamente como Pandas filtra linhas de uma tabela (Módulo 4) — dominar o conceito aqui, com arrays simples, torna o Pandas muito mais intuitivo depois.

Como funciona (com exemplo comentado)

```

import numpy as np

notas = np.array([7.5, 8.0, 4.5, 9.0, 3.0, 6.0])

# Passo 1: criar a máscara -- um array de booleanos, mesmo tamanho do orig
mascara_aprovados = notas >= 6.0
print(mascara_aprovados) # [ True  True False  True False  True]

# Passo 2: usar a máscara para indexar o array -- retorna só os elementos
aprovados = notas[mascara_aprovados]
print(aprovados) # [7.5 8.  9.  6. ]

# Você não precisa guardar a máscara numa variável -- pode indexar direto
reprovados = notas[notas < 6.0]
print(reprovados) # [4.5 3. ]

# Combinando condições: use & (E) e | (OU) -- NÃO use "and"/"or" do Python
# que não funcionam elemento a elemento em arrays. Cada condição precisa d
medianos = notas[(notas >= 5.0) & (notas < 8.0)]
print(medianos) # [7.5 6. ]

extremos = notas[(notas < 5.0) | (notas >= 9.0)]
print(extremos) # [4.5 9.  3. ]

# np.where: retorna os ÍNDICES onde a condição é True (ou substitui valor
indices_aprovados = np.where(notas >= 6.0)
print(indices_aprovados) # (array([0, 1, 3, 5]),) -- tupla com os índices

# np.where também funciona como um "if/else vetorizado":
situacao = np.where(notas >= 6.0, "aprovado", "reprovado")
print(situacao) # ['aprovado' 'aprovado' 'reprovado' 'aprovado' 'reprova

# Filtragem também funciona em 2D, retornando um array 1D com os valores q
vendas = np.array([
    [100, 120, 90, 130],
    [200, 210, 195, 220],
    [50, 60, 55, 58],
])
vendas_altas = vendas[vendas > 150]
print(vendas_altas) # [200 210 195 220] -- perdeu a estrutura de matriz,

# Modificando valores que atendem a uma condição, direto pela máscara
notas_ajustadas = notas.copy()
notas_ajustadas[notas_ajustadas < 5.0] = 5.0 # "arredonda pra cima" quem
print(notas_ajustadas) # [7.5 8.  5.  9.  5.  6. ]

```

Erros comuns de quem está começando

- Usar `and/or` do Python em vez de `&/|` ao combinar condições em arrays. `and/or` esperam um único valor booleano, não um array inteiro deles, e geram erro (The truth value of an array... is ambiguous).
- Esquecer os parênteses ao redor de cada condição ao combinar com `&/|`: `notas >= 5.0 & notas < 8.0` (sem parênteses) dá erro de precedência de operadores — o correto é `(notas >= 5.0) & (notas < 8.0)`.
- Confundir `np.where(condicao)` (retorna índices) com indexação booleana direta `array[condicao]` (retorna os valores) — os dois resolvem problemas parecidos mas retornam coisas diferentes.

Exercício prático

Uma loja registrou o faturamento diário (em reais) de 10 dias:

```

faturamento = np.array([1200, 850, 2100, 400, 1750, 3000, 620, 990, 1450,

```

1. Crie uma máscara booleana para os dias em que o faturamento foi maior que R\$ 1500.
2. Use a máscara para obter um array só com esses valores.
3. Conte quantos dias tiveram faturamento maior que R\$ 1500 (dica: `True` vale 1 e `False` vale 0 em operações de soma — `np.sum()` numa máscara conta os `True`).
4. Use `np.where` para criar um array de textos classificando cada dia como "bom" (`>= 1500`) ou "fraco" (`< 1500`).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei criar uma máscara booleana comparando um array com uma condição
- ☐ Sei filtrar um array usando a máscara (`array[mascara]`)
- ☐ Uso `&|` com parênteses (nunca `and/or`) para combinar condições em arrays
- ☐ Sei a diferença entre `np.where(condicao)` (índices) e `array[condicao]` (valores)
- ☐ Resolvi o exercício sem olhar a solução

Checklist do Módulo 3

Antes de seguir para o Módulo 4 (Pandas), confirme que você consegue, sem consultar a apostila:

- ☐ Criar arrays com `np.array`, `np.arange`, `np.zeros`, `np.ones` e `np.linspace`
- ☐ Indexar e fatiar arrays 1D e 2D
- ☐ Aplicar operações matemáticas a um array inteiro sem usar loop
- ☐ Explicar, em uma frase, o que é broadcasting
- ☐ Calcular soma, média, mínimo, máximo e desvio padrão, inclusive por linha/coluna com `axis`
- ☐ Usar `reshape` para reorganizar um array e `.T` para transpor uma matriz
- ☐ Filtrar um array usando uma máscara booleana

Pronto? Siga para o Módulo 4 — Pandas Essencial.

ewpage

Módulo 4 — Pandas Essencial

Com NumPy no repertório (Módulo 3), chegou a hora do **Pandas** — a biblioteca que profissionais de dados usam no dia a dia para carregar, explorar, filtrar, transformar e resumir tabelas de dados. Pandas é construído em cima do NumPy: uma coluna de um `DataFrame` é, por baixo, um array NumPy — então tudo que você aprendeu no Módulo 3 (vetorização, broadcasting, boolean masking) continua valendo aqui, só que aplicado a tabelas com nomes de coluna e rótulos, não apenas números soltos.

A partir daqui, além da “Loja da Ana”, os exemplos também usam **datasets públicos reais** (sempre com uma versão inline de backup no próprio tópico, caso você não consiga baixar nada) — é hora de praticar com dados do jeito que eles aparecem no mundo real: um pouco bagunçados, com colunas demais e nem tudo do jeito que você gostaria.

Tópicos

1. Introdução ao Pandas: Series e DataFrame
2. Lendo e escrevendo dados (CSV, Excel, JSON)
3. Explorando um DataFrame
4. Seleção e indexação (`loc`, `iloc`)

5. Filtragem de dados
6. Criando e modificando colunas
7. Ordenação
8. Agrupamento com groupby
9. Agregações com agg
10. Merge, join e concatenação
11. Introdução a valores ausentes
12. Tabelas dinâmicas (pivot_table)

Depois de terminar todos os tópicos, siga para o Módulo 5 — Limpeza de Dados.

ewpage

Introdução ao Pandas: Series e DataFrame

Módulo 4 — Pandas Essencial · Tópico 1 de 12

O que é e por que importa

Pandas é a biblioteca de Python para trabalhar com dados em formato de **tabela** — como uma planilha do Excel, mas manipulável por código, com operações rápidas (graças ao NumPy por baixo) e prontas para as tarefas mais comuns de análise de dados: filtrar, ordenar, agrupar, juntar tabelas, calcular estatísticas.

Pandas tem duas estruturas de dados centrais:

- **Series**: uma coluna só, uma sequência de valores com um **índice** (rótulo) associado a cada um. Pense nela como um array NumPy 1D, mas onde cada posição tem um “nome” além do número da posição.
- **DataFrame**: uma tabela inteira — várias colunas (cada uma uma Series), todas compartilhando o mesmo índice de linhas. É a estrutura que você vai usar o tempo todo.

Instalação (se ainda não tiver):

```
pip install pandas
```

A convenção universal é importar com o apelido `pd`:

```
import pandas as pd
```

Como funciona (com exemplo comentado)

```

import pandas as pd

# Series: uma coluna de dados com índice
precos = pd.Series([10.0, 25.5, 8.75, 42.0])
print(precos)
# 0    10.00
# 1    25.50
# 2     8.75
# 3    42.00
# dtype: float64
# -- a coluna à esquerda (0, 1, 2, 3) é o ÍNDICE, criado automaticamente

# Você pode dar nomes ao índice em vez de usar números
precos_nomeados = pd.Series(
    [10.0, 25.5, 8.75],
    index=["Caneta", "Caderno", "Lapis"],
)
print(precos_nomeados)
# Caneta    10.00
# Caderno   25.50
# Lapis      8.75
# dtype: float64

print(precos_nomeados["Caderno"]) # 25.5 -- acessa pelo rótulo do índice

# DataFrame: criado a partir de um dicionário, onde cada chave vira uma co
loja = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB"],
    "quantidade": [3, 1, 2, 5],
    "preco_unitario": [2.50, 15.90, 89.90, 1.20],
})

print(loja)
#      produto  quantidade  preco_unitario
# 0  Caneta Azul           3           2.50
# 1    Caderno           1          15.90
# 2   Mochila           2          89.90
# 3  Lapis HB           5           1.20

# Cada coluna de um DataFrame é uma Series
print(type(loja["produto"])) # <class 'pandas.core.series.Series'>
print(loja["produto"])
# 0    Caneta Azul
# 1      Caderno
# 2     Mochila
# 3    Lapis HB
# Name: produto, dtype: object

# Atributos úteis, parecidos com os de arrays NumPy (Módulo 3)
print(loja.shape) # (4, 3) -- 4 linhas, 3 colunas
print(loja.columns) # Index(['produto', 'quantidade', 'preco_unitario'], d
print(loja.index) # RangeIndex(start=0, stop=4, step=1)
print(loja.dtypes) # tipo de cada coluna (object, int64, float64, ...)

```

Repare que o dtype de colunas de texto aparece como object — é como o Pandas representa strings (por baixo, guarda referências a objetos Python, diferente das colunas numéricas que usam arrays NumPy “puros” de int64/float64).

Erros comuns de quem está começando

- Confundir Series com DataFrame: `loja["produto"]` (uma coluna) é uma Series; `loja[["produto"]]` (colchete duplo, com uma lista dentro) é um DataFrame de uma coluna só. A diferença de colchetes simples/duplos aparece bastante em Pandas e vale prestar atenção.
- Achar que o índice de um DataFrame é sempre uma sequência de números 0, 1, 2... — ele pode ser texto, datas, ou qualquer coisa, e algumas operações (como as que vêm

nos próximos tópicos) dependem de entender qual é o índice atual.

- Esquecer o `import pandas as pd` ou usar um apelido fora do padrão — assim como `np` para NumPy, `pd` é universalmente esperado ao ler código Pandas.

Exercício prático

1. Crie uma Series com os índices ["Segunda", "Terça", "Quarta", "Quinta", "Sexta"] e valores [1200, 850, 2100, 400, 1750] representando o faturamento diário da Loja da Ana.
2. Acesse e imprima o valor de “Quarta” usando o rótulo do índice.
3. Crie um DataFrame com colunas `produto` (["Caneta", "Caderno", "Mochila"]), `preco` ([2.5, 15.9, 89.9]) e `estoque` ([100, 40, 15]).
4. Imprima o shape e os dtypes do DataFrame criado.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei a diferença entre Series (uma coluna) e DataFrame (uma tabela)
- ☐ Sei criar um DataFrame a partir de um dicionário
- ☐ Entendo o que é o índice e sei acessar valores por ele
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Lendo e escrevendo dados (CSV, Excel, JSON)

| Módulo 4 — Pandas Essencial · Tópico 2 de 12

O que é e por que importa

Na prática, você quase nunca digita os dados na mão como no tópico anterior — eles vêm de um arquivo (CSV exportado de um sistema, uma planilha Excel, uma resposta de API em JSON) ou de um banco de dados. Pandas tem funções `read_*` prontas para carregar os formatos mais comuns direto num DataFrame, e `to_*` para salvar de volta.

Esse é o ponto em que a apostila passa a usar **datasets públicos reais** além da Loja da Ana. Sempre que um exemplo depender de baixar um arquivo da internet, ele também trará uma versão inline (uma string ou dicionário) como backup, para o caso de você estar sem internet ou o link não funcionar mais.

Como funciona (com exemplo comentado)

```

import pandas as pd

# Lendo um CSV (o mais comum em análise de dados)
# df = pd.read_csv("vendas.csv")

# Exemplo prático: sem depender de um arquivo real, simulamos um CSV com S
from io import StringIO

csv_texto = """produto,quantidade,preco_unitario
Caneta Azul,3,2.50
Caderno,1,15.90
Mochila,2,89.90
Lapis HB,5,1.20
"""

df = pd.read_csv(StringIO(csv_texto))
print(df)
#      produto  quantidade  preco_unitario
# 0  Caneta Azul           3           2.50
# 1    Caderno           1          15.90
# 2    Mochila           2          89.90
# 3    Lapis HB           5           1.20

# Parâmetros comuns do read_csv:
# pd.read_csv("arquivo.csv", sep=";")          -- separador diferente de v
# pd.read_csv("arquivo.csv", encoding="latin1") -- arquivos com acentuação
# pd.read_csv("arquivo.csv", index_col=0)       -- usa a primeira coluna c
# pd.read_csv("arquivo.csv", usecols=["produto", "preco_unitario"]) -- só
# pd.read_csv("arquivo.csv", nrows=100)        -- só as 100 primeiras lin

# Salvando de volta para CSV
df.to_csv("vendas_salvas.csv", index=False)
# index=False evita salvar o índice numérico (0, 1, 2...) como uma coluna
# -- normalmente é o que você quer, a menos que o índice tenha significado

# Lendo Excel (precisa da biblioteca openpyxl: pip install openpyxl)
# df_excel = pd.read_excel("vendas.xlsx", sheet_name="Vendas2024")
# df.to_excel("vendas_salvas.xlsx", index=False, sheet_name="Vendas")

# Lendo JSON
import json

dados_json = json.dumps([
    {"produto": "Caneta Azul", "quantidade": 3, "preco_unitario": 2.50},
    {"produto": "Caderno", "quantidade": 1, "preco_unitario": 15.90},
])
df_json = pd.read_json(StringIO(dados_json))
print(df_json)

# Salvando em JSON
df_json.to_json("vendas.json", orient="records", indent=2)
# orient="records" gera uma lista de objetos (o formato mais comum de JSON)

# Lendo direto de uma URL (quando o dataset é público) -- read_csv aceita
# df_publico = pd.read_csv("https://exemplo.com/dataset.csv")

```

Erros comuns de quem está começando

- Não passar `index=False` ao salvar com `to_csv/to_excel`, gerando uma coluna extra sem nome (Unnamed: 0) toda vez que o arquivo é lido de volta — isso acontece porque o índice do DataFrame foi salvo como se fosse dado.
- Assumir que o separador é sempre vírgula. Muitos CSVs exportados no Brasil (Excel em português, por exemplo) usam ponto e vírgula (;) como separador — se as colunas aparecerem todas “grudadas” numa só ao ler, tente `sep=";"`.
- Esquecer de instalar a dependência extra do Excel (`openpyxl` para `.xlsx`) — Pandas lê CSV nativamente, mas Excel e alguns outros formatos exigem uma biblioteca auxiliar

instalada à parte.

Exercício prático

1. Usando `StringIO` (sem precisar de um arquivo real), crie um CSV inline com colunas `nome,idade,cidade` e pelo menos 3 linhas de dados fictícios. Carregue-o com `pd.read_csv`.
2. Salve esse DataFrame em um arquivo `pessoas.csv` de verdade, usando `index=False`.
3. Leia o arquivo `pessoas.csv` de volta com `pd.read_csv` e confirme (imprimindo) que os dados batem com o original.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei ler um CSV com `pd.read_csv` e ajustar `sep/encoding` quando necessário
- ☐ Sei salvar um DataFrame com `to_csv(..., index=False)` e explicar por que o `index=False` importa
- ☐ Sei que Excel e outros formatos podem exigir bibliotecas extras instaladas
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Explorando um DataFrame

Módulo 4 — Pandas Essencial · Tópico 3 de 12

O que é e por que importa

Antes de filtrar, transformar ou calcular qualquer coisa, o primeiro passo com qualquer dataset novo é **olhar para ele**: quantas linhas e colunas tem, que tipo de dado cada coluna guarda, se há valores faltando, quais são os valores típicos. Pandas tem um punhado de métodos feitos exatamente para essa “primeira olhada” — usá-los sempre no início evita surpresas mais tarde (como descobrir no meio de uma análise que uma coluna que parecia numérica na verdade é texto).

Como funciona (com exemplo comentado)

```
import pandas as pd
from io import StringIO

csv_texto = """produto,categoria,quantidade,preco_unitario,em_estoque
Caneta Azul,Papelaria,3,2.50,True
Caderno,Papelaria,1,15.90,True
Mochila,Acessorios,2,89.90,False
Lapis HB,Papelaria,5,1.20,True
Estojo,Acessorios,,25.00,True
"""

df = pd.read_csv(StringIO(csv_texto))

# .head(n) -- primeiras n linhas (padrão 5) -- a primeira coisa a rodar se
print(df.head(3))

# .tail(n) -- últimas n linhas
print(df.tail(2))
```

```

# .shape -- (linhas, colunas), igual ao shape de arrays NumPy (Módulo 3)
print(df.shape) # (5, 5)

# .info() -- resumo: tipos de cada coluna, quantos valores não-nulos, uso
df.info()
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 5 entries, 0 to 4
# Data columns (total 5 columns):
# #   Column          Non-Null Count  Dtype
# ---  ---
# 0   produto         5 non-null      object
# 1   categoria        5 non-null      object
# 2   quantidade       4 non-null      float64    <- repare: 4, não 5! tem
# 3   preco_unitario   5 non-null      float64
# 4   em_estoque       5 non-null      bool
# -- .info() já é uma forma rápida de detectar valores ausentes (mais sobr

# .describe() -- estatísticas resumidas das colunas numéricas
print(df.describe())
#          quantidade  preco_unitario
# count    4.000000      5.000000
# mean      2.750000     26.900000
# std       1.707825     35.499...
# min       1.000000     1.200000
# 25%      1.750000     2.500000
# 50%      2.500000    15.900000
# 75%      3.500000    40.225000
# max       5.000000    89.900000
# -- é o mesmo tipo de agregação vista no Módulo 3 (mean, min, max, std),
#    o Pandas já calcula tudo de uma vez e organiza numa tabela

# .describe(include="object") -- estatísticas para colunas de texto
print(df.describe(include="object"))
#          produto  categoria
# count          5          5
# unique         5          2
# top    Caneta Azul  Papelaria
# freq          1          3

# .dtypes -- tipo de cada coluna
print(df.dtypes)

# .columns e .index -- nomes das colunas e do índice
print(df.columns.tolist()) # ['produto', 'categoria', 'quantidade', 'preco

# .value_counts() -- conta quantas vezes cada valor aparece numa coluna (5
print(df["categoria"].value_counts())
# Papelaria      3
# Acessorios     2
# Name: count, dtype: int64

# .nunique() -- quantos valores ÚNICOS existem numa coluna
print(df["categoria"].nunique()) # 2

# .isnull().sum() -- conta valores ausentes por coluna (mais detalhes no T
print(df.isnull().sum())
# produto      0
# categoria    0
# quantidade   1
# preco_unitario 0
# em_estoque   0

```

Erros comuns de quem está começando

- Pular direto para filtros e cálculos sem antes rodar `.head()`, `.info()` e `.describe()` — isso costuma resultar em descobrir tarde demais que uma coluna tem valores

ausentes, tipos errados, ou nomes de coluna diferentes do esperado.

- Confundir `.describe()` (estatísticas resumidas) com `.info()` (tipos e contagem de não-nulos) — são complementares, não a mesma coisa: `.info()` responde “o que tem aqui e está tudo preenchido?”, `.describe()` responde “como esses números se distribuem?”.
- Esquecer que `.describe()` por padrão só olha colunas numéricas — colunas de texto ficam de fora a menos que você passe `include="object"` (ou `include="all"` para ver tudo junto).

Exercício prático

Usando o `df` do exemplo acima (produtos da loja):

1. Rode `.info()` e identifique qual coluna tem valores ausentes e quantos.
2. Rode `.describe()` e identifique qual produto tem o maior preço unitário olhando o valor de `max`.
3. Use `.value_counts()` para descobrir quantos produtos existem em cada categoria.
4. Use `.nunique()` para descobrir quantas categorias distintas existem.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sempre rodo `.head()`, `.info()` e `.describe()` ao carregar um dataset novo
- ☐ Sei ler a saída de `.info()` para identificar tipos e valores ausentes
- ☐ Sei usar `.value_counts()` e `.nunique()` para explorar colunas categóricas
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Seleção e indexação (`loc`, `iloc`)

| Módulo 4 — Pandas Essencial · Tópico 4 de 12

O que é e por que importa

Selecionar pedaços de um `DataFrame` — uma coluna, algumas linhas, uma célula específica — é algo que você vai fazer o tempo todo. Pandas tem várias formas de fazer isso, e a mais confiável (a que sempre funciona do jeito esperado) é usar `.loc[]` e `.iloc[]`:

- `.loc[]` seleciona por **rótulo** (label): o nome do índice, o nome da coluna.
- `.iloc[]` seleciona por **posição** (integer location): o número da linha/coluna, começando em 0 — igual à indexação de listas e arrays NumPy (Módulo 3).

Entender a diferença evita um dos erros mais comuns em Pandas: usar colchetes simples (`df[...]`) de forma ambígua e obter um resultado inesperado.

Como funciona (com exemplo comentado)

```
import pandas as pd

df = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB"],
    "quantidade": [3, 1, 2, 5],
    "preco_unitario": [2.50, 15.90, 89.90, 1.20],
}, index=["p1", "p2", "p3", "p4"]) # índice com rótulos de texto, não núm

# Selecionar UMA coluna -- retorna uma Series
print(df["produto"])

# Selecionar VÁRIAS colunas -- colchete duplo, retorna um DataFrame
print(df[["produto", "preco_unitario"]])

# .loc -- seleciona por RÓTULO (nome do índice e/ou nome da coluna)
print(df.loc["p2"]) # Linha inteira com índice "p2" (retorna
print(df.loc["p2", "produto"]) # "Caderno" -- célula específica
print(df.loc["p1":"p3"]) # Linhas de "p1" até "p3" -- ATENÇÃO: com
print(df.loc[:, "produto"]) # todas as linhas, só a coluna "produto"
print(df.loc[["p1", "p3"], ["produto", "quantidade"]]) # Linhas e colunas

# .iloc -- seleciona por POSIÇÃO (números, como em listas/arrays)
print(df.iloc[0]) # primeira linha (posição 0), seja qual for o r
print(df.iloc[0, 1]) # linha 0, coluna 1 (quantidade da primeira lin
print(df.iloc[0:2]) # Linhas 0 e 1 -- com .iloc, o fim é EXCLUSIVO
print(df.iloc[:, 0]) # todas as linhas, coluna de posição 0 (produto
print(df.iloc[[0, 2], [0, 1]]) # Linhas 0 e 2, colunas 0 e 1

# Um erro comum: usar colchete simples com número em índice não-numérico
# df[0] # ERRO -- o índice aqui é texto ("p1", "p2"...), não existe colun

# Quando o índice É numérico (o padrão, 0, 1, 2...), .loc e .iloc podem pa
# iguais, mas ainda se comportam diferente com fatias:
df_padrao = df.reset_index(drop=True) # volta para índice numérico 0,1,2,
print(df_padrao.loc[0:2]) # Linhas 0, 1 E 2 (fim inclusivo)
print(df_padrao.iloc[0:2]) # Linhas 0 e 1 apenas (fim exclusivo)
```

Erros comuns de quem está começando

- Usar `df[0]` esperando pegar a primeira linha — colchete simples com um índice de texto tenta achar uma **coluna** chamada 0, e dá erro. Para pegar linhas por posição, use `.iloc[0]`.
- Esquecer que `.loc` inclui o valor final em fatias (`df.loc["p1":"p3"]` inclui "p3"), enquanto `.iloc` exclui (`df.iloc[0:2]` não inclui a posição 2) — a mesma diferença de comportamento entre slicing “por rótulo” e “por posição” que existe em outras partes do Pandas.
- Misturar rótulo e posição no mesmo `.loc/.iloc`, como `df.loc[0]` num DataFrame cujo índice não é numérico (não existe rótulo 0) — sempre confirme qual é o índice atual do DataFrame (`df.index`) antes de usar `.loc`.

Exercício prático

```
df = pd.DataFrame({
    "cidade": ["São Paulo", "Rio de Janeiro", "Belo Horizonte", "Salvador"],
    "populacao_milhoes": [12.3, 6.7, 2.5, 2.9],
    "regiao": ["Sudeste", "Sudeste", "Sudeste", "Nordeste"],
}, index=["c1", "c2", "c3", "c4"])
```

1. Use `.loc` para selecionar a linha da cidade "c3".
2. Use `.iloc` para selecionar a última linha do DataFrame, sem usar o número 3 diretamente (dica: -1 funciona em `.iloc` como em listas).
3. Use `.loc` para selecionar só as colunas cidade e regiao das linhas "c1" e "c4".
4. Use `.iloc` para pegar a célula da linha de posição 1, coluna de posição 1 (deve ser

6.7).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei quando usar `.loc` (por rótulo) e quando usar `.iloc` (por posição)
- ☐ Sei a diferença de comportamento de fatias entre os dois (fim inclusivo x exclusivo)
- ☐ Sei selecionar uma coluna, várias colunas, uma linha e uma célula específica
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Filtragem de dados

| Módulo 4 — Pandas Essencial · Tópico 5 de 12

O que é e por que importa

Filtrar é selecionar só as linhas que atendem a alguma condição: “só os produtos com estoque baixo”, “só as vendas de junho”, “só os clientes de São Paulo”. É provavelmente a operação mais usada em análise de dados no dia a dia, e funciona com o **mesmo princípio de boolean masking** que você já viu em arrays NumPy no Módulo 3 (Tópico 7) — a diferença é que agora a máscara filtra linhas inteiras de uma tabela, não só números soltos.

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB", "Estojo"]
    "categoria": ["Papeleria", "Papeleria", "Acessorios", "Papeleria", "Ac
    "quantidade": [3, 1, 2, 5, 4],
    "preco_unitario": [2.50, 15.90, 89.90, 1.20, 25.00],
})

# Passo 1: criar a máscara -- comparar uma coluna com um valor retorna uma
mascara = df["preco_unitario"] > 10
print(mascara)
# 0    False
# 1     True
# 2     True
# 3    False
# 4     True
# Name: preco_unitario, dtype: bool

# Passo 2: usar a máscara para filtrar o DataFrame
caros = df[mascara]
print(caros)

# Na prática, quase sempre se escreve direto, sem variável intermediária:
caros = df[df["preco_unitario"] > 10]
baratos = df[df["preco_unitario"] <= 10]

# Combinando condições: & (E) e | (OU) -- assim como em NumPy, cada condiç
# precisa de parênteses, e NUNCA use "and"/"or" do Python puro aqui
papeleria_barata = df[(df["categoria"] == "Papeleria") & (df["preco_unitar
print(papeleria_barata)

caros_ou_papeleria = df[(df["preco_unitario"] > 50) | (df["categoria"] ==
print(caros_ou_papeleria)

# .isin() -- filtra por uma lista de valores possíveis (equivalente a vári
selecionados = df[df["produto"].isin(["Caneta Azul", "Mochila"])]
print(selecionados)

# ~ (til) -- inverte a condição (equivalente a "NÃO")
nao_papeleria = df[~(df["categoria"] == "Papeleria")]
print(nao_papeleria) # o mesmo que df[df["categoria"] != "Papeleria"]

# .query() -- forma alternativa, com a condição como string (alguns acham
resultado = df.query("preco_unitario > 10 and categoria == 'Papeleria'")
print(resultado)
# repare que dentro de .query() SE usa "and"/"or" normalmente -- é uma str
# interpretada pelo Pandas, não código Python puro sendo avaliado direto

# Filtrando e já selecionando colunas específicas do resultado
nomes_dos_caros = df.loc[df["preco_unitario"] > 10, "produto"]
print(nomes_dos_caros) # só a coluna "produto" das linhas filtradas

```

Erros comuns de quem está começando

- Usar and/or em vez de &/| ao combinar condições fora do .query() — mesmo erro visto com arrays NumPy, e pela mesma razão: essas condições retornam uma Series inteira de booleanos, não um único True/False.
- Esquecer os parênteses ao redor de cada condição: df[df["preco"] > 10 & df["categoria"] == "X"] (sem parênteses) quebra por causa da precedência de operadores — sempre (condicao1) & (condicao2).
- Comparar strings com case sensitivity diferente do esperado — "papeleria" != "Papeleria" para o Pandas. Se a filtragem por texto não estiver encontrando nada, vale conferir com .unique() os valores exatos que existem na coluna antes de assumir que a condição está errada.

Exercício prático

```
df = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB", "Estojo",
    "categoria": ["Papeleria", "Papeleria", "Acessorios", "Papeleria", "Ac
    "quantidade": [3, 1, 2, 5, 4, 10],
    "preco_unitario": [2.50, 15.90, 89.90, 1.20, 25.00, 0.90],
})
```

1. Filtre os produtos com quantidade maior que 3.
2. Filtre os produtos da categoria "Papeleria" com preco_unitario menor que 2.
3. Filtre os produtos que são "Mochila" ou "Estojo" usando `.isin()`.
4. Usando `.loc`, obtenha só a coluna produto dos itens com quantidade maior ou igual a 4.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei filtrar um DataFrame com uma condição simples e com condições combinadas (&/|)
- ☐ Sei usar `.isin()` para filtrar por uma lista de valores
- ☐ Sei usar `~` para inverter uma condição
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Criando e modificando colunas

Módulo 4 — Pandas Essencial · Tópico 6 de 12

O que é e por que importa

Depois de carregar e filtrar dados, é comum precisar **transformá-los**: calcular uma coluna nova a partir de outras (o total de uma venda = preço vezes quantidade), renomear colunas com nomes ruins, mudar o tipo de uma coluna, ou aplicar uma função personalizada linha a linha. Essa é a etapa onde os dados brutos começam a virar informação útil.

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB"],
    "quantidade": [3, 1, 2, 5],
    "preco_unitario": [2.50, 15.90, 89.90, 1.20],
})

# Criar uma coluna nova: operação vetorizada, igual a arrays NumPy (Módulo 3)
df["total"] = df["quantidade"] * df["preco_unitario"]
print(df)
#      produto  quantidade  preco_unitario  total
# 0  Caneta Azul          3          2.50     7.5
# 1    Caderno          1         15.90    15.9
# 2    Mochila          2         89.90   179.8
# 3   Lapis HB          5          1.20     6.0

# Criar coluna a partir de uma condição -- combina com boolean masking (Tó
df["categoria_preco"] = "barato"
df.loc[df["preco_unitario"] > 10, "categoria_preco"] = "caro"
print(df["categoria_preco"])

# Forma equivalente e mais direta com np.where (visto no Módulo 3, Tópico 3)
import numpy as np
df["categoria_preco_v2"] = np.where(df["preco_unitario"] > 10, "caro", "ba

# Modificar uma coluna existente -- mesma sintaxe de criar, só que o nome
df["preco_unitario"] = df["preco_unitario"] * 1.10 # reajuste de 10% em t

# Renomear colunas
df = df.rename(columns={"total": "valor_total", "produto": "nome_produto"})
print(df.columns.tolist())

# Remover colunas
df = df.drop(columns=["categoria_preco_v2"])
# (também é possível usar: df.drop("categoria_preco_v2", axis=1))

# .apply() -- aplica uma função a cada valor de uma coluna (ou linha, com
# use quando a transformação não dá pra fazer com operações vetorizadas si
def classificar_estoque(quantidade):
    if quantidade >= 5:
        return "alto"
    elif quantidade >= 2:
        return "medio"
    return "baixo"

df["nivel_estoque"] = df["quantidade"].apply(classificar_estoque)
print(df[["nome_produto", "quantidade", "nivel_estoque"]])

# .apply(..., axis=1) -- aplica uma função a cada LINHA inteira (recebe a
def resumo_linha(linha):
    return f"{linha['nome_produto']}: {linha['quantidade']} un."

df["resumo"] = df.apply(resumo_linha, axis=1)
print(df["resumo"])

# Mudando o tipo de uma coluna com .astype()
df["quantidade"] = df["quantidade"].astype(float)
print(df["quantidade"].dtype) # float64

```

Uma regra prática importante: sempre que a transformação puder ser escrita como uma **operação vetorizada** (`coluna_a * coluna_b`, `np.where(...)`), prefira isso a `.apply()`. `.apply()` roda uma função Python por linha, o que é bem mais lento em DataFrames grandes — reserve `.apply()` para lógicas que realmente não dá para vetorizar.

Erros comuns de quem está começando

- Usar `.apply()` para contas simples (soma, multiplicação, comparação) que já funcionam vetorizadas — funciona, mas é desnecessariamente lento e menos idiomático.
- Esquecer que a maioria dos métodos do Pandas (como `.rename()`, `.drop()`) **retorna um novo DataFrame** por padrão, em vez de alterar o original. Se você chamar `df.rename(columns=...)` sem reatribuir (`df = df.rename(...)`), a mudança se perde. (Alguns métodos aceitam `inplace=True` para alterar direto, mas o time do Pandas recomenda evitar esse parâmetro — reatribuir é mais previsível.)
- Esquecer `axis=1` ao usar `.apply()` para operar em linhas inteiras — sem isso, o Pandas assume que você quer aplicar a função a cada **coluna** (`axis=0`, o padrão), o que raramente é a intenção quando a função espera uma linha completa.

Exercício prático

```
df = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB"],
    "quantidade": [3, 1, 2, 5],
    "preco_unitario": [2.50, 15.90, 89.90, 1.20],
})
```

1. Crie a coluna `total` (`quantidade * preco_unitario`).
2. Crie a coluna `frete_gratis`, que vale `True` quando `total` for maior que R\$ 50, e `False` caso contrário (use `np.where` ou `.loc`).
3. Renomeie a coluna `produto` para `nome`.
4. Usando `.apply()`, crie a coluna `etiqueta` com o texto "{nome} (x{quantidade})" para cada linha (por exemplo, "Caderno (x1)").

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei criar colunas novas com operações vetorizadas entre colunas existentes
- ☐ Sei usar `.loc` ou `np.where` para criar colunas baseadas em condições
- ☐ Sei quando usar `.apply()` e por que evitá-lo quando dá pra vetorizar
- ☐ Sei que a maioria dos métodos do Pandas retorna um DataFrame novo (preciso reatribuir)
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Ordenação

| Módulo 4 — Pandas Essencial · Tópico 7 de 12

O que é e por que importa

Ordenar dados é uma das tarefas mais simples e mais usadas em análise: “quais os 10 produtos mais vendidos”, “quem são os clientes que mais compraram”, “do dia com mais faturamento para o com menos”. Pandas faz isso com `.sort_values()` (ordenar pelos valores de uma ou mais colunas) e `.sort_index()` (ordenar pelo índice).

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB", "Estojo"]
    "categoria": ["Papeleria", "Papeleria", "Acessorios", "Papeleria", "Ac
    "quantidade": [3, 1, 2, 5, 4],
    "preco_unitario": [2.50, 15.90, 89.90, 1.20, 25.00],
})

# sort_values -- ordena pelos valores de uma coluna, do menor para o maior
por_preco = df.sort_values("preco_unitario")
print(por_preco)

# ascending=False -- do maior para o menor
por_preco_desc = df.sort_values("preco_unitario", ascending=False)
print(por_preco_desc)

# Ordenando por múltiplas colunas -- lista de colunas, na ordem de priorid
# primeiro por categoria (A-Z), dentro de cada categoria por preço (maior
multi = df.sort_values(["categoria", "preco_unitario"], ascending=[True, F
print(multi)

# Depois de ordenar, o índice original "vem junto" (fica fora de ordem) --
# use reset_index(drop=True) se quiser um índice novo, sequencial
top_precos = df.sort_values("preco_unitario", ascending=False).reset_index
print(top_precos)
# drop=True descarta o índice antigo; sem isso, ele viraria uma coluna nov

# .head(n) depois de ordenar -- padrão clássico para "top N"
top_3_mais_caros = df.sort_values("preco_unitario", ascending=False).head(
print(top_3_mais_caros)

# .nlargest() / .nsmallest() -- atalho para o mesmo resultado, geralmente
top_3_v2 = df.nlargest(3, "preco_unitario")
print(top_3_v2)

mais_baratos = df.nsmallest(2, "preco_unitario")
print(mais_baratos)

# sort_index -- ordena pelo índice (linhas), útil depois de filtros/concat
# que deixam o índice fora de ordem
embaralhado = df.sample(frac=1) # embaralha as linhas aleatoriamente (úti
print(embaralhado.sort_index()) # volta para a ordem original 0, 1, 2, 3,

```

Erros comuns de quem está começando

- Esquecer `ascending=False` e se confundir achando que o `.sort_values()` “não funcionou” quando na verdade ordenou do menor para o maior (o padrão), e não do maior para o menor como esperado.
- Esquecer `reset_index(drop=True)` depois de ordenar e ficar com um índice fora de ordem (3, 0, 4, 1, 2 em vez de 0, 1, 2, 3, 4) — não é um erro em si (o DataFrame continua funcionando normalmente), mas pode confundir em análises seguintes que dependem de posição.
- Passar uma lista de colunas para `sort_values` mas só um valor (não-lista) para `ascending`, esperando que ele se aplique a todas — quando são várias colunas com ordens diferentes, `ascending` também precisa ser uma lista do mesmo tamanho, na mesma ordem das colunas.

Exercício prático

```

df = pd.DataFrame({
    "aluno": ["Marcos", "Julia", "Pedro", "Ana", "Carlos"],
    "nota": [7.5, 9.0, 5.5, 9.0, 6.0],
    "turma": ["A", "B", "A", "B", "A"],
})

```

1. Ordene o DataFrame pela nota, da maior para a menor.
2. Descubra os 2 alunos com as maiores notas usando `.nlargest()`.
3. Ordene por turma (A-Z) e, dentro de cada turma, por nota (maior para menor).
4. Depois de ordenar por nota (item 1), resete o índice para ficar sequencial de novo (0, 1, 2, 3, 4), sem manter o índice antigo como coluna.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Sei ordenar por uma coluna, ascendente e descendente
- ☐ Sei ordenar por múltiplas colunas com prioridades diferentes
- ☐ Sei usar `.nlargest()/.nsmallest()` e `reset_index(drop=True)`
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Agrupamento com groupby

| Módulo 4 — Pandas Essencial · Tópico 8 de 12

O que é e por que importa

`groupby` é, sem exagero, uma das ferramentas mais poderosas do Pandas. Responde perguntas do tipo “para cada X, qual é o Y?” — “para cada categoria, qual o faturamento total?”, “para cada cliente, quantas compras ele fez?”, “para cada mês, qual a média de vendas?”. Esse padrão (agrupar por uma categoria e depois resumir) é tão comum em análise de dados que vale a pena internalizar bem a lógica por trás dele.

A ideia por trás do `groupby` é sempre a mesma, conhecida como **split-apply-combine**:

1. **Split**: divide o DataFrame em grupos, um para cada valor único da coluna escolhida.
2. **Apply**: aplica uma função de agregação (soma, média, contagem...) em cada grupo, separadamente.
3. **Combine**: junta os resultados de volta em uma tabela só.

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB", "Estojo",
    "categoria": ["Papeleria", "Papeleria", "Acessorios", "Papeleria", "Ac
    "quantidade": [3, 1, 2, 5, 4, 10],
    "preco_unitario": [2.50, 15.90, 89.90, 1.20, 25.00, 0.90],
})

df["total"] = df["quantidade"] * df["preco_unitario"]

# groupby sozinho não retorna nada "visível" ainda -- é preciso dizer o qu
agrupado = df.groupby("categoria")
print(agrupado) # <pandas.core.groupby.generic.DataFrameGroupBy object ..

# Agora sim: soma do total, por categoria
soma_por_categoria = df.groupby("categoria")["total"].sum()
print(soma_por_categoria)
# categoria
# Acessorios      204.8
# Papeleria       27.5
# Name: total, dtype: float64

# Outras agregações comuns -- mesmas funções vistas em NumPy (Módulo 3, Tó
print(df.groupby("categoria")["total"].mean()) # média por categoria
print(df.groupby("categoria")["total"].count()) # quantas linhas por cate
print(df.groupby("categoria")["total"].max()) # maior valor por catego
print(df.groupby("categoria").size()) # equivalente a count(),

# Agrupando por mais de uma coluna -- passa uma lista
por_categoria_e_qtd = df.groupby(["categoria", "quantidade"])[["total"].sum
print(por_categoria_e_qtd)

# Agregando MÚLTIPLAS colunas de uma vez, sem escolher só uma
resumo = df.groupby("categoria")[["quantidade", "total"]].sum()
print(resumo)
#               quantidade    total
# categoria
# Acessorios              6    204.8
# Papeleria              19    27.5

# .reset_index() -- transforma o resultado (que tem a categoria como indic
# de volta em colunas normais, útil para salvar em CSV ou continuar proces
resumo_tabela = resume.reset_index()
print(resumo_tabela)
#   categoria  quantidade  total
# 0  Acessorios           6    204.8
# 1  Papeleria          19    27.5

# Depois de agrupar, dá pra ordenar normalmente (Tópico 7)
resumo_ordenado = resume_tabela.sort_values("total", ascending=False)
print(resumo_ordenado)

```

Erros comuns de quem está começando

- Esquecer de escolher qual coluna agregar e/ou qual função aplicar — `df.groupby("categoria")` sozinho não calcula nada, só prepara os grupos; é preciso encadear `["coluna"].sum()` (ou outra agregação) para obter um resultado.
- Esquecer o `.reset_index()` quando o resultado precisa voltar a ser um DataFrame “normal” (por exemplo, para ordenar por uma coluna que virou índice, ou para salvar em CSV com a coluna de agrupamento visível).
- Tentar agregar uma coluna de texto com `.sum().mean()` sem querer — isso costuma gerar erro ou resultado sem sentido (concatenar strings em vez de somar números); sempre confira quais colunas fazem sentido agregar antes de aplicar `.sum().mean()` a um groupby inteiro sem selecionar colunas.

Exercício prático

```
df = pd.DataFrame({
    "vendedor": ["Marcos", "Julia", "Marcos", "Pedro", "Julia", "Marcos"],
    "produto": ["Caneta", "Caderno", "Mochila", "Estojo", "Lapis", "Cadern
    "valor": [7.5, 15.9, 89.9, 25.0, 6.0, 15.9],
})
```

1. Agrupe por vendedor e calcule a soma do valor vendido por cada um.
2. Descubra quantas vendas (linhas) cada vendedor fez, usando `.size()` ou `.count()`.
3. Ordene o resultado do item 1 do vendedor que mais vendeu para o que menos vendeu.
4. Descubra qual vendedor teve a maior venda individual (dica: `.max()` por vendedor).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Entendo a lógica split-apply-combine por trás do groupby
- ☐ Sei agrupar por uma ou mais colunas e aplicar sum/mean/count/max
- ☐ Sei quando e por que usar `.reset_index()` depois de um groupby
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Agregações com agg

— Módulo 4 — Pandas Essencial · Tópico 9 de 12

O que é e por que importa

No tópico anterior você aplicou uma agregação por vez (`.sum()`, `.mean()`) depois de um groupby. Mas frequentemente você quer **várias agregações ao mesmo tempo** — por exemplo, para cada categoria, ver soma, média E contagem juntas — ou aplicar **agregações diferentes para colunas diferentes**. É para isso que existe `.agg()` (abreviação de *aggregate*): a forma flexível de combinar múltiplas agregações num único comando.

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB", "Estojo",
    "categoria": ["Papeleria", "Papeleria", "Acessorios", "Papeleria", "Ac
    "quantidade": [3, 1, 2, 5, 4, 10],
    "preco_unitario": [2.50, 15.90, 89.90, 1.20, 25.00, 0.90],
})

df["total"] = df["quantidade"] * df["preco_unitario"]

# .agg() com uma lista de funções -- aplica TODAS na mesma coluna
resumo = df.groupby("categoria")["total"].agg(["sum", "mean", "count", "ma
print(resumo)
#          sum          mean  count  max
# categoria
# Acessorios  204.8  102.400000      2  179.8
# Papeleria   27.5   9.166667      4   15.9

# .agg() com um dicionário -- escolhe agregações DIFERENTES para colunas d
resumo_customizado = df.groupby("categoria").agg({
    "total": "sum",          # soma do total
    "quantidade": "mean",    # média da quantidade
    "produto": "count",      # quantidade de produtos (linhas) no grupo
})
print(resumo_customizado)

# .agg() com nomes de coluna personalizados no resultado (named aggregatio
# a sintaxe é: nome_da_coluna_nova=("coluna_original", "funcao")
resumo_nomeado = df.groupby("categoria").agg(
    total_vendido=("total", "sum"),
    ticket_medio=("total", "mean"),
    qtd_produtos=("produto", "count"),
)
print(resumo_nomeado)
#          total_vendido  ticket_medio  qtd_produtos
# categoria
# Acessorios           204.8    102.400000           2
# Papeleria            27.5     9.166667           4
# -- essa forma é a mais recomendada quando você precisa de nomes claros,
#    porque evita colunas com nomes genéricos como "sum" ou "mean"

# .agg() também aceita funções personalizadas, não só nomes prontos como "
def amplitude(serie):
    return serie.max() - serie.min()

resumo_com_funcao_customizada = df.groupby("categoria")["total"].agg(["mea
print(resumo_com_funcao_customizada)

# Sem groupby, .agg() também funciona no DataFrame inteiro -- útil para um
resumo_geral = df[["quantidade", "total"]].agg(["sum", "mean"])
print(resumo_geral)

```

Erros comuns de quem está começando

- Usar `.agg(["sum", "mean"])` numa coluna e depois se confundir com o resultado ter as funções como **colunas** (quando `.agg` é aplicado numa única coluna) — vale sempre imprimir e conferir a forma do resultado antes de seguir usando.
- Esquecer que os nomes das funções em `.agg()` são strings ("sum", não sum) quando referenciando funções prontas do Pandas — usar o nome sem aspas (sum) na verdade chama a função sum embutida do Python, que também funciona em alguns casos mas não é a forma idiomática esperada.
- Preferir sempre `.sum().mean()` soltos mesmo quando precisa de várias agregações ao mesmo tempo, encadeando vários `groupby` separados em vez de um único `.agg()` — além de mais verboso, é mais lento (recalcula os grupos várias vezes).

Exercício prático

```
df = pd.DataFrame({
    "vendedor": ["Marcos", "Julia", "Marcos", "Pedro", "Julia", "Marcos"],
    "produto": ["Caneta", "Caderno", "Mochila", "Estojo", "Lapis", "Cadern
    "valor": [7.5, 15.9, 89.9, 25.0, 6.0, 15.9],
})
```

1. Use `.agg()` com uma lista para calcular soma, média e contagem do valor, agrupado por vendedor.
2. Use named aggregation (`nome=("coluna", "funcao")`) para criar um resumo com as colunas `total_vendido`, `ticket_medio` e `qtd_vendas`.
3. Ordene o resultado do item 2 por `total_vendido`, do maior para o menor.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei usar `.agg()` com uma lista de funções para múltiplas agregações na mesma coluna
- ☐ Sei usar `.agg()` com dicionário para agregações diferentes por coluna
- ☐ Sei usar named aggregation para nomear colunas de resultado claramente
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Merge, join e concatenação

| Módulo 4 — Pandas Essencial · Tópico 10 de 12

O que é e por que importa

Dados do mundo real quase nunca vêm todos numa tabela só. Uma loja pode ter uma tabela de pedidos e outra de clientes; um sistema pode exportar vendas de janeiro num arquivo e de fevereiro em outro. Pandas tem duas operações principais para juntar tabelas:

- `pd.concat()`: empilha DataFrames — útil quando as tabelas têm as **mesmas colunas** e você só quer juntar as linhas (ex: vendas de janeiro
 - vendas de fevereiro).
- `pd.merge()`: combina DataFrames **pelo valor de uma coluna em comum** (uma chave), parecido com um JOIN de banco de dados — útil quando as tabelas têm informações complementares (ex: pedidos + dados dos clientes que fizeram os pedidos).

Como funciona (com exemplo comentado)

```

import pandas as pd

# concat -- empilhando duas tabelas com as mesmas colunas
vendas_jan = pd.DataFrame({
    "produto": ["Caneta", "Caderno"],
    "quantidade": [10, 5],
})
vendas_fev = pd.DataFrame({
    "produto": ["Mochila", "Estojo"],
    "quantidade": [3, 7],
})

vendas_total = pd.concat([vendas_jan, vendas_fev])
print(vendas_total)
#      produto  quantidade
# 0   Caneta           10
# 1  Caderno            5
# 0  Mochila            3
# 1   Estojo            7
# -- repare que o índice se repete (0, 1, 0, 1)! use reset_index (Tópico 7)

vendas_total = pd.concat([vendas_jan, vendas_fev], ignore_index=True)
print(vendas_total) # agora o índice é 0, 1, 2, 3 -- sequencial

# merge -- combinando duas tabelas por uma coluna em comum (a "chave")
pedidos = pd.DataFrame({
    "pedido_id": [1, 2, 3, 4],
    "cliente_id": [101, 102, 101, 103],
    "valor": [150.0, 89.9, 45.0, 200.0],
})

clientes = pd.DataFrame({
    "cliente_id": [101, 102, 103],
    "nome": ["Marcos", "Julia", "Pedro"],
    "cidade": ["São Paulo", "Rio de Janeiro", "Belo Horizonte"],
})

# merge padrão ("inner"): só mantém linhas cuja chave existe nas DUAS tabelas
pedidos_com_cliente = pd.merge(pedidos, clientes, on="cliente_id")
print(pedidos_com_cliente)
#      pedido_id  cliente_id  valor  nome  cidade
# 0             1           101  150.0  Marcos  São Paulo
# 1             3           101   45.0  Marcos  São Paulo
# 2             2           102   89.9  Julia  Rio de Janeiro
# 3             4           103  200.0  Pedro  Belo Horizonte

# how="left" -- mantém TODAS as linhas da tabela da esquerda (pedidos),
# mesmo que não encontre correspondência na direita (preenche com NaN)
pedidos_todos = pd.merge(pedidos, clientes, on="cliente_id", how="left")

# how="right" -- mantém TODAS as linhas da tabela da direita (clientes)
clientes_com_pedidos = pd.merge(pedidos, clientes, on="cliente_id", how="right")

# how="outer" -- mantém TODAS as linhas de ambas, preenchendo com NaN onde
# não houver correspondência
tudo = pd.merge(pedidos, clientes, on="cliente_id", how="outer")

# Quando o nome da coluna-chave é diferente nas duas tabelas, use left_on/
# right_on
pd.merge(pedidos, clientes, left_on="cliente_id", right_on="id_do_cliente")

```

A escolha de `how` é a decisão mais importante num `merge`: pense em qual tabela é a “principal” (você quer manter todas as linhas dela mesmo sem correspondência) e escolha `left/right` de acordo, ou `outer` se quiser tudo de ambas, ou o padrão `inner` se só interessam os casos que batem nas duas.

Erros comuns de quem está começando

- Usar `pd.concat()` para juntar tabelas com colunas diferentes por engano (por

exemplo, pedidos e clientes) — o resultado fica cheio de NaN porque concat só empilha linhas, não combina por chave. Para combinar informações de tabelas diferentes, o certo é merge.

- Esquecer o how no merge e ser surpreendido pelo padrão inner, que **descarta silenciosamente** linhas sem correspondência — se o resultado do merge tem menos linhas do que o esperado, o primeiro suspeito é sempre o how.
- Fazer merge por uma coluna-chave com tipos diferentes nas duas tabelas (uma como texto, outra como número) — o merge não encontra correspondência nenhuma e o resultado sai vazio ou cheio de NaN, sem erro explícito. Confira .dtypes das colunas-chave antes de dar merge se o resultado parecer estranho.

Exercício prático

```
produtos = pd.DataFrame({
    "produto_id": [1, 2, 3],
    "nome": ["Caneta", "Caderno", "Mochila"],
    "preco": [2.5, 15.9, 89.9],
})

vendas = pd.DataFrame({
    "venda_id": [1, 2, 3, 4],
    "produto_id": [1, 2, 1, 4], # repare: produto_id 4 não existe em "pro
    "quantidade": [3, 1, 2, 5],
})
```

1. Faça um merge de vendas com produtos usando how="inner" e veja quantas linhas sobram (deve ser 3, já que o produto_id 4 não existe em produtos).
2. Faça o mesmo merge com how="left" (mantendo todas as vendas) e observe os valores NaN que aparecem para o produto_id 4.
3. No resultado do merge com how="left", crie uma coluna total com quantidade * preco (linhas com preco ausente vão gerar NaN no total — normal, veremos como tratar isso no Módulo 5).

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei quando usar concat (empilhar) e quando usar merge (combinar por chave)
- ☐ Sei explicar a diferença entre how="inner", "left", "right" e "outer"
- ☐ Sei diagnosticar um merge com resultado vazio ou menor que o esperado
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Introdução a valores ausentes

Módulo 4 — Pandas Essencial · Tópico 11 de 12

O que é e por que importa

Dados reais quase sempre têm buracos: um cliente sem telefone cadastrado, uma venda sem a categoria preenchida, uma medição que falhou naquele dia. Pandas representa esses buracos como NaN (*Not a Number*), e saber identificá-los, contá-los e decidir o que fazer com eles é essencial antes de calcular qualquer estatística — uma média calculada sem perceber valores ausentes pode dar um resultado enganoso.

Este tópico é só uma introdução ao tema — o **Módulo 5 (Limpeza de Dados)** aprofunda estratégias de tratamento. Aqui o objetivo é reconhecer valores ausentes e saber as operações básicas para lidar com eles.

Como funciona (com exemplo comentado)

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "produto": ["Caneta Azul", "Caderno", "Mochila", "Lapis HB"],
    "categoria": ["Papeleria", None, "Acessorios", "Papeleria"],
    "quantidade": [3, 1, np.nan, 5],
    "preco_unitario": [2.50, 15.90, 89.90, np.nan],
})

print(df)
#      produto  categoria  quantidade  preco_unitario
# 0  Caneta Azul  Papeleria         3.0             2.50
# 1    Caderno     None         1.0             15.90
# 2    Mochila  Acessorios        NaN             89.90
# 3   Lapis HB  Papeleria         5.0              NaN

# .isnull() (ou .isna(), são sinônimos) -- retorna True onde o valor é aus
print(df.isnull())

# Contar valores ausentes por coluna -- o padrão mais usado no dia a dia
print(df.isnull().sum())
# produto      0
# categoria     1
# quantidade     1
# preco_unitario 1
# dtype: int64

# Porcentagem de valores ausentes por coluna
print((df.isnull().sum() / len(df)) * 100)

# .notnull() -- o oposto, True onde o valor EXISTE
print(df["categoria"].notnull())

# Filtrar linhas com valor ausente numa coluna específica (combina com Tóp
sem_categoria = df[df["categoria"].isnull()]
print(sem_categoria)

# Filtrar linhas SEM valores ausentes numa coluna
com_categoria = df[df["categoria"].notnull()]
print(com_categoria)

# .dropna() -- remove linhas com QUALQUER valor ausente (padrão)
df_sem_nulos = df.dropna()
print(df_sem_nulos) # só sobra a linha 0, já que as outras têm pelo menos

# dropna(subset=[...]) -- remove linhas só considerando colunas específica
df_sem_nulos_preco = df.dropna(subset=["preco_unitario"])
print(df_sem_nulos_preco)

# .fillna() -- preenche valores ausentes com algo no lugar de remover a li
df_preenchido = df.fillna({
    "categoria": "Sem categoria",
    "quantidade": 0,
})
print(df_preenchido)
# preco_unitario continua com NaN porque não foi incluído no dicionário

# Preencher com a média da própria coluna -- estratégia comum para números
media_preco = df["preco_unitario"].mean()
df["preco_unitario"] = df["preco_unitario"].fillna(media_preco)
print(df["preco_unitario"])
```

A decisão entre **remover** (`dropna`) e **preencher** (`fillna`) depende do contexto: remover é mais simples mas perde dados (e pode enviesar a análise se os ausentes não forem aleatórios); preencher mantém as linhas mas exige escolher um valor razoável — usar a média, um valor fixo, ou olhar para colunas relacionadas. O Módulo 5 explora essas estratégias com mais calma.

Erros comuns de quem está começando

- Calcular médias/somas sem checar `.isnull().sum()` antes, sem perceber que a agregação está “ignorando” os NaN silenciosamente (é o comportamento padrão do Pandas) e sem saber quantos valores ficaram de fora dessa conta.
- Usar `== np.nan` para checar se um valor é ausente — isso **nunca** funciona (`NaN != NaN` é uma peculiaridade matemática), o correto é sempre `.isnull()/isna()`.
- Chamar `.dropna()` sem `subset` quando só uma coluna específica importa, perdendo linhas inteiras por causa de um valor ausente numa coluna que nem seria usada na análise.

Exercício prático

```
df = pd.DataFrame({
    "cliente": ["Marcos", "Julia", "Pedro", "Ana"],
    "email": ["marcos@x.com", None, "pedro@x.com", None],
    "idade": [34, 28, np.nan, 41],
    "cidade": ["São Paulo", "Rio de Janeiro", "Belo Horizonte", None],
})
```

1. Conte quantos valores ausentes existem em cada coluna.
2. Filtre e imprima só os clientes sem email cadastrado.
3. Crie uma cópia do DataFrame preenchendo email ausente com o texto "não informado" e idade ausente com a média das idades existentes.
4. Remova as linhas onde cidade está ausente (use `dropna` com `subset`).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei identificar e contar valores ausentes com `.isnull().sum()`
- ☐ Sei filtrar linhas com/sem valores ausentes numa coluna
- ☐ Sei a diferença entre `dropna()` e `fillna()` e quando usar cada um
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Tabelas dinâmicas (`pivot_table`)

— Módulo 4 — Pandas Essencial · Tópico 12 de 12 (último do módulo)

O que é e por que importa

Se você já usou tabela dinâmica no Excel ou Google Sheets, `pivot_table` do Pandas é exatamente essa ideia em código: reorganizar dados “compridos” (uma linha por registro) numa tabela “larga” que cruza duas dimensões — por exemplo, produtos nas linhas e meses nas colunas, com o faturamento em cada célula. É extremamente útil para criar resumos que ficam fáceis de ler e comparar visualmente.

`pivot_table` é, na prática, um `groupby` (Tópico 8) que organiza o resultado de um jeito diferente: em vez de uma coluna por categoria de agrupamento, ele espalha uma das dimensões pelas **colunas** da tabela.

Como funciona (com exemplo comentado)

```

import pandas as pd

vendas = pd.DataFrame({
    "produto": ["Caneta", "Caneta", "Caderno", "Caderno", "Mochila", "Mochila"],
    "mes": ["Jan", "Fev", "Jan", "Fev", "Jan", "Fev"],
    "quantidade": [10, 15, 5, 8, 2, 3],
    "faturamento": [25.0, 37.5, 79.5, 127.2, 179.8, 269.7],
})

print(vendas)

# pivot_table: index = vai virar linha, columns = vai virar coluna,
# values = o que preencher nas células, aggfunc = como agregar (padrão: me
tabela = vendas.pivot_table(
    index="produto",
    columns="mes",
    values="faturamento",
    aggfunc="sum",
)
print(tabela)
# mes      Fev      Jan
# produto
# Caderno   127.2    79.5
# Caneta     37.5    25.0
# Mochila   269.7   179.8

# Comparando com o groupby equivalente (Tópico 8) -- o resultado tem os me
# números, só que "compridos" em vez de "largos":
equivalente_groupby = vendas.groupby(["produto", "mes"])["faturamento"].su
print(equivalente_groupby)
# produto mes
# Caderno Fev    127.2
#          Jan     79.5
# Caneta Fev     37.5
#          Jan     25.0
# Mochila Fev   269.7
#          Jan   179.8

# fill_value -- preenche combinações que não existem nos dados com um valo
# (por padrão ficariam como NaN)
tabela_preenchida = vendas.pivot_table(
    index="produto", columns="mes", values="faturamento",
    aggfunc="sum", fill_value=0,
)

# margins=True -- adiciona uma linha e coluna de totais gerais
tabela_com_totais = vendas.pivot_table(
    index="produto", columns="mes", values="faturamento",
    aggfunc="sum", margins=True, margins_name="Total",
)
print(tabela_com_totais)
#          Fev      Jan      Total
# produto
# Caderno   127.2    79.5    206.7
# Caneta     37.5    25.0     62.5
# Mochila   269.7   179.8   449.5
# Total     434.4   284.3   718.7

# Múltiplos valores agregados de uma vez
tabela_multipla = vendas.pivot_table(
    index="produto", columns="mes",
    values=["faturamento", "quantidade"],
    aggfunc="sum",
)
print(tabela_multipla)

# Voltando de "Larga" para "comprida" com melt (operação inversa ao pivot)
comprida = tabela.reset_index().melt(id_vars="produto", var_name="mes", va
print(comprida)

```

Erros comuns de quem está começando

- Esquecer `aggfunc` quando há mais de um valor para a mesma combinação de `index/columns` — o padrão é `"mean"` (média), o que pode não ser o que você quer (frequentemente é `"sum"` para totais de vendas, por exemplo).
- Não usar `fill_value=0` quando faz sentido, e depois se confundir com `NaN` aparecendo em células cuja combinação simplesmente não tem dados (por exemplo, um produto que só foi vendido num mês).
- Achar que `pivot_table` é uma ferramenta completamente diferente de `groupby`, quando na prática resolve o mesmo tipo de pergunta — vale a pena pensar em qual formato de resultado (comprido, do `groupby`, ou largo, do `pivot_table`) é mais útil para o próximo passo da análise.

Exercício prático

```
notas = pd.DataFrame({
    "aluno": ["Marcos", "Marcos", "Julia", "Julia", "Pedro", "Pedro"],
    "matéria": ["Matemática", "Português", "Matemática", "Português", "Mat",
    "nota": [7.5, 8.0, 9.0, 6.5, 5.5, 7.0],
})
```

1. Crie uma tabela dinâmica com alunos nas linhas, matérias nas colunas e a nota como valor (use `aggfunc="mean"`, já que cada combinação aluno/matéria tem só um valor).
2. Adicione `margins=True` para ver a média geral por aluno e por matéria.
3. Descubra, olhando a tabela, qual aluno teve a maior média geral.

Solução comentada

- Clique para ver a solução

Checklist antes de avançar

- ☐ Sei criar uma tabela dinâmica com `index`, `columns`, `values` e `aggfunc`
- ☐ Entendo a relação entre `pivot_table` e `groupby` (mesma informação, formatos diferentes)
- ☐ Sei usar `fill_value` e `margins` quando fazem sentido
- ☐ Resolvi o exercício sem olhar a solução

Checklist do Módulo 4

Antes de seguir para o Módulo 5 (Limpeza de Dados), confirme que você consegue, sem consultar a apostila:

- ☐ Explicar a diferença entre `Series` e `DataFrame`
- ☐ Ler e salvar dados em CSV (e saber que Excel/JSON seguem padrão parecido)
- ☐ Explorar um dataset novo com `.head()`, `.info()`, `.describe()`, `.value_counts()`
- ☐ Selecionar dados com `.loc` e `.iloc`, sabendo a diferença entre eles
- ☐ Filtrar linhas com condições simples e combinadas (`&`, `|`, `.isin()`)
- ☐ Criar e modificar colunas, inclusive com `.apply()` quando necessário
- ☐ Ordenar um `DataFrame` por uma ou mais colunas
- ☐ Agrupar dados com `groupby` e agregar com `.agg()`
- ☐ Combinar tabelas com `concat` e `merge`, escolhendo o `how` certo
- ☐ Identificar e tratar valores ausentes básicos com `dropna/fillna`
- ☐ Criar uma tabela dinâmica com `pivot_table`

Pronto? Siga para o Módulo 5 — Limpeza de Dados.

Módulo 5 — Limpeza de Dados

Com o essencial de Pandas dominado (Módulo 4), é hora de encarar de frente algo que ocupa a maior parte do tempo real de quem trabalha com dados: **dados sujos**. Datasets do mundo real vêm com valores ausentes, duplicados, tipos errados, texto inconsistente (“SP”, “São Paulo”, “sao paulo” querendo dizer a mesma coisa) e valores absurdos que não deveriam estar ali. Antes de analisar ou tirar qualquer conclusão, é preciso limpar.

Este módulo aprofunda o que foi visto de leve no Tópico 11 do Módulo 4 (valores ausentes) e adiciona as outras frentes de limpeza mais comuns: duplicatas, tipos de dados, texto inconsistente, datas e outliers.

Tópicos

1. Estratégias para valores ausentes
2. Duplicatas
3. Tipos de dados e conversão
4. Limpeza de texto
5. Trabalhando com datas
6. Outliers: detecção e tratamento
7. Checklist de qualidade de dados

Depois de terminar todos os tópicos, siga para o Módulo 6 — Estatística e EDA.

Estratégias para valores ausentes

| Módulo 5 — Limpeza de Dados · Tópico 1 de 7

O que é e por que importa

No Módulo 4 (Tópico 11) você aprendeu a **identificar** e fazer operações básicas (`dropna`, `fillna`) com valores ausentes. Aqui o foco muda: como **decidir** o que fazer com eles. Essa decisão não é mecânica — depende do que os dados significam e de quanto cada abordagem pode distorcer a análise.

Existem, na prática, quatro caminhos principais:

1. **Remover a linha** — quando o dado ausente é raro e a linha inteira perde sentido sem ele.
2. **Remover a coluna** — quando a maior parte dos valores da coluna está ausente (ela adiciona pouca informação).
3. **Preencher com um valor fixo ou estatística** (média, mediana, moda, “desconhecido”) — quando remover perderia dados bons demais.
4. **Preencher usando outras colunas ou linhas próximas** (interpolação, preenchimento por grupo) — quando existe uma forma mais informada de estimar o valor que faltou.

Como funciona (com exemplo comentado)

```

import pandas as pd
import numpy as np

df = pd.DataFrame({
    "produto": ["Caneta", "Caderno", "Mochila", "Lapis", "Estojo", "Borrac"],
    "categoria": ["Papellaria", "Papellaria", "Acessorios", None, "Acessorio"],
    "preco": [2.5, 15.9, 89.9, 1.2, np.nan, 0.9],
    "avaliacao": [4.5, np.nan, np.nan, np.nan, np.nan, 3.8],
})

# Passo 1: sempre visualizar a extensao do problema, coluna a coluna
print(df.isnull().sum())
print((df.isnull().sum() / len(df) * 100).round(1))
# categoria: 1 de 6 (16.7%) -- razoavel preencher
# preco: 1 de 6 (16.7%) -- razoavel preencher
# avaliacao: 4 de 6 (66.7%) -- muito ausente, considerar remover a coluna

# Estrategia 1: remover coluna com ausencia muito alta
df_sem_avaliacao = df.drop(columns=["avaliacao"])

# Estrategia 2: remover linhas especificas (quando o dado e essencial e ra
df_sem_preco_ausente = df.dropna(subset=["preco"])

# Estrategia 3: preencher com estatistica -- media/mediana para numeros
mediana_preco = df["preco"].median()
df["preco"] = df["preco"].fillna(mediana_preco)
# mediana costuma ser preferivel a media quando ha valores muito extremos
# (outliers) puxando a media para longe do "tipico" -- mais no Topico 6

# Estrategia 3b: preencher categorico com a moda (valor mais frequente) ou
moda_categoria = df["categoria"].mode()[0] # .mode() retorna uma Series (
df["categoria"] = df["categoria"].fillna(moda_categoria)
# alternativa, muitas vezes mais honesta que "inventar" uma categoria:
# df["categoria"] = df["categoria"].fillna("Nao informado")

# Estrategia 4: preencher por grupo -- usa a media/mediana DENTRO de cada
# mais preciso do que usar uma media geral que ignora o contexto
df2 = pd.DataFrame({
    "produto": ["A", "B", "C", "D", "E"],
    "categoria": ["X", "X", "Y", "Y", "Y"],
    "preco": [10.0, np.nan, 50.0, 55.0, np.nan],
})
df2["preco"] = df2.groupby("categoria")["preco"].transform(lambda s: s.fill
print(df2)
#      produto categoria  preco
# 0         A         X   10.0
# 1         B         X   10.0  <- preenchido com a media do grupo X (so ti
# 2         C         Y   50.0
# 3         D         Y   55.0
# 4         E         Y   52.5  <- preenchido com a media do grupo Y (50 e

# Estrategia 4b: interpolacao -- estima valores ausentes com base na tend
# dos vizinhos, util sobretudo em series temporais (mais sobre datas no Tó
serie_temporal = pd.Series([10, 12, np.nan, 16, 18])
print(serie_temporal.interpolate()) # [10. 12. 14. 16. 18.] -- preencheu

```

Erros comuns de quem está começando

- Preencher todos os valores ausentes com a **média geral** sem considerar contexto (grupos, categorias), gerando estimativas piores do que preencher por grupo — como no exemplo do `df2` acima, a média geral ignoraria que os produtos da categoria “Y” custam bem mais que os da “X”.
- Remover linhas com qualquer valor ausente (`dropna()` sem `subset`) de forma automática, sem checar antes se isso descarta uma fatia grande e possivelmente enviesada dos dados (por exemplo, se só clientes de uma região específica não preenchem um campo).

- Preencher valores ausentes numéricos com 0 “porque é neutro”, quando 0 na verdade tem um significado real diferente de “não sei” (por exemplo, preço 0 parece uma promoção, não um dado ausente) — nesses casos, NaN é mais honesto do que um 0 inventado.

Exercício prático

```
df = pd.DataFrame({
    "cidade": ["São Paulo", "São Paulo", "Rio de Janeiro", "Rio de Janeiro",
    "aluguel_medio": [3500, np.nan, 2200, 2400, np.nan],
    "corretor": ["Marcos", "Julia", "Marcos", None, "Pedro"],
})
```

1. Calcule a porcentagem de valores ausentes em cada coluna.
2. Preencha `aluguel_medio` usando a média **por cidade** (com `groupby + transform`, como no exemplo).
3. Preencha `corretor` ausente com o texto "Não atribuído".
4. Confirme, ao final, que não sobrou nenhum valor ausente no `DataFrame`.

Solução comentada

- Clique para ver a solução

Checklist antes de avançar

- ☐ Sei calcular a porcentagem de valores ausentes por coluna antes de decidir o que fazer
- ☐ Sei escolher entre remover, preencher com estatística geral ou preencher por grupo
- ☐ Entendo por que preencher com 0 nem sempre é uma boa ideia
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Duplicatas

— Módulo 5 — Limpeza de Dados · Tópico 2 de 7

O que é e por que importa

Linhas duplicadas aparecem por vários motivos: um formulário enviado duas vezes por engano, um erro num `merge` (Módulo 4, Tópico 10) que multiplicou linhas, uma importação de dados rodada mais de uma vez. Se não forem tratadas, duplicatas inflam contagens e somas — um faturamento “duplicado” parece maior do que realmente é. Identificar e remover duplicatas corretamente é um passo básico de qualquer limpeza de dados.

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "pedido_id": [1, 2, 3, 2, 4, 3],
    "cliente": ["Marcos", "Julia", "Pedro", "Julia", "Ana", "Pedro"],
    "valor": [150.0, 89.9, 45.0, 89.9, 200.0, 45.0],
})
print(df)

# .duplicated() -- retorna True para linhas que são cópia EXATA de uma ant
print(df.duplicated())
# 0    False
# 1    False
# 2    False
# 3     True  <- igual à linha 1
# 4    False
# 5     True  <- igual à linha 2
# dtype: bool

# Contar quantas linhas duplicadas existem
print(df.duplicated().sum()) # 2

# Ver quais linhas são as duplicadas
print(df[df.duplicated()])

# .drop_duplicates() -- remove as duplicatas, mantendo a PRIMEIRA ocorrênc
df_sem_duplicatas = df.drop_duplicates()
print(df_sem_duplicatas)

# keep="last" -- mantém a ÚLTIMA ocorrência em vez da primeira
df_mantendo_ultima = df.drop_duplicates(keep="last")

# keep=False -- remove TODAS as ocorrências envolvidas em duplicação (não
df_remove_todas = df.drop_duplicates(keep=False)

# subset -- considera duplicata só olhando algumas colunas, não a linha in
# útil quando "pedido_id" repetido já é suficiente para considerar duplica
# mesmo que outras colunas (por acaso) sejam diferentes
df_por_pedido_id = df.drop_duplicates(subset=["pedido_id"])
print(df_por_pedido_id)

# Sempre reset_index depois de remover linhas, se a ordem/índice sequencia
df_limpo = df.drop_duplicates().reset_index(drop=True)
print(df_limpo)

```

Um ponto sutil: `duplicated()` (sem `subset`) considera duas linhas iguais somente se **todas as colunas** baterem. Isso significa que duas linhas podem representar o mesmo evento do mundo real (o mesmo pedido, por exemplo) sem serem detectadas como duplicatas, se algum campo (como um timestamp de processamento) for ligeiramente diferente. Por isso é importante pensar em qual é a **chave de identidade** real dos seus dados — normalmente um ID — e usar `subset` para refletir isso.

Erros comuns de quem está começando

- Rodar `drop_duplicates()` sem `subset` acreditando que remove duplicatas “de negócio” (ex: o mesmo pedido registrado duas vezes), quando na verdade só remove linhas 100% idênticas em todas as colunas — se houver qualquer diferença mínima entre elas (mesmo um espaço extra no texto), não são detectadas.
- Não verificar `duplicated().sum()` antes de agregar ou somar dados, chegando a totais inflados sem perceber a causa.
- Usar `keep=False` (remove todas as ocorrências) quando o objetivo era só “manter uma cópia” (`keep="first"`, o padrão) — isso descarta até a primeira ocorrência legítima do dado, que não deveria ser removida.

Exercício prático

```
df = pd.DataFrame({
    "cliente_id": [1, 2, 3, 1, 4, 2],
    "email": ["a@x.com", "b@x.com", "c@x.com", "a@x.com", "d@x.com", "b@x.",
    "cadastro": ["2024-01-10", "2024-01-11", "2024-01-12", "2024-02-01", "
```

1. Descubra quantas linhas são duplicatas exatas (considerando todas as colunas).
2. Descubra quantos cliente_id aparecem mais de uma vez, mesmo que a data de cadastro seja diferente (use subset=["cliente_id"]).
3. Remova duplicatas por cliente_id, mantendo a ocorrência com a data de cadastro **mais antiga** (dica: ordene por cadastro antes de usar drop_duplicates(subset=..., keep="first")).

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei identificar duplicatas exatas e contar quantas existem
- ☐ Sei usar subset para detectar duplicatas por uma chave de negócio específica
- ☐ Sei escolher entre keep="first", "last" e False de acordo com o objetivo
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Tipos de dados e conversão

Módulo 5 — Limpeza de Dados · Tópico 3 de 7

O que é e por que importa

Um problema muito comum em dados importados (de CSV, de planilhas, de formulários) é o **tipo errado**: um preço lido como texto porque tinha um “R\$” na frente, uma data guardada como string, uma coluna numérica que virou object porque um único valor estava escrito errado (como "120" em vez de "120", com a letra O no lugar do número zero). Colunas com tipo errado quebram cálculos, ordenações e comparações de forma silenciosa — o Pandas não avisa, só produz resultado errado.

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "produto": ["Caneta", "Caderno", "Mochila", "Lapis"],
    "preco": ["R$ 2,50", "R$ 15,90", "R$ 89,90", "R$ 1,20"], # texto, não
    "quantidade": ["3", "1", "2", "cinco"], # texto, e um valor inválido
    "em_estoque": ["sim", "sim", "nao", "sim"],
})

print(df.dtypes) # tudo "object" (texto), mesmo preco e quantidade

# Limpando texto antes de converter: remover "R$", espaço e trocar vírgula
df["preco"] = (
    df["preco"]
    .str.replace("R$", "", regex=False)
    .str.replace(",", ".", regex=False)
    .str.strip()
)
df["preco"] = df["preco"].astype(float)
print(df["preco"])
print(df["preco"].dtype) # float64

# pd.to_numeric -- converte para número, com controle sobre erros de conver
# errors="coerce" transforma valores que não dão para converter em NaN, em
df["quantidade"] = pd.to_numeric(df["quantidade"], errors="coerce")
print(df["quantidade"])
# 0    3.0
# 1    1.0
# 2    2.0
# 3    NaN  <- "cinco" não é um número, virou NaN em vez de dar erro
print(df["quantidade"].dtype) # float64 (NaN obriga o tipo a ser float, n

# errors="raise" (padrão) -- gera erro se algum valor não puder ser conver
# útil quando você QUER ser avisado de dados inválidos em vez de ignorá-lo
# pd.to_numeric(df["quantidade"], errors="raise") # ex: dispararia ValueError

# Convertendo texto para booleano
df["em_estoque"] = df["em_estoque"].map({"sim": True, "nao": False})
print(df["em_estoque"])
print(df["em_estoque"].dtype) # bool

# .astype() -- forma geral de converter tipo, quando os dados já estão "Li
# (sem texto misturado, sem símbolos)
df["quantidade_int"] = df["quantidade"].fillna(0).astype(int)
# fillna antes do astype(int) porque int não aceita NaN -- precisa resolver
# ausentes primeiro (Tópico 1)

# category -- tipo especial para colunas de texto com poucos valores único
# economiza memória e deixa algumas operações mais rápidas
df["produto"] = df["produto"].astype("category")
print(df["produto"].dtype) # category

print(df.dtypes)

```

Erros comuns de quem está começando

- Usar `.astype(float)` direto numa coluna de texto com símbolos ("R\$", ",", espaços) sem limpar antes — gera erro, porque "R\$ 2,50" não é um número válido em Python/Pandas. Sempre limpar o texto primeiro (remover símbolos, trocar vírgula por ponto) e só depois converter.
- Usar `errors="coerce"` sem depois checar quantos NaN foram criados — isso silenciosamente descarta dados inválidos como se fossem ausentes; é importante rodar `.isnull().sum()` (Tópico 1) depois da conversão para saber quantos valores realmente não puderam ser convertidos.
- Tentar `.astype(int)` numa coluna com NaN — inteiros do NumPy/Pandas não suportam NaN nativamente (só float suporta), então é preciso resolver os valores ausentes antes (com `fillna`) ou usar o tipo especial `Int64` (com I maiúsculo) do

Pandas, que aceita NaN.

Exercício prático

```
df = pd.DataFrame({
    "produto": ["Caneta", "Caderno", "Mochila"],
    "preco_texto": ["2.50 reais", "15.90 reais", "89.90 reais"],
    "avaliacao": ["4.5", "N/A", "3.8"],
})
```

1. Limpe a coluna `preco_texto` (remova o texto " reais") e converta para float, guardando em uma nova coluna `preco`.
2. Converta `avaliacao` para número usando `pd.to_numeric` com `errors="coerce"`, guardando em uma coluna `avaliacao_num` (o valor "N/A" deve virar NaN).
3. Confira quantos valores viraram NaN na conversão do item 2.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Sei limpar texto (remover símbolos, trocar separador decimal) antes de converter para número
- ☐ Sei usar `pd.to_numeric(..., errors="coerce")` e checar quantos NaN foram criados
- ☐ Sei que `int` não aceita NaN e por isso preciso tratar ausentes antes de converter
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Limpeza de texto

| Módulo 5 — Limpeza de Dados · Tópico 4 de 7

O que é e por que importa

Colunas de texto são as que mais sofrem com inconsistência: a mesma cidade escrita como "São Paulo", "sao paulo", " São Paulo " (com espaços extras) ou "SP" — para um humano é óbvio que é a mesma coisa, mas para o Pandas são valores completamente diferentes, o que quebra `groupby`, `value_counts()` e filtros. Padronizar texto é um passo quase obrigatório antes de qualquer análise séria envolvendo colunas categóricas.

Pandas expõe operações de string através do acessor `.str`, que aplica métodos de string do Python (que você já viu no Módulo 1, tópicos 14-15) a uma coluna inteira de uma vez, de forma vetorizada.

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "cidade": [" São Paulo", "sao paulo", "SÃO PAULO ", "Rio de Janeiro",
    "email": ["Marcos@Email.com", " julia@email.com", "PEDRO@EMAIL.COM", "
})

# .str.strip() -- remove espaços extras no início/fim (igual ao .strip() d
df["cidade"] = df["cidade"].str.strip()

# .str.lower() / .str.upper() / .str.title() -- padroniza capitalização
df["cidade"] = df["cidade"].str.lower()
print(df["cidade"].unique())
# ['são paulo' 'sao paulo' 'rio de janeiro']
# -- "são paulo" e "sao paulo" ainda são diferentes por causa do acento!

# Removendo acentos para padronizar de verdade (usando unicodedata, do Pyt
import unicodedata

def remover_acentos(texto):
    nfkd = unicodedata.normalize("NFKD", texto)
    return "".join(c for c in nfkd if not unicodedata.combining(c))

df["cidade"] = df["cidade"].apply(remover_acentos)
print(df["cidade"].unique())
# ['sao paulo' 'rio de janeiro'] -- agora sim, valores únicos de verdade

# .str.title() -- deixa com a primeira letra de cada palavra maiúscula, pa
df["cidade_exibicao"] = df["cidade"].str.title()
print(df["cidade_exibicao"]) # "Sao Paulo", "Rio De Janeiro"

# .str.contains() -- filtra linhas cujo texto contém um padrão (aceita reg
gmail_ou_email = df[df["email"].str.strip().str.contains("email.com")]
print(gmail_ou_email)

# .str.replace() -- substituir parte do texto (vetorizado, igual visto no
df["email"] = df["email"].str.strip().str.lower()
print(df["email"])

# .str.split() -- divide o texto em partes, retorna listas (ou colunas sep
nomes = pd.Series(["Marcos Silva", "Julia Santos", "Pedro Costa"])
partes = nomes.str.split(" ", expand=True)
print(partes)
#           0           1
# 0  Marcos  Silva
# 1  Julia   Santos
# 2  Pedro   Costa

# map() com dicionário -- para padronizar abreviações conhecidas em valore
mapa_estados = {"SP": "São Paulo", "RJ": "Rio de Janeiro", "MG": "Minas Ge
siglas = pd.Series(["SP", "RJ", "SP", "MG"])
print(siglas.map(mapa_estados))

```

Erros comuns de quem está começando

- Rodar `.unique()` ou `.value_counts()` numa coluna de texto e assumir que os valores estão corretos, sem notar variações sutis de espaço, acento ou capitalização que fazem o Pandas contar “São Paulo” e “sao paulo” como categorias diferentes.
- Esquecer o `.str` antes do método de string (`df["cidade"].lower()` em vez de `df["cidade"].str.lower()`) — sem o acessor `.str`, o Pandas tenta achar um método `lower()` na própria Series (que não existe) e gera erro.
- Aplicar `.str.strip()`/`.str.lower()` só nas colunas que “parecem” precisar, sem checar todas as colunas de texto do dataset — inconsistência de espaço/capitalização costuma aparecer em mais lugares do que se espera.

Exercício prático

```
df = pd.DataFrame({
    "categoria": [" Eletrônicos", "eletronicos", "ELETRÔNICOS ", "Roupas",
    "cliente": ["Ana Paula Souza", "Jose Roberto Lima", "Marcia Andrade"],
})
```

(considere só a coluna `categoria` para os itens 1-3, com 5 valores)

1. Remova espaços extras e converta `categoria` para minúsculas.
2. Remova os acentos da coluna `categoria` (use a função `remove_acentos` do exemplo) e confirme com `.unique()` que sobraram só 2 categorias distintas.
3. Na coluna `cliente` (3 valores), separe em duas colunas `primeiro_nome` e `sobrenome_completo` usando `.str.split(" ", n=1, expand=True)` (o parâmetro `n=1` limita a divisão ao primeiro espaço).

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Sei usar `.str.strip()`, `.str.lower()/upper()/title()` para padronizar texto
- ☐ Sei que acentos precisam de tratamento à parte (não bastam `.lower()/strip()`)
- ☐ Sei usar `.str.split(expand=True)` para dividir texto em colunas
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Trabalhando com datas

Módulo 5 — Limpeza de Dados · Tópico 5 de 7

O que é e por que importa

Datas são um dos tipos de dado mais problemáticos em limpeza: vêm em formatos diferentes ("10/01/2024", "2024-01-10", "10 jan 2024"), frequentemente como texto em vez de um tipo de data de verdade, e análises de negócio comuns ("vendas por mês", "quantos dias desde a última compra") exigem que o Pandas reconheça a coluna como data para funcionar. O tipo correto para isso é o `datetime64`, e o Pandas converte para ele com `pd.to_datetime()`.

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "pedido_id": [1, 2, 3, 4],
    "data_texto": ["10/01/2024", "15/01/2024", "02/02/2024", "28/02/2024"]
    "valor": [150.0, 89.9, 45.0, 200.0],
})
print(df.dtypes) # data_texto é "object" (texto), não data

# pd.to_datetime -- converte texto para datetime
# format especifica o padrão esperado -- sem isso, o Pandas tenta adivinha
# o que pode confundir dia e mês (10/01 pode virar 10 de janeiro OU 1 de o
# dependendo de qual convenção o Pandas assumir)
df["data"] = pd.to_datetime(df["data_texto"], format="%d/%m/%Y")
print(df["data"])
print(df["data"].dtype) # datetime64[ns]

# Códigos de formato mais comuns: %d dia, %m mês, %Y ano com 4 dígitos,
# %y ano com 2 dígitos, %H:%M:%S horário

# Uma vez convertida, a coluna ganha o acessor .dt, com componentes da dat
df["ano"] = df["data"].dt.year
df["mes"] = df["data"].dt.month
df["dia"] = df["data"].dt.day
df["dia_da_semana"] = df["data"].dt.day_name() # nome do dia (em inglês p
print(df[["data", "ano", "mes", "dia", "dia_da_semana"]])

# Comparações e filtros com datas -- funcionam como comparações numéricas
inicio_fevereiro = pd.Timestamp("2024-02-01")
vendas_fevereiro = df[df["data"] >= inicio_fevereiro]
print(vendas_fevereiro)

# Diferença entre datas -- retorna um Timedelta
hoje = pd.Timestamp("2024-03-01")
df["dias_desde_pedido"] = (hoje - df["data"]).dt.days
print(df[["data", "dias_desde_pedido"]])

# Agrupando por mês -- combina com groupby (Módulo 4, Tópico 8)
df["ano_mes"] = df["data"].dt.to_period("M") # "2024-01", "2024-02", ...
vendas_por_mes = df.groupby("ano_mes")["valor"].sum()
print(vendas_por_mes)

# errors="coerce" -- igual visto em pd.to_numeric (Tópico 3): datas inváli
# (Not a Time, o equivalente a NaN para datas) em vez de quebrar a convers
datas_com_erro = pd.Series(["10/01/2024", "data invalida", "15/02/2024"])
convertido = pd.to_datetime(datas_com_erro, format="%d/%m/%Y", errors="coe
print(convertido)
# 0    2024-01-10
# 1              NaT
# 2    2024-02-15

```

Erros comuns de quem está começando

- Não especificar `format` ao converter datas ambíguas (como `10/01/2024`, que pode ser 10 de janeiro ou 1 de outubro dependendo da convenção) e deixar o Pandas adivinhar errado silenciosamente — sempre que souber o formato de origem, é mais seguro informá-lo explicitamente.
- Comparar uma coluna de data (já convertida) com uma **string** de data (`df["data"] >= "2024-02-01"`) — isso até funciona na maioria dos casos (o Pandas converte a string automaticamente), mas é menos explícito e mais propenso a erro do que comparar com um `pd.Timestamp(...)` de verdade.
- Esquecer de checar `NaT` depois de uma conversão com `errors="coerce"` — assim como `NaN` para números, `NaT` precisa ser tratado (Tópico 1) antes de seguir com cálculos de datas.

Exercício prático

```
df = pd.DataFrame({
    "cliente": ["Marcos", "Julia", "Pedro", "Ana"],
    "data_cadastro": ["05/03/2023", "12/07/2023", "20/11/2023", "01/01/2024"]
})
```

1. Converta `data_cadastro` para `datetime`, especificando o `format` correto ("%d/%m/%Y").
2. Crie uma coluna `ano_cadastro` com o ano de cada data.
3. Calcule há quantos dias cada cliente está cadastrado, considerando “hoje” como `pd.Timestamp("2024-06-01")`.
4. Filtre os clientes cadastrados a partir de julho de 2023 (inclusive).

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei converter texto para data com `pd.to_datetime()` especificando o `format`
- ☐ Sei extrair ano/mês/dia com o acessor `.dt`
- ☐ Sei filtrar e calcular diferenças entre datas
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Outliers: detecção e tratamento

| Módulo 5 — Limpeza de Dados · Tópico 6 de 7

O que é e por que importa

Outlier é um valor muito diferente do padrão dos demais — um funcionário com salário de R\$ 500.000 numa base onde a maioria ganha entre R\$ 3.000 e R\$ 8.000, ou uma venda de R\$ -50 (impossível) por erro de digitação. Alguns outliers são erros de dados (e devem ser corrigidos ou removidos); outros são eventos reais e importantes (e devem ser mantidos, só tratados com cuidado nas análises). A parte difícil não é detectar valores extremos — é decidir o que fazer com cada um.

Este módulo (05) foca na detecção mecânica; o Módulo 6 (Estatística e EDA) aprofunda a interpretação estatística por trás disso.

Como funciona (com exemplo comentado)

```

import pandas as pd
import numpy as np

df = pd.DataFrame({
    "funcionario": ["A", "B", "C", "D", "E", "F", "G"],
    "salario": [3200, 3500, 4100, 3800, 3900, 4200, 50000], # o último é outlier
})

# Método 1: olhar estatísticas descritivas primeiro (Módulo 4, Tópico 3)
print(df["salario"].describe())
# a média fica bem puxada para cima pelo outlier; a mediana (50%) é mais "típica"
# sobre o valor típico

# Método 2: IQR (Interquartile Range / Amplitude Interquartil) -- técnica
q1 = df["salario"].quantile(0.25)
q3 = df["salario"].quantile(0.75)
iqr = q3 - q1

limite_inferior = q1 - 1.5 * iqr
limite_superior = q3 + 1.5 * iqr
print(f"Limites: {limite_inferior} a {limite_superior}")

outliers = df[(df["salario"] < limite_inferior) | (df["salario"] > limite_superior)]
print(outliers) # funcionário G aparece como outlier

# Método 3: z-score -- quantos desvios padrão o valor está distante da média
# valores com |z-score| > 3 costumam ser considerados outliers
media = df["salario"].mean()
desvio_padrao = df["salario"].std()
df["z_score"] = (df["salario"] - media) / desvio_padrao
print(df[["funcionario", "salario", "z_score"]])

outliers_zscore = df[df["z_score"].abs() > 2]
print(outliers_zscore)

# Depois de identificar, as estratégias de tratamento são parecidas com as de valores ausentes (Tópico 1):

# Estratégia A: remover a linha (quando é claramente um erro de dados)
df_sem_outlier = df[(df["salario"] >= limite_inferior) & (df["salario"] <= limite_superior)]

# Estratégia B: "capping" (também chamado de winsorização) -- substitui o valor extremo pelo limite mais próximo, em vez de descartar a linha inteira
df["salario_com_cap"] = df["salario"].clip(lower=limite_inferior, upper=limite_superior)
print(df[["funcionario", "salario", "salario_com_cap"]])

# Estratégia C: manter o valor, mas usar estatísticas robustas a outliers
# na análise (mediana em vez de média, por exemplo) -- não altera o dado, só a forma de resumi-lo
print("Média:", df["salario"].mean()) # puxada pelo outlier
print("Mediana:", df["salario"].median()) # não afetada da mesma forma

```

Erros comuns de quem está começando

- Remover todo outlier automaticamente sem investigar se é um erro de dados ou um evento real legítimo (uma venda excepcionalmente alta pode ser informação valiosa, não ruído) — a decisão deveria vir depois de olhar o contexto, não antes.
- Usar a **média** para resumir uma coluna que tem outliers, sem considerar a **mediana** como alternativa mais representativa do valor “típico” — isso volta com mais profundidade no Módulo 6.
- Aplicar o método IQR ou z-score cegamente em qualquer coluna, mesmo quando ela naturalmente tem distribuição bem espalhada (por exemplo, uma coluna de faturamento de empresas de tamanhos muito diferentes) — os “outliers” detectados nesse caso podem ser normais para o domínio, não erros.

Exercício prático

```
df = pd.DataFrame({
    "produto": ["A", "B", "C", "D", "E", "F"],
    "preco": [25.0, 30.0, 28.0, 32.0, 27.0, 999.0], # o último parece err
})
```

1. Calcule Q1, Q3 e o IQR da coluna `preco`.
2. Calcule os limites inferior e superior (regra $1.5 \times \text{IQR}$) e identifique quais linhas são outliers.
3. Crie uma coluna `preco_com_cap` usando `.clip()` para limitar os valores extremos aos limites calculados.
4. Compare a média e a mediana de `preco` antes do tratamento, e comente (em um comentário no código) qual delas parece mais representativa do preço “típico” dos produtos.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei calcular limites de outlier com a regra IQR ($1.5 \times \text{IQR}$)
- ☐ Sei calcular z-score e usá-lo como critério alternativo de detecção
- ☐ Sei escolher entre remover, limitar (`clip`) ou manter um outlier, com justificativa
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Checklist de qualidade de dados

| Módulo 5 — Limpeza de Dados · Tópico 7 de 7 (último do módulo)

O que é e por que importa

Os tópicos anteriores trataram cada problema de limpeza separadamente (ausentes, duplicatas, tipos, texto, datas, outliers). Na prática, ao receber um dataset novo, você não sabe de antemão quais desses problemas existem — é preciso um **processo sistemático** para descobrir. Este último tópico junta tudo em um roteiro de diagnóstico que você pode aplicar a qualquer dataset novo, do início ao fim, antes de partir para análise de verdade (Módulo 6).

Como funciona (com exemplo comentado)

```
import pandas as pd
import numpy as np

# Dataset fictício com vários problemas de uma vez, para praticar o roteiro
from io import StringIO

csv_texto = """produto,categoria,preco,quantidade,data_venda
Caneta Azul,Papelaria,R$ 2,50,3,10/01/2024
Caderno, papelaria ,R$ 15,90,1,15/01/2024
Mochila,Acessorios,R$ 89,90,2,02/02/2024
Caneta Azul,Papelaria,R$ 2,50,3,10/01/2024
Lapis HB,Papelaria,R$ 1,20,,28/02/2024
Estojo,acessorios,R$ 25,00,999,05/03/2024
"""
```

```

df = pd.read_csv(StringIO(csv_texto))

# ROTEIRO DE DIAGNÓSTICO

# 1. Visão geral: shape, tipos, primeiras Linhas
print(df.shape)
df.info()
print(df.head())

# 2. Valores ausentes (Tópico 1)
print(df.isnull().sum())
# "quantidade" tem 1 ausente

# 3. Duplicatas (Tópico 2)
print(df.duplicated().sum())
# 1 linha duplicada (Caneta Azul aparece duas vezes, idêntica)

# 4. Tipos de dados suspeitos (Tópico 3)
print(df.dtypes)
# "preco" é object (por causa do "R$" e da vírgula) -- deveria ser float
# "data_venda" é object -- deveria ser datetime

# 5. Inconsistências de texto (Tópico 4)
print(df["categoria"].unique())
# ['Papeleria' ' papeleria ' 'Acessorios' 'acessorios'] -- 4 valores para

# 6. Valores fora do esperado / outliers (Tópico 6)
print(df["quantidade"].describe())
# 999 destoa completamente do resto (3, 1, 2) -- provável erro de digitação

# APLICANDO AS CORREÇÕES, NA ORDEM QUE FAZ SENTIDO PARA ESTE CASO:

# a) remover duplicatas exatas primeiro (evita distorcer os passos seguint
df = df.drop_duplicates().reset_index(drop=True)

# b) corrigir tipo de preco
df["preco"] = (
    df["preco"].str.replace("R$", "", regex=False)
    .str.replace(",", ".", regex=False)
    .str.strip()
    .astype(float)
)

# c) corrigir tipo de data
df["data_venda"] = pd.to_datetime(df["data_venda"], format="%d/%m/%Y")

# d) padronizar texto de categoria
df["categoria"] = df["categoria"].str.strip().str.lower()

# e) tratar outlier em quantidade (999 claramente é erro -- decide-se remo
df = df[df["quantidade"] != 999]

# f) tratar valor ausente em quantidade (preenchendo com a mediana, já sem
df["quantidade"] = df["quantidade"].fillna(df["quantidade"].median())

# Checagem final -- confirma que os problemas foram resolvidos
print(df.isnull().sum().sum()) # 0
print(df.duplicated().sum()) # 0
print(df.dtypes) # preco float64, data_venda datetime64
print(df["categoria"].unique()) # só 2 valores, padronizados
print(df)

```

Erros comuns de quem está começando

- Aplicar as correções em ordem aleatória, sem pensar nas dependências entre elas — por exemplo, calcular uma mediana para preencher ausentes **antes** de remover um outlier que distorce essa mediana, ou tentar converter tipo antes de limpar o texto que

impede a conversão.

- Corrigir só o que “salta aos olhos” numa primeira olhada, sem rodar o roteiro completo (ausentes, duplicatas, tipos, texto, outliers) em cada coluna — problemas de qualidade de dados raramente vêm um de cada vez.
- Não validar o resultado final. Depois de limpar, é fácil assumir que “deu certo” sem checar de novo `isnull().sum()`, `duplicated().sum()`, `dtypes` e `.unique()` das colunas de texto — a validação final é parte do processo, não um passo opcional.

Exercício prático

```
csv_texto = """cliente,cidade,valor_compra,data
Marcos,  Sao Paulo,R$ 150,00,12/03/2024
Julia,Rio de Janeiro ,R$ 89,90,15/03/2024
Marcos,  Sao Paulo,R$ 150,00,12/03/2024
Pedro,rio de janeiro,R$ -20,00,20/03/2024
Ana,Belo Horizonte,R$ 300,50,
"""
```

Aplique o roteiro completo de diagnóstico e limpeza neste dataset:

1. Rode o diagnóstico (`.info()`, `.isnull().sum()`, `.duplicated().sum()`, `.unique()` das colunas de texto) e liste os problemas encontrados.
2. Remova duplicatas exatas.
3. Converta `valor_compra` para `float` (removendo "R\$" e trocando vírgula por ponto).
4. Padronize `cidade` (remova espaços, deixe minúsculo).
5. Trate o valor R\$ -20,00 (compra com valor negativo — decida remover a linha, já que valor de compra negativo não faz sentido de negócio).
6. Converta `data` para `datetime`, com `errors="coerce"` (a última linha não tem data).
7. Valide o resultado final.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Tenho um roteiro mental para diagnosticar um dataset novo (ausentes, duplicatas, tipos, texto, outliers)
- ☐ Sei pensar na ordem correta de aplicar correções (dependências entre elas)
- ☐ Sempre valido o resultado final depois de limpar
- ☐ Resolvi o exercício sem olhar a solução

Checklist do Módulo 5

Antes de seguir para o Módulo 6 (Estatística e EDA), confirme que você consegue, sem consultar a apostila:

- ☐ Decidir entre remover, preencher com estatística geral ou preencher por grupo para valores ausentes
- ☐ Identificar e remover duplicatas, com e sem `subset`
- ☐ Converter colunas de texto “sujo” para tipos numéricos e de data
- ☐ Padronizar texto (espaços, capitalização, acentos)
- ☐ Detectar outliers com IQR ou z-score, e escolher um tratamento justificado
- ☐ Aplicar um roteiro completo de diagnóstico e limpeza a um dataset novo

Pronto? Siga para o Módulo 6 — Estatística e EDA.

ewpage

Módulo 6 — Estatística e EDA

Com os dados já limpos (Módulo 5), chegou a hora de fazer perguntas a eles. EDA (*Exploratory Data Analysis*, Análise Exploratória de Dados) é o processo de investigar um dataset — resumir, visualizar e cruzar variáveis — para entender seus padrões, relações e surpresas antes de tirar qualquer conclusão ou construir um modelo em cima dele. A estatística descritiva (médias, dispersão, correlação) é a caixa de ferramentas dessa investigação.

Este módulo foca em estatística **descritiva** e exploração — o objetivo não é decidir com certeza matemática se algo é verdade (isso é inferência estatística, mencionada de leve no fim do módulo), mas sim entender bem os dados que você tem na mão e comunicar o que eles mostram.

Tópicos

1. Medidas de tendência central
2. Medidas de dispersão
3. Distribuições e histogramas
4. Correlação entre variáveis
5. Análise univariada com Pandas
6. Análise bivariada e segmentação
7. Amostragem e introdução à inferência
8. Estruturando uma EDA completa

Depois de terminar todos os tópicos, siga para o Módulo 7 — Visualização.

ewpage

Medidas de tendência central

| Módulo 6 — Estatística e EDA · Tópico 1 de 8

O que é e por que importa

Medidas de tendência central respondem “qual é o valor típico desses dados?” — um único número que resume o “centro” de uma coluna inteira. Você já usou algumas delas (`mean()`, `median()`) nos módulos anteriores; aqui o foco é entender **quando cada uma é apropriada**, porque escolher a errada pode levar a conclusões enganosas.

As três principais são:

- **Média (mean)**: soma de todos os valores dividida pela quantidade. Sensível a outliers — um único valor muito alto ou baixo puxa a média na sua direção (visto no Módulo 5, Tópico 6).
- **Mediana (median)**: o valor do meio quando os dados são ordenados (ou a média dos dois valores centrais, se a quantidade for par). Não é afetada por outliers da mesma forma que a média.
- **Moda (mode)**: o valor que mais se repete. É a única das três que também funciona bem para dados categóricos (texto), não só números.

Como funciona (com exemplo comentado)

```
import pandas as pd

salarios = pd.Series([3200, 3500, 3800, 3900, 4100, 4200, 50000])

print(salarios.mean())      # 10385.71... -- fortemente puxada pelo valor 50
print(salarios.median())    # 3900.0 -- representa melhor o "salário típico"
print(salarios.mode())      # sem moda clara neste caso (todos os valores são

# Quando os dados NÃO têm outliers extremos, média e mediana ficam próximas
notas = pd.Series([7.0, 7.5, 8.0, 6.5, 7.2])
print(notas.mean())         # 7.24
print(notas.median())       # 7.2
# valores próximos -- sinal de que a distribuição é razoavelmente "equilibrada"

# Moda em dados categóricos (texto) -- a média/mediana não fazem sentido aqui
categorias = pd.Series(["Papeleria", "Papeleria", "Acessorios", "Papeleria"])
print(categorias.mode())    # 0    Papeleria -- a categoria mais frequente

# .mode() pode retornar mais de um valor em caso de empate
empatado = pd.Series([1, 1, 2, 2, 3])
print(empatado.mode())     # 0    1 \n 1    2 -- dois valores empatados como 1 e 2

# Aplicando a um DataFrame inteiro (colunas numéricas)
df = pd.DataFrame({
    "produto": ["A", "B", "C", "D", "E"],
    "preco": [25.0, 30.0, 28.0, 32.0, 27.0],
    "avaliacao": [4.5, 3.0, 4.8, 4.2, 4.0],
})
print(df[["preco", "avaliacao"]].mean())
print(df[["preco", "avaliacao"]].median())

# .describe() (Módulo 4, Tópico 3) já traz mean e median (como "50%") juntamente
print(df["preco"].describe())

# Comparar mean e median é uma forma rápida de suspeitar de outliers ou
# assimetria na distribuição, mesmo antes de visualizar (Tópico 3)
diferenca = df["preco"].mean() - df["preco"].median()
print(diferenca)           # perto de 0 -> distribuição razoavelmente simétrica
```

Uma regra prática: sempre que `mean()` e `median()` estiverem bem diferentes uma da outra, é sinal de que a distribuição tem assimetria (mais valores concentrados de um lado) ou outliers puxando a média — vale investigar antes de reportar só a média como “o valor típico”.

Erros comuns de quem está começando

- Reportar só a média em dados que têm outliers (como salários, preços de imóveis, faturamento de empresas), sem perceber que ela não representa bem o “típico” nesses casos — a mediana costuma ser mais informativa quando a distribuição é bem assimétrica.
- Calcular média ou mediana de uma coluna categórica (texto) por engano — essas medidas só fazem sentido para números; para texto, a medida equivalente é a moda.
- Ignorar `.mode()` quando ela retorna mais de um valor (empate) e assumir erroneamente que só existe uma moda — sempre checar quantos valores `.mode()` retornou antes de usar `.mode()[0]` diretamente.

Exercício prático

```
vendas_diarias = pd.Series([1200, 1350, 1100, 1280, 1400, 1150, 15000])
categoria_mais_vendida = pd.Series(["Papeleria", "Eletronicos", "Papeleria"])
```

1. Calcule média e mediana de `vendas_diarias` e compare os dois valores.
2. Explique (em um comentário) por que eles são tão diferentes.

3. Calcule a moda de `categoria_mais_vendida`.
4. Recalcule a média de `vendas_diarias` **excluindo** o valor 15000 e compare com a mediana calculada no item 1 — elas deveriam ficar bem mais próximas.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei calcular média, mediana e moda com Pandas
- ☐ Sei explicar quando a mediana é mais apropriada que a média
- ☐ Sei que moda funciona tanto para números quanto para texto
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Medidas de dispersão

| Módulo 6 — Estatística e EDA · Tópico 2 de 8

O que é e por que importa

Saber o “valor típico” (Tópico 1) não conta a história toda. Duas turmas podem ter a mesma média de notas (7.0), mas uma com notas todas próximas de 7 (6.8, 7.0, 7.2) e outra com notas muito espalhadas (2.0, 7.0, 12.0 — bom, notas não passam de 10, mas a ideia é essa). **Medidas de dispersão** dizem o quanto os dados variam em torno do centro — sem elas, comparar dois conjuntos de dados só pela média pode esconder diferenças importantes.

As principais são:

- **Amplitude (range)**: diferença entre o maior e o menor valor. Simples, mas muito sensível a um único outlier.
- **Variância**: média dos quadrados das diferenças entre cada valor e a média. Difícil de interpretar diretamente (está na unidade “ao quadrado”).
- **Desvio padrão (standard deviation)**: raiz quadrada da variância — volta para a unidade original dos dados, por isso é mais usado na prática que a variância pura.
- **Amplitude interquartil (IQR)**: diferença entre o terceiro quartil (Q3) e o primeiro quartil (Q1) — já visto no Módulo 5, Tópico 6, para detectar outliers. Mede a dispersão só da “metade do meio” dos dados, ignorando os extremos.

Como funciona (com exemplo comentado)

```

import pandas as pd

turma_a = pd.Series([6.8, 7.0, 7.2, 6.9, 7.1])
turma_b = pd.Series([2.0, 5.0, 7.0, 9.0, 12.0])

print(turma_a.mean(), turma_b.mean()) # 7.0, 7.0 -- MESMA média!

# Amplitude -- diferença entre máximo e mínimo
amplitude_a = turma_a.max() - turma_a.min()
amplitude_b = turma_b.max() - turma_b.min()
print(amplitude_a, amplitude_b) # 0.4, 10.0 -- turma B muito mais espalha

# Variância
print(turma_a.var()) # bem pequena
print(turma_b.var()) # bem maior

# Desvio padrão -- raiz quadrada da variância, na mesma unidade dos dados
print(turma_a.std()) # ~0.16 -- notas bem próximas da média
print(turma_b.std()) # ~3.87 -- notas bem espalhadas em torno da média

# Coeficiente de variação -- desvio padrão relativo à média, útil para
# comparar dispersão de colunas com escalas diferentes (ex: preço vs quant
cv_a = turma_a.std() / turma_a.mean()
cv_b = turma_b.std() / turma_b.mean()
print(cv_a, cv_b) # quanto maior, mais disperso em relação ao próprio tam

# Quartis e IQR (revisão do Módulo 5, Tópico 6)
q1 = turma_b.quantile(0.25)
q3 = turma_b.quantile(0.75)
iqr = q3 - q1
print(f"Q1={q1}, Q3={q3}, IQR={iqr}")

# .describe() já traz quartis e desvio padrão juntos
print(turma_b.describe())
# count      5.000000
# mean       7.000000
# std        3.872983
# min        2.000000
# 25%        5.000000
# 50%        7.000000
# 75%        9.000000
# max       12.000000

# Aplicando a colunas de um DataFrame -- útil para comparar dispersão entr
df = pd.DataFrame({
    "loja": ["A", "A", "A", "B", "B", "B"],
    "venda_diaria": [1000, 1050, 980, 500, 1500, 800],
})
resumo = df.groupby("loja")["venda_diaria"].agg(["mean", "std"])
print(resumo)
# mesmo com médias parecidas, a loja B pode ter desvio padrão bem maior --
# suas vendas são mais imprevisíveis dia a dia

```

Erros comuns de quem está começando

- Comparar apenas as médias de dois grupos e concluir que são “iguais” sem olhar a dispersão — como no exemplo das turmas A e B, médias iguais podem esconder distribuições completamente diferentes.
- Usar amplitude (max - min) como medida principal de dispersão em dados com outliers — como ela só olha os dois valores extremos, um único ponto fora do padrão distorce a métrica inteira; desvio padrão ou IQR costumam ser mais robustos.
- Confundir variância com desvio padrão ao interpretar um resultado — a variância está numa unidade “ao quadrado” (ex: reais², anos²), o que não tem interpretação prática direta; o desvio padrão (raiz da variância) volta para a unidade original e é o que normalmente se usa para comunicar dispersão.

Exercício prático

```
vendedor_x = pd.Series([500, 520, 480, 510, 490])
vendedor_y = pd.Series([200, 800, 300, 700, 500])
```

1. Calcule a média de vendas de cada vendedor e compare.
2. Calcule o desvio padrão de cada um e compare.
3. Com base nos dois resultados, escreva (em um comentário) qual vendedor tem performance mais **consistente** (não necessariamente quem vende mais em média).
4. Calcule o IQR de cada vendedor.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei calcular amplitude, variância, desvio padrão e IQR com Pandas
- ☐ Entendo por que desvio padrão é preferido à variância para comunicar resultados
- ☐ Sei que duas séries com médias iguais podem ter dispersões muito diferentes
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Distribuições e histogramas

| Módulo 6 — Estatística e EDA · Tópico 3 de 8

O que é e por que importa

Distribuição é como os valores de uma coluna se espalham: quais faixas de valor são mais comuns, quais são raras, se os dados são simétricos ou “puxados” para um lado. Média, mediana e desvio padrão (Tópicos 1 e 2) resumem a distribuição em números; um **histograma** mostra ela visualmente, agrupando os valores em faixas (chamadas de *bins*) e contando quantos caem em cada uma.

Este módulo usa gráficos de forma simples, via `.plot()` do próprio Pandas (que por baixo usa Matplotlib) — o Módulo 7 (Visualização) aprofunda a construção de gráficos com mais controle e qualidade visual. Aqui o objetivo é usar o histograma como **ferramenta de investigação**, não como gráfico final para apresentar.

Como funciona (com exemplo comentado)

```

import pandas as pd

precos = pd.Series([10, 12, 11, 13, 45, 12, 11, 10, 14, 13, 12, 46, 11, 12

# .plot(kind="hist") -- gera um histograma rapidamente, ótimo para explorar
# (requer matplotlib instalado: pip install matplotlib)
precos.plot(kind="hist", bins=8, title="Distribuição de preços")
# em um notebook Jupyter, o gráfico aparece direto; em um script .py,
# use plt.show() do matplotlib.pyplot para abrir a janela do gráfico

# Sem depender de gráfico, dá pra "aproximar" um histograma com pd.cut,
# que divide os valores em faixas (bins) e conta quantos caem em cada uma
faixas = pd.cut(precos, bins=5)
print(faixas.value_counts().sort_index())
# (9.964, 17.2]    12
# (17.2, 24.4]    0
# (24.4, 31.6]    0
# (31.6, 38.8]    0
# (38.8, 46.0]    3
# -- confirma visualmente (em texto) que a maioria dos valores está entre
#     com um grupo separado bem mais alto (perto de 45)

# Formas comuns de distribuição, e o que dizem sobre os dados:
# - Simétrica (parecida com um sino): média e mediana próximas
# - Assimétrica à direita (cauda longa de valores altos): média > mediana
#   (comum em preços, salários, tempos de espera)
# - Assimétrica à esquerda (cauda longa de valores baixos): média < mediana
# - Bimodal (dois "picos"): sugere que os dados vêm de dois grupos misturados
#   (ex: preços de produtos populares E de produtos de luxo juntos na mesma

print(precos.mean(), precos.median())
# média puxada para cima pela cauda de valores altos (45, 46) -> assimetria

# .skew() -- mede numericamente a assimetria (skewness)
print(precos.skew())
# positivo -> assimetria à direita (cauda para valores altos)
# negativo -> assimetria à esquerda
# perto de 0 -> distribuição aproximadamente simétrica

# Detectando possível bimodalidade com value_counts de faixas (bins)
vendas_por_horario = pd.Series([9, 9, 10, 12, 12, 13, 18, 19, 19, 20, 12,
faixas_horario = pd.cut(vendas_por_horario, bins=4)
print(faixas_horario.value_counts().sort_index())
# se os valores se concentrarem em dois grupos separados (não um centro só
# é um indício de bimodalidade -- vale investigar se há dois "tipos" de
# situação misturados na mesma coluna (ex: horário de almoço x horário de

```

Erros comuns de quem está começando

- Escolher um número de bins (faixas) arbitrário sem testar variações — poucos bins escondem detalhes da distribuição; bins demais criam ruído visual que parece padrão sem ser. Vale testar 2-3 valores diferentes de bins antes de tirar conclusões.
- Olhar só a média e ignorar a forma da distribuição — como visto no exemplo, uma cauda de valores altos distorce a média sem necessariamente “quebrar” nada; entender a forma da distribuição evita interpretar mal esse tipo de padrão.
- Confundir distribuição bimodal com outliers — os dois são fenômenos diferentes: outliers são poucos pontos isolados fora do padrão; uma distribuição bimodal tem **dois grupos inteiros** de dados concentrados em faixas diferentes, cada um com vários pontos.

Exercício prático

```

tempos_de_espera = pd.Series([2, 3, 3, 4, 2, 3, 5, 4, 3, 25, 2, 3, 4, 3, 2

```

(tempos de espera em minutos de atendimentos, com dois atendimentos que demoraram muito mais que o normal)

1. Calcule média, mediana e `.skew()` dessa série.
2. Use `pd.cut` com 5 faixas e `.value_counts()` para ver como os valores se distribuem.
3. Com base nos resultados, escreva (em um comentário) se a distribuição parece simétrica, assimétrica à direita, ou à esquerda.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei explicar o que é um histograma e o que ele revela sobre os dados
- ☐ Sei interpretar assimetria comparando média e mediana, e usando `.skew()`
- ☐ Sei diferenciar uma distribuição bimodal de outliers isolados
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Correlação entre variáveis

| Módulo 6 — Estatística e EDA · Tópico 4 de 8

O que é e por que importa

Correlação mede o quanto duas variáveis numéricas se movem **juntas**: quando uma aumenta, a outra também aumenta (correlação positiva)? Diminui (correlação negativa)? Ou não há relação aparente (correlação perto de zero)? É uma das perguntas mais comuns em EDA — “o preço tem relação com a avaliação do produto?”, “gastos em marketing se relacionam com vendas?” — e o primeiro passo antes de qualquer modelo preditivo.

O coeficiente mais comum é o de **Pearson**, que varia de -1 a 1:

- **+1**: correlação positiva perfeita (uma sobe exatamente na mesma proporção que a outra).
- **-1**: correlação negativa perfeita (uma sobe exatamente na proporção inversa da outra).
- **0**: nenhuma correlação linear detectável.

Valores intermediários (0.3, 0.7, -0.5...) indicam correlações mais fracas ou mais fortes.

Como funciona (com exemplo comentado)

```
import pandas as pd

df = pd.DataFrame({
    "horas_estudo": [1, 2, 3, 4, 5, 6, 7, 8],
    "nota_prova": [3.0, 4.2, 5.0, 5.8, 7.1, 7.5, 8.9, 9.5],
    "faltas": [8, 7, 6, 5, 3, 2, 1, 0],
})

# .corr() entre duas colunas -- retorna um único número
correlacao_horas_nota = df["horas_estudo"].corr(df["nota_prova"])
print(correlacao_horas_nota) # perto de 1 -- forte correlação positiva

correlacao_horas_faltas = df["horas_estudo"].corr(df["faltas"])
print(correlacao_horas_faltas) # perto de -1 -- forte correlação negativa

# .corr() no DataFrame inteiro -- matriz de correlação entre TODAS as colunas
matriz_correlacao = df.corr(numeric_only=True)
print(matriz_correlacao)
#               horas_estudo  nota_prova  faltas
# horas_estudo      1.000000    0.992... -0.994...
# nota_prova         0.992...    1.000000 -0.980...
# faltas             -0.994...   -0.980...  1.000000
# -- a diagonal é sempre 1 (uma coluna tem correlação perfeita com ela mesma)
# a matriz é simétrica (corr entre A e B é igual à corr entre B e A)

# Interpretação prática de faixas de força (regra geral, não absoluta):
# |r| < 0.3          -> fraca ou nenhuma
# 0.3 <= |r| < 0.7   -> moderada
# |r| >= 0.7         -> forte

# Correlação NÃO implica causalidade -- clássico aviso da estatística.
# Duas variáveis podem se mover juntas por coincidência ou por uma terceira
# causa comum, sem que uma cause a outra diretamente. Exemplo clássico:
# vendas de sorvete e afogamentos sobem juntos no verão -- não é que sorve
# cause afogamento, o calor do verão influencia os dois.

# Correlação captura relação LINEAR -- duas variáveis podem ter uma relação
# forte, só que não-linear (curva), e o coeficiente de Pearson não detecta
# isso bem. Sempre vale visualizar (Módulo 7) além de calcular o número.
```

Erros comuns de quem está começando

- Concluir que uma variável **causa** o comportamento da outra só porque a correlação é alta — correlação é só um indício estatístico, nunca prova de causalidade. Estabelecer causa exige desenho experimental (fora do escopo desta apostila), não apenas cálculo de correlação.
- Calcular `.corr()` em colunas de texto/catóricas sem converter antes — `numeric_only=True` evita erro ao rodar `.corr()` num DataFrame com colunas mistas, ignorando automaticamente as não-numéricas.
- Confiar cegamente no número de correlação sem visualizar os dados — relações não-lineares (em forma de U, por exemplo) podem ter correlação de Pearson próxima de zero mesmo havendo uma relação forte entre as variáveis, só que não-linear.

Exercício prático

```
df = pd.DataFrame({
    "temperatura": [15, 18, 22, 25, 28, 30, 32, 20],
    "vendas_sorvete": [50, 80, 120, 180, 250, 300, 320, 100],
    "vendas_casaco": [200, 150, 90, 60, 20, 10, 5, 130],
})
```

1. Calcule a correlação entre temperatura e vendas_sorvete.
2. Calcule a correlação entre temperatura e vendas_casaco.
3. Gere a matriz de correlação completa do DataFrame.

4. Com base nos números, escreva (em um comentário) uma frase interpretando cada correlação, tomando cuidado para não afirmar causalidade.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei calcular correlação entre duas colunas e a matriz de correlação de um DataFrame
- ☐ Sei interpretar o sinal (positivo/negativo) e a força (perto de 0, 0.5, 1) de uma correlação
- ☐ Sei explicar por que correlação não implica causalidade
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Análise univariada com Pandas

| Módulo 6 — Estatística e EDA · Tópico 5 de 8

O que é e por que importa

Análise **univariada** examina **uma variável de cada vez**, isoladamente — é o primeiro passo de qualquer EDA, antes de cruzar variáveis entre si (Tópico 6). Para cada coluna do dataset, o objetivo é responder: qual é o tipo dela (numérica ou categórica)? Qual sua distribuição? Tem valores estranhos? Este tópico junta as ferramentas dos tópicos anteriores (medidas de tendência central, dispersão, distribuição) num fluxo de trabalho aplicado, coluna por coluna.

Como funciona (com exemplo comentado)

```

import pandas as pd

df = pd.DataFrame({
    "produto": ["Caneta", "Caderno", "Mochila", "Lapis", "Estojo", "Borrac",
    "categoria": ["Papellaria", "Papellaria", "Acessorios", "Papellaria", "Ac",
    "preco": [2.5, 15.9, 89.9, 1.2, 25.0, 0.9, 2.5],
    "quantidade_vendida": [120, 45, 12, 200, 30, 300, 95],
})

# ANÁLISE UNIVARIADA DE UMA COLUNA NUMÉRICA: "preco"
print(df["preco"].describe())
# count, mean, std, min, 25%, 50%, 75%, max -- tudo de uma vez

print("Assimetria (skew):", df["preco"].skew())
print("Valores únicos:", df["preco"].nunique())

# faixas (bins) para "ver" a distribuição sem gráfico ainda
print(pd.cut(df["preco"], bins=4).value_counts().sort_index())

# ANÁLISE UNIVARIADA DE UMA COLUNA CATEGÓRICA: "categoria"
print(df["categoria"].value_counts()) # contagem absoluta
print(df["categoria"].value_counts(normalize=True)) # proporção (0 a 1)
print(df["categoria"].value_counts(normalize=True) * 100) # em porcentagem

print("Categorias distintas:", df["categoria"].nunique())
print("Categoria mais comum:", df["categoria"].mode()[0])

# Função reutilizável para resumir QUALQUER coluna numérica de uma vez
def resumo_numerico(serie):
    return pd.Series({
        "media": serie.mean(),
        "mediana": serie.median(),
        "desvio_padrao": serie.std(),
        "minimo": serie.min(),
        "maximo": serie.max(),
        "assimetria": serie.skew(),
        "valores_ausentes": serie.isnull().sum(),
    })

print(resumo_numerico(df["preco"]))
print(resumo_numerico(df["quantidade_vendida"]))

# Aplicando a função a todas as colunas numéricas de uma vez com .apply()
# (revisão de Módulo 4, Tópico 6 -- .apply() em colunas de um DataFrame, a
colunas_numericas = df.select_dtypes(include="number")
print(colunas_numericas.apply(resumo_numerico))

# .select_dtypes() -- útil para separar automaticamente colunas numéricas
colunas_categoricas = df.select_dtypes(include="object")
print(colunas_categoricas.columns.tolist())

```

Erros comuns de quem está começando

- Pular a análise univariada e ir direto para cruzamentos entre variáveis (Tópico 6) — sem entender cada coluna isoladamente primeiro, é fácil interpretar mal um cruzamento que na verdade reflete um problema simples numa única coluna (um outlier, uma categoria mal escrita).
- Tratar uma coluna numérica com poucos valores distintos (como uma nota de 1 a 5, ou um código de categoria numérico) como se fosse contínua, calculando média/desvio padrão quando na verdade ela deveria ser analisada como categórica (com `value_counts()`).
- Analisar `value_counts()` de uma coluna categórica sem `normalize=True` quando o objetivo é comparar proporções entre datasets de tamanhos diferentes — contagens absolutas de datasets de tamanho diferente não são diretamente comparáveis.

Exercício prático

```
df = pd.DataFrame({
    "cliente": ["A", "B", "C", "D", "E", "F", "G"],
    "idade": [25, 34, 41, 29, 55, 38, 45],
    "plano": ["Basico", "Premium", "Basico", "Basico", "Premium", "Premium", "Premium"],
    "gasto_mensal": [50, 150, 45, 60, 200, 180, 55],
})
```

1. Faça a análise univariada completa da coluna `idade` (média, mediana, desvio padrão, min, max, skew).
2. Faça a análise univariada da coluna `plano` (contagem absoluta e percentual de cada categoria).
3. Use `.select_dtypes()` para separar as colunas numéricas das categóricas.
4. Aplique a função `resumo_numerico` (do exemplo) a todas as colunas numéricas de uma vez.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei fazer a análise univariada de uma coluna numérica (tendência central + dispersão + forma)
- ☐ Sei fazer a análise univariada de uma coluna categórica com `value_counts()`
- ☐ Sei usar `.select_dtypes()` para separar colunas numéricas e categóricas
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Análise bivariada e segmentação

Módulo 6 — Estatística e EDA · Tópico 6 de 8

O que é e por que importa

Depois de entender cada coluna sozinha (Tópico 5), o próximo passo é cruzá-las: “o gasto médio muda entre os planos Básico e Premium?”, “clientes de qual cidade têm mais atraso no pagamento?”. Isso é análise **bivariada** (duas variáveis) — e é aqui que `groupby` (Módulo 4, Tópico 8), `pivot_table` (Módulo 4, Tópico 12) e correlação (Tópico 4 deste módulo) se combinam para revelar padrões que uma coluna sozinha não mostra.

Como funciona (com exemplo comentado)

```
import pandas as pd

df = pd.DataFrame({
    "cliente": ["A", "B", "C", "D", "E", "F", "G", "H"],
    "plano": ["Basico", "Premium", "Basico", "Basico", "Premium", "Premium", "Premium", "Premium"],
    "cidade": ["SP", "SP", "RJ", "RJ", "SP", "RJ", "SP", "RJ"],
    "gasto_mensal": [50, 150, 45, 60, 200, 180, 55, 190],
    "meses_ativo": [12, 24, 6, 18, 30, 15, 9, 20],
})

# NUMÉRICA x CATEGÓRICA: comparar uma métrica entre grupos -- groupby + agg
resumo_por_plano = df.groupby("plano")["gasto_mensal"].agg(["mean", "median", "std", "count"])
print(resumo_por_plano)
#          mean  median      std  count
# plano
# Basico    52.5    52.5    6.454972     4
# Premium  180.0   185.0   20.816660     4
# -- Premium gasta bem mais em média, e tem mais variação entre clientes

# CATEGÓRICA x CATEGÓRICA: tabela de contingência com pd.crosstab
tabela_cruzada = pd.crosstab(df["plano"], df["cidade"])
print(tabela_cruzada)
# cidade  RJ  SP
# plano
# Basico    2   2
# Premium    2   2

# crosstab com porcentagens (normalize) -- mais fácil de comparar proporção
tabela_percentual = pd.crosstab(df["plano"], df["cidade"], normalize="index")
print(tabela_percentual)
# dentro de cada plano, qual % está em cada cidade

# NUMÉRICA x NUMÉRICA: correlação (Tópico 4) e agrupamento por faixas
correlacao = df["meses_ativo"].corr(df["gasto_mensal"])
print(correlacao) # clientes mais antigos gastam mais? checa aqui

# Segmentando uma variável numérica em faixas para cruzar como se fosse ca
df["faixa_tempo"] = pd.cut(
    df["meses_ativo"],
    bins=[0, 10, 20, 31],
    labels=["novo (0-10m)", "intermediario (11-20m)", "antigo (21m+)"],
)
resumo_por_faixa = df.groupby("faixa_tempo", observed=True)["gasto_mensal"]
print(resumo_por_faixa)

# pivot_table combina duas dimensões categóricas com uma métrica numérica
tabela_dinamica = df.pivot_table(
    index="plano", columns="cidade", values="gasto_mensal", aggfunc="mean"
)
print(tabela_dinamica)
```

O padrão geral da análise bivariada é sempre um destes três:

- **numérica x categórica** → groupby + agregação (comparar a métrica entre grupos)
- **categórica x categórica** → pd.crosstab (contar/cruzar combinações)
- **numérica x numérica** → .corr() (medir associação linear)

Erros comuns de quem está começando

- Comparar médias entre grupos sem olhar também a dispersão (std) e o tamanho de cada grupo (count) — um grupo com poucas observações pode ter uma média enganosa, fácil de ser distorcida por um ou dois valores atípicos.
- Usar pd.crosstab com contagens absolutas quando os grupos têm tamanhos muito diferentes, dificultando a comparação — nesses casos, normalize="index" ou normalize="columns" deixa a comparação de proporções mais justa.
- Tentar cruzar uma variável numérica contínua diretamente com outra categórica

através de `crosstab` (que espera categorias discretas) — o correto é primeiro discretizar a numérica com `pd.cut` (como no exemplo de `faixa_tempo`), ou usar `groupby` em vez de `crosstab`.

Exercício prático

```
df = pd.DataFrame({
    "funcionario": ["A", "B", "C", "D", "E", "F"],
    "departamento": ["Vendas", "TI", "Vendas", "TI", "Vendas", "TI"],
    "nivel": ["Junior", "Senior", "Pleno", "Junior", "Senior", "Pleno"],
    "salario": [3000, 8000, 5000, 3200, 9000, 5500],
})
```

1. Calcule média e desvio padrão de `salario` por departamento.
2. Crie uma tabela cruzada (`crosstab`) entre departamento e `nivel`.
3. Crie uma tabela dinâmica (`pivot_table`) com a média salarial por departamento e `nivel`.
4. Segmente `salario` em 3 faixas usando `pd.cut` e calcule a contagem de funcionários por faixa e departamento com `crosstab`.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Sei escolher a técnica certa (`groupby`, `crosstab` ou `corr`) de acordo com os tipos das variáveis
- ☐ Sei usar `pd.crosstab` com `normalize` para comparar proporções
- ☐ Sei discretizar uma variável numérica com `pd.cut` para cruzá-la com outras
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Amostragem e introdução à inferência

| Módulo 6 — Estatística e EDA · Tópico 7 de 8

O que é e por que importa

Até aqui, toda a estatística que você calculou foi **descritiva**: resume os dados que você **tem**. Mas frequentemente os dados que você tem são só uma **amostra** de um grupo maior (uma **população**) — uma pesquisa com 500 clientes representando milhões, um teste A/B com uma fração dos usuários de um site. Estatística **inferencial** é o ramo que usa a amostra para tirar conclusões sobre a população inteira, com um nível de confiança calculável.

Este tópico é só uma introdução conceitual — dá a base para você reconhecer quando uma pergunta é de inferência (não só descrição) e entender a ideia de intervalo de confiança. Testes de hipótese formais e inferência aprofundada ficam fora do escopo desta apostila introdutória.

Como funciona (com exemplo comentado)

```

import pandas as pd
import numpy as np

# Simulando uma "população" inteira de 10.000 clientes (na prática você
# raramente tem a população inteira -- aqui é só para fins didáticos)
np.random.seed(42)
populacao = pd.Series(np.random.normal(loc=150, scale=40, size=10000))
# loc=150 é a média "real" da população, scale=40 é o desvio padrão "real"

print("Média real da população:", populacao.mean()) # próxima de 150

# Na prática, você não tem a população inteira -- só consegue uma AMOSTRA
amostra = populacao.sample(n=100, random_state=1)
print("Média da amostra:", amostra.mean()) # próxima de 150, mas não exat

# Quanto maior a amostra, mais a média da amostra tende a se aproximar da
# média real da população -- é a ideia central por trás de "quanto maior a
# amostra, mais confiável a estimativa"
for tamanho in [10, 50, 100, 1000]:
    amostra_teste = populacao.sample(n=tamanho, random_state=1)
    print(f"n={tamanho}: média da amostra = {amostra_teste.mean():.2f}")

# Erro padrão da média -- estima o quanto a média de UMA amostra tende a
# variar em torno da média real, dependendo do tamanho da amostra
erro_padrao = amostra.std() / np.sqrt(len(amostra))
print("Erro padrão:", erro_padrao)

# Intervalo de confiança de ~95% (aproximação simples, usando 1.96 desvios
# padrão -- regra prática para amostras razoavelmente grandes)
media_amostra = amostra.mean()
margem = 1.96 * erro_padrao
intervalo = (media_amostra - margem, media_amostra + margem)
print(f"Média estimada: {media_amostra:.2f}")
print(f"Intervalo de confiança de 95%: {intervalo[0]:.2f} a {intervalo[1]:.2f}")
# interpretação: "temos 95% de confiança de que a média REAL da população
# está entre esses dois valores" -- não é uma garantia absoluta, é uma
# estimativa com margem de erro conhecida

# Amostragem aleatória simples vs. amostragem estratificada
df = pd.DataFrame({
    "cliente": range(1, 11),
    "plano": ["Basico"] * 7 + ["Premium"] * 3,
    "gasto": [50, 55, 48, 60, 52, 58, 51, 200, 210, 195],
})

# amostra aleatória simples -- pode, por azar, pegar poucos ou nenhum Prem
amostra_simples = df.sample(n=4, random_state=1)
print(amostra_simples)

# amostragem estratificada -- garante representação proporcional de cada g
amostra_estratificada = df.groupby("plano", group_keys=False).apply(
    lambda grupo: grupo.sample(frac=0.4, random_state=1)
)
print(amostra_estratificada)
# mantém a PROPORÇÃO de Básico/Premium da população original na amostra

```

Erros comuns de quem está começando

- Tratar qualquer conjunto de dados disponível como se fosse a população inteira, sem se perguntar se ele é representativo do grupo maior que realmente interessa (por exemplo, dados só de clientes que responderam uma pesquisa, ignorando os que não responderam).
- Tirar conclusões fortes de amostras muito pequenas, sem considerar que o erro padrão (e portanto a incerteza da estimativa) cresce bastante quando n é pequeno.
- Usar amostragem aleatória simples quando existem subgrupos importantes de tamanhos muito diferentes na população (como planos Básico e Premium com

proporções bem desiguais) — isso pode gerar amostras que sub-representam ou super-representam um subgrupo por acaso; amostragem estratificada resolve esse problema.

Exercício prático

```
np.random.seed(7)
populacao_notas = pd.Series(np.random.normal(loc=7.0, scale=1.2, size=5000))
```

1. Calcule a média real dessa “população” de notas.
2. Retire uma amostra de 30 valores (`random_state=1`) e calcule sua média.
3. Calcule o erro padrão da amostra e o intervalo de confiança aproximado de 95% ($1.96 * \text{erro_padrao}$).
4. Repita o processo com uma amostra de 300 valores e compare o tamanho do intervalo de confiança com o do item 3 — ele deveria ficar mais estreito (mais preciso) com a amostra maior.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Sei explicar a diferença entre estatística descritiva e inferencial
- ☐ Sei calcular erro padrão e um intervalo de confiança aproximado
- ☐ Sei explicar por que amostras maiores dão estimativas mais precisas
- ☐ Sei quando amostragem estratificada é preferível à aleatória simples
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Estruturando uma EDA completa

Módulo 6 — Estatística e EDA · Tópico 8 de 8 (último do módulo)

O que é e por que importa

Você já tem todas as ferramentas individuais (tendência central, dispersão, distribuição, correlação, análise uni e bivariada, noção de amostragem). Este último tópico junta tudo num **roteiro** que você pode aplicar a qualquer dataset novo — a estrutura que analistas de dados seguem (com variações) sempre que abrem um dataset pela primeira vez, antes de visualizar (Módulo 7) ou modelar.

Como funciona (com exemplo comentado)

```
import pandas as pd
import numpy as np
from io import StringIO

csv_texto = """cliente,plano,cidade,idade,gasto_mensal,meses_ativo
1,Basico,SP,25,50,12
2,Premium,SP,34,150,24
3,Basico,RJ,41,45,6
4,Basico,RJ,29,60,18
5,Premium,SP,55,200,30
6,Premium,RJ,38,180,15
7,Basico,SP,45,55,9
8,Premium,RJ,31,100,20
```

```

0, Premium, RJ, 51, 180, 20
"""

df = pd.read_csv(StringIO(csv_texto))

# ROTEIRO DE UMA EDA (assumindo que a limpeza do Módulo 5 já foi feita)

# 1. Visão geral do dataset
print(df.shape)
df.info()

# 2. Separar colunas numéricas e categóricas (Tópico 5)
numericas = df.select_dtypes(include="number").columns.tolist()
categoricas = df.select_dtypes(include="object").columns.tolist()
print("Numéricas:", numericas)
print("Categóricas:", categoricas)

# 3. Análise univariada de cada coluna numérica (Tópicos 1, 2, 3, 5)
for coluna in numericas:
    if coluna == "cliente":
        continue # ID não é uma métrica de negócio, pular
    print(f"\n--- {coluna} ---")
    print(df[coluna].describe())
    print("Assimetria:", df[coluna].skew())

# 4. Análise univariada de cada coluna categórica (Tópico 5)
for coluna in categoricas:
    if coluna == "cliente":
        continue
    print(f"\n--- {coluna} ---")
    print(df[coluna].value_counts(normalize=True) * 100)

# 5. Matriz de correlação entre as numéricas (Tópico 4)
print(df[numericas].corr(numeric_only=True))

# 6. Cruzamentos relevantes para as perguntas de negócio (Tópico 6)
# pergunta: "clientes Premium gastam significativamente mais?"
print(df.groupby("plano")["gasto_mensal"].agg(["mean", "std", "count"]))

# pergunta: "há relação entre tempo de ativo e gasto mensal?"
print(df["meses_ativo"].corr(df["gasto_mensal"]))

# pergunta: "a distribuição de planos varia por cidade?"
print(pd.crosstab(df["cidade"], df["plano"], normalize="index") * 100)

# 7. Sintetizar achados em texto -- a EDA só é útil se comunicada.
# Um bom resumo de EDA responde:
# - Quantas linhas/colunas, e qual a qualidade geral dos dados?
# - Quais as principais métricas numéricas (tendência central e dispersão)
# - Que padrões interessantes apareceram nos cruzamentos?
# - Que perguntas ficaram em aberto ou precisam de mais dados?

achados = """
Achados da EDA:
- Dataset com 8 clientes, sem valores ausentes.
- Clientes Premium gastam em média 180, contra 52.5 dos Básicos --
  diferença expressiva, mas amostra pequena para confirmar estatisticament
- Correlação entre meses_ativo e gasto_mensal: forte e positiva --
  clientes mais antigos tendem a gastar mais (ou vice-versa: quem gasta ma
  permanece mais tempo -- direção da causa não é possível afirmar só com i
- Distribuição de planos parece similar entre SP e RJ, sem padrão geográfi
"""

print(achados)

```

Erros comuns de quem está começando

- Rodar análises soltas, sem um roteiro, e esquecer de examinar alguma coluna importante — seguir uma sequência fixa (geral → univariada → bivariada → síntese)

evita esse tipo de lacuna.

- Fazer toda a análise e não escrever uma síntese em texto — números e tabelas sozinhos raramente comunicam bem para quem vai usar a análise; o passo final de “traduzir em frases” é parte do trabalho, não um extra.
- Tirar conclusões fortes demais de amostras pequenas (como no exemplo, 8 clientes) sem reconhecer a limitação — vale sempre mencionar o tamanho da amostra ao relatar um achado, especialmente quando ele é pequeno (conexão com o Tópico 7, sobre amostragem).

Exercício prático

Usando o dataset abaixo, escreva o roteiro completo de EDA (visão geral, univariada, bivariada, síntese em texto):

```
df = pd.DataFrame({
    "produto": ["A", "B", "C", "D", "E", "F", "G", "H"],
    "categoria": ["Eletronico", "Roupa", "Eletronico", "Roupa", "Eletronico", "Roupa", "Eletronico", "Roupa"],
    "preco": [899.0, 79.9, 1200.0, 129.9, 450.0, 199.9, 89.9, 59.9],
    "avaliacao": [4.5, 3.8, 4.7, 4.0, 4.2, 3.5, 4.1, 3.9],
    "vendas_mes": [50, 200, 20, 150, 80, 60, 100, 300],
})
```

1. Visão geral (shape, info).
2. Análise univariada de preco, avaliacao e vendas_mes.
3. Análise univariada de categoria.
4. Correlação entre preco, avaliacao e vendas_mes — o preço parece afetar as vendas?
5. Gasto/vendas médias por categoria (groupby).
6. Escreva 3-4 frases de síntese com os principais achados.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei seguir um roteiro estruturado de EDA do início ao fim
- ☐ Sei sintetizar os achados numéricos em frases claras
- ☐ Reconheço as limitações de conclusões tiradas de amostras pequenas
- ☐ Resolvi o exercício sem olhar a solução

Checklist do Módulo 6

Antes de seguir para o Módulo 7 (Visualização), confirme que você consegue, sem consultar a apostila:

- ☐ Calcular e interpretar média, mediana e moda, sabendo quando cada uma é mais apropriada
- ☐ Calcular e interpretar variância, desvio padrão e IQR
- ☐ Reconhecer o formato de uma distribuição (simétrica, assimétrica, bimodal) a partir de estatísticas
- ☐ Calcular e interpretar correlação entre variáveis, sem confundir com causalidade
- ☐ Fazer análise univariada e bivariada de um dataset novo
- ☐ Explicar a diferença entre estatística descritiva e inferencial
- ☐ Estruturar e comunicar os achados de uma EDA completa

Pronto? Siga para o Módulo 7 — Visualização.

ewpage

Módulo 7 — Visualização

Você já sabe resumir dados em números (Módulo 6) — agora é hora de resumir em **imagens**. Um bom gráfico comunica um padrão em segundos, algo que uma tabela de números exigiria minutos de leitura atenta para perceber. Este módulo ensina a construir gráficos com Matplotlib (a biblioteca de visualização mais fundamental do ecossistema Python) e Seaborn (construída em cima dela, com gráficos estatísticos prontos e visual mais elegante por padrão).

O objetivo não é decorar toda a API de nenhuma das duas bibliotecas — é saber escolher o gráfico certo para cada pergunta e construí-lo sem lutar contra a ferramenta.

Tópicos

1. Introdução ao Matplotlib
2. Gráficos de linha e barra
3. Histogramas e boxplots
4. Scatter plots e relação entre variáveis
5. Customização de gráficos
6. Introdução ao Seaborn
7. Escolhendo o gráfico certo

Depois de terminar todos os tópicos, siga para o Projeto Final.

ewpage

Introdução ao Matplotlib

Módulo 7 — Visualização · Tópico 1 de 7

O que é e por que importa

Matplotlib é a biblioteca de visualização mais fundamental do Python — quase todas as outras (incluindo o `.plot()` do Pandas, visto de leve no Módulo 6, e o Seaborn, visto no Tópico 6 deste módulo) são construídas em cima dela ou inspiradas nela. Entender sua estrutura básica (mesmo que você acabe usando Seaborn no dia a dia) é o que permite customizar qualquer detalhe de um gráfico quando as bibliotecas de mais alto nível não oferecem a opção pronta.

Instalação (se ainda não tiver):

```
pip install matplotlib
```

A convenção universal é importar o submódulo `pyplot` com o apelido `plt`:

```
import matplotlib.pyplot as plt
```

Como funciona (com exemplo comentado)

```

import matplotlib.pyplot as plt

# Estrutura básica: uma FIGURE (a "folha" inteira) contém um ou mais AXES
# (os "eixos", a área onde o gráfico de fato é desenhado)
fig, ax = plt.subplots() # cria uma figura com um único conjunto de eixos

x = [1, 2, 3, 4, 5]
y = [10, 25, 15, 30, 22]

ax.plot(x, y) # desenha uma linha conectando os pontos (x, y)
ax.set_title("Vendas por dia")
ax.set_xlabel("Dia")
ax.set_ylabel("Vendas (R$)")

plt.show() # em um script .py, abre a janela do gráfico
# em um notebook Jupyter, o gráfico costuma aparecer automaticamente após
# célula, mas plt.show() continua funcionando sem problema

# Forma mais direta (sem criar fig/ax explicitamente) -- comum para gráficos
# rápidos de exploração, mas dá menos controle
plt.plot(x, y)
plt.title("Vendas por dia (forma direta)")
plt.show()

# Salvando um gráfico em arquivo, em vez de (ou além de) mostrar na tela
fig, ax = plt.subplots()
ax.plot(x, y)
fig.savefig("grafico_vendas.png", dpi=150, bbox_inches="tight")
# dpi controla a resolução; bbox_inches="tight" evita cortar rótulos nas b

# Múltiplos gráficos na mesma figura -- subplots com mais de um eixo
fig, eixos = plt.subplots(1, 2, figsize=(10, 4)) # 1 linha, 2 colunas de
eixos[0].plot(x, y)
eixos[0].set_title("Gráfico 1")
eixos[1].bar(x, y)
eixos[1].set_title("Gráfico 2")
plt.tight_layout() # ajusta espaçamento para os títulos/rótulos não se so
plt.show()

```

A dupla `fig, ax = plt.subplots()` é o ponto de partida recomendado para praticamente todo gráfico a partir de agora: `fig` controla propriedades da figura inteira (tamanho, salvar em arquivo), `ax` controla o conteúdo do gráfico em si (dados, títulos, eixos). Essa é a chamada **abordagem orientada a objetos** do Matplotlib, mais previsível que a forma direta (`plt.plot(...)` sem `ax`) quando os gráficos ficam mais complexos.

Erros comuns de quem está começando

- Misturar a forma direta (`plt.plot`, `plt.title`) com a orientada a objetos (`ax.plot`, `ax.set_title`) sem perceber a diferença — os métodos têm nomes ligeiramente diferentes (`plt.title()` vs `ax.set_title()`), e misturar as duas formas de forma inconsistente costuma gerar confusão em gráficos com múltiplos subplots.
- Esquecer `plt.show()` em um script `.py` rodado fora de notebook e achar que “nada aconteceu” — sem essa chamada, a janela do gráfico nunca é aberta (em notebooks isso normalmente já acontece automaticamente, o que mascara o hábito de esquecer o `.show()`).
- Não usar `fig, ax = plt.subplots()` desde o início e precisar reescrever tudo depois para adicionar um segundo gráfico — vale começar já com essa estrutura mesmo em gráficos simples, para facilitar expandir depois.

Exercício prático

1. Crie uma figura com `plt.subplots()` e desenha uma linha com os dados `x = [1, 2, 3, 4, 5]` e `y = [5, 7, 4, 9, 12]`.
2. Adicione título “Meu primeiro gráfico”, rótulo “X” no eixo horizontal e “Y” no eixo

vertical.

3. Crie uma figura com 2 subplots lado a lado (1, 2): no primeiro, desenhe a linha do item 1; no segundo, desenhe um gráfico de barras (`ax.bar`) com os mesmos dados.
4. Salve a figura do item 3 em um arquivo `meus_graficos.png`.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Entendo a diferença entre `figure` e `axes` no Matplotlib
- ☐ Sei criar um gráfico simples com `fig, ax = plt.subplots()`
- ☐ Sei criar múltiplos subplots numa mesma figura
- ☐ Sei salvar um gráfico em arquivo com `fig.savefig()`
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Gráficos de linha e barra

| Módulo 7 — Visualização · Tópico 2 de 7

O que é e por que importa

Linha e barra são os dois gráficos mais usados em análise de dados de negócio, e cada um tem um propósito claro:

- **Gráfico de linha:** mostra **evolução ao longo de uma sequência contínua** — quase sempre tempo (vendas por mês, usuários ativos por dia). A linha conecta pontos, sugerindo continuidade entre eles.
- **Gráfico de barra:** compara **valores entre categorias distintas** — faturamento por produto, número de clientes por cidade. Cada barra é independente das outras, sem sugerir uma sequência contínua.

Usar o gráfico errado para o tipo de dado (por exemplo, linha para comparar categorias sem ordem natural) pode sugerir uma tendência que não existe.

Como funciona (com exemplo comentado)

```

import matplotlib.pyplot as plt
import pandas as pd

# GRÁFICO DE LINHA -- para série temporal / sequência contínua
meses = ["Jan", "Fev", "Mar", "Abr", "Mai", "Jun"]
faturamento = [12000, 13500, 11800, 15200, 16000, 17500]

fig, ax = plt.subplots()
ax.plot(meses, faturamento, marker="o") # marker="o" destaca cada ponto c
ax.set_title("Faturamento mensal")
ax.set_xlabel("Mês")
ax.set_ylabel("Faturamento (R$)")
plt.show()

# múltiplas linhas na mesma figura -- útil para comparar séries
faturamento_2023 = [10000, 11000, 10500, 12000, 13000, 14000]
fig, ax = plt.subplots()
ax.plot(meses, faturamento, marker="o", label="2024")
ax.plot(meses, faturamento_2023, marker="s", label="2023")
ax.set_title("Faturamento mensal: 2023 vs 2024")
ax.legend() # mostra a legenda com os "label" de cada linha
plt.show()

# GRÁFICO DE BARRA -- para comparar categorias
produtos = ["Caneta", "Caderno", "Mochila", "Estojo"]
vendas = [320, 150, 45, 90]

fig, ax = plt.subplots()
ax.bar(produtos, vendas, color="steelblue")
ax.set_title("Vendas por produto")
ax.set_xlabel("Produto")
ax.set_ylabel("Unidades vendidas")
plt.show()

# barra horizontal -- útil quando os nomes das categorias são longos
fig, ax = plt.subplots()
ax.barh(produtos, vendas, color="steelblue")
ax.set_title("Vendas por produto (horizontal)")
plt.show()

# ordenando as barras do maior para o menor -- quase sempre melhora a leit
df = pd.DataFrame({"produto": produtos, "vendas": vendas}).sort_values("ve
fig, ax = plt.subplots()
ax.barh(df["produto"], df["vendas"], color="steelblue")
ax.set_title("Vendas por produto (ordenado)")
plt.show()

# Gerando um gráfico direto de um DataFrame com .plot() (visto de leve no
# combina groupby (Módulo 4) + plot em uma linha só, muito comum no dia a
vendas_df = pd.DataFrame({
    "mes": meses,
    "faturamento": faturamento,
})
vendas_df.plot(x="mes", y="faturamento", kind="line", marker="o", title="F
plt.show()

```

Erros comuns de quem está começando

- Usar gráfico de linha para comparar categorias sem ordem natural (como produtos, cidades) — a linha sugere uma progressão contínua entre os pontos que não existe nesse tipo de dado; o correto é barra.
- Não ordenar as barras quando a comparação entre categorias é o ponto principal do gráfico — barras em ordem alfabética ou “como vieram no dataset” dificultam identificar rapidamente o maior e o menor valor; ordenar (Módulo 4, Tópico 7) quase sempre melhora a leitura.
- Esquecer `ax.legend()` ao desenhar múltiplas linhas/barras com `label` definido —

sem essa chamada, os rótulos passados em `label=...` não aparecem no gráfico, mesmo estando corretamente atribuídos.

Exercício prático

```
trimestres = ["T1", "T2", "T3", "T4"]
vendas_loja_a = [5000, 6200, 5800, 7100]
vendas_loja_b = [4800, 5100, 6000, 6500]
```

```
categorias_populares = ["Eletrônicos", "Roupas", "Casa", "Beleza"]
receita_categoria = [45000, 32000, 18000, 27000]
```

1. Crie um gráfico de linha comparando `vendas_loja_a` e `vendas_loja_b` ao longo dos trimestres, com legenda.
2. Crie um gráfico de barra vertical com a receita por categoria.
3. Ordene as categorias da maior para a menor receita e refaça o gráfico como barra horizontal.

Solução comentada

- [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei quando usar gráfico de linha e quando usar gráfico de barra
- ☐ Sei desenhar múltiplas linhas/barras na mesma figura, com legenda
- ☐ Sei ordenar as categorias de um gráfico de barra para facilitar a leitura
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Histogramas e boxplots

— Módulo 7 — Visualização · Tópico 3 de 7

O que é e por que importa

No Módulo 6 (Tópico 3) você já usou `pd.cut` para “aproximar” um histograma em texto, e mencionou que gráficos de verdade viriam neste módulo. Agora é hora de desenhá-los de fato, junto com o **boxplot** — outro gráfico central para entender a distribuição de uma variável numérica, especialmente útil para comparar a dispersão entre grupos e identificar outliers visualmente (retomando o Módulo 5, Tópico 6).

- **Histograma:** mostra a forma da distribuição — onde os valores se concentram, se há assimetria, se existem múltiplos “picos”.
- **Boxplot** (ou “diagrama de caixa”): resume a distribuição usando os quartis (Q1, mediana, Q3) e destaca outliers como pontos fora dos “bigodes” — é uma forma compacta de comparar várias distribuições lado a lado.

Como funciona (com exemplo comentado)

```

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# HISTOGRAMA
np.random.seed(0)
precos = np.concatenate([np.random.normal(30, 5, 200), np.random.normal(80

fig, ax = plt.subplots()
ax.hist(precos, bins=20, color="steelblue", edgecolor="white")
ax.set_title("Distribuição de preços")
ax.set_xlabel("Preço (R$)")
ax.set_ylabel("Frequência")
plt.show()

# testar mais de um número de bins ajuda a não tirar conclusões precipitad
# (revisão do Módulo 6, Tópico 3, sobre a escolha de bins)
fig, eixos = plt.subplots(1, 3, figsize=(12, 4))
for ax, bins in zip(eixos, [5, 20, 50]):
    ax.hist(precos, bins=bins, color="steelblue", edgecolor="white")
    ax.set_title(f"bins={bins}")
plt.tight_layout()
plt.show()

# BOXPLOT com Matplotlib puro
fig, ax = plt.subplots()
ax.boxplot(precos, vert=True)
ax.set_title("Boxplot de preços")
ax.set_ylabel("Preço (R$)")
plt.show()
# a "caixa" vai de Q1 a Q3 (Módulo 5/6); a linha no meio é a mediana; os
# "bigodes" se estendem até 1.5x o IQR (mesma regra do Módulo 5, Tópico 6)
# pontos além disso são desenhados individualmente como possíveis outliers

# Boxplot comparando grupos -- muito mais informativo que um só
df = pd.DataFrame({
    "loja": ["A"] * 100 + ["B"] * 100,
    "venda_diaria": np.concatenate([
        np.random.normal(1000, 100, 100),
        np.random.normal(1000, 300, 100), # mesma média, muito mais varia
    ]),
})

fig, ax = plt.subplots()
df.boxplot(column="venda_diaria", by="loja", ax=ax)
# boxplot direto do Pandas, agrupando por uma coluna categórica
ax.set_title("Vendas diárias por loja")
ax.set_ylabel("Venda diária (R$)")
plt.suptitle("") # remove o título automático duplicado que o Pandas adic
plt.show()
# mesmo com médias parecidas, a caixa da Loja B é bem mais "alta" (mais di
# -- o mesmo insight do Módulo 6, Tópico 2, agora visível de relance

# Histograma direto do Pandas (revisão do Módulo 6)
pd.Series(precos).plot(kind="hist", bins=20, title="Preços (via Pandas)")
plt.show()

```

Erros comuns de quem está começando

- Usar poucos bins num histograma e concluir que a distribuição é simétrica, quando mais faixas revelariam uma cauda ou um segundo pico — vale sempre testar mais de uma quantidade de bins antes de decidir sobre a forma da distribuição.
- Interpretar todo ponto fora dos “bigodes” de um boxplot como erro de dado — como visto no Módulo 5 (Tópico 6), nem todo outlier é um erro; o boxplot só sinaliza que o valor é estatisticamente incomum, a decisão sobre o que fazer com ele exige contexto.
- Comparar boxplots de grupos com tamanhos de amostra muito diferentes sem

mentonar isso — um grupo com poucos pontos pode ter uma caixa “enganosamente” pequena ou grande só por ter menos dados.

Exercício prático

```
import numpy as np
np.random.seed(1)
tempo_atendimento_a = np.random.normal(5, 1, 150) # minutos, atendente A
tempo_atendimento_b = np.random.normal(5, 3, 150) # mesma média, mais va
```

1. Crie um histograma de tempo_atendimento_a com 15 bins.
2. Crie um boxplot comparando tempo_atendimento_a e tempo_atendimento_b lado a lado (dica: ax.boxplot([tempo_atendimento_a, tempo_atendimento_b], labels=["Atendente A", "Atendente B"])).
3. Com base no boxplot, escreva (em um comentário) qual atendente tem tempo de atendimento mais previsível.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Sei criar um histograma e testar diferentes valores de bins
- ☐ Sei ler um boxplot (Q1, mediana, Q3, bigodes, outliers)
- ☐ Sei comparar distribuições de grupos diferentes com boxplot
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Scatter plots e relação entre variáveis

| Módulo 7 — Visualização · Tópico 4 de 7

O que é e por que importa

No Módulo 6 (Tópico 4) você calculou correlação como um **número**. O **scatter plot** (gráfico de dispersão) é a forma visual dessa mesma pergunta: cada ponto representa uma linha do dataset, posicionado pelo valor de duas variáveis numéricas (uma no eixo X, outra no eixo Y). É a melhor ferramenta para checar visualmente se duas variáveis parecem relacionadas, e principalmente para detectar relações **não-lineares** que o coeficiente de correlação de Pearson pode não capturar bem (mencionado no Módulo 6, Tópico 4).

Como funciona (com exemplo comentado)

```

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

np.random.seed(0)
horas_estudo = np.random.uniform(0, 10, 50)
nota = horas_estudo * 0.8 + np.random.normal(0, 1, 50) # relação aproxima

# scatter plot básico
fig, ax = plt.subplots()
ax.scatter(horas_estudo, nota, alpha=0.7)
# alpha controla transparência -- útil quando há pontos sobrepostos
ax.set_title("Horas de estudo x Nota")
ax.set_xlabel("Horas de estudo")
ax.set_ylabel("Nota")
plt.show()

# Colorindo pontos por uma terceira variável categórica -- adiciona uma dimensão
df = pd.DataFrame({
    "horas_estudo": horas_estudo,
    "nota": nota,
    "turno": np.random.choice(["Manhã", "Noite"], size=50),
})

fig, ax = plt.subplots()
for turno, cor in [("Manhã", "steelblue"), ("Noite", "orange")]:
    subset = df[df["turno"] == turno]
    ax.scatter(subset["horas_estudo"], subset["nota"], label=turno, alpha=0.7)
ax.set_title("Horas de estudo x Nota, por turno")
ax.legend()
plt.show()

# Tamanho do ponto representando uma quarta variável (bubble chart)
tamanho_turma = np.random.uniform(10, 100, 50)
fig, ax = plt.subplots()
ax.scatter(horas_estudo, nota, s=tamanho_turma, alpha=0.5, color="steelblue")
ax.set_title("Horas de estudo x Nota (tamanho = alunos na turma)")
plt.show()

# Adicionando uma linha de tendência simples (regressão linear com numpy)
coeficientes = np.polyfit(horas_estudo, nota, deg=1) # deg=1 = linha reta
linha_tendencia = np.poly1d(coeficientes)

fig, ax = plt.subplots()
ax.scatter(horas_estudo, nota, alpha=0.6, color="steelblue")
x_ordenado = np.sort(horas_estudo)
ax.plot(x_ordenado, linha_tendencia(x_ordenado), color="red", linewidth=2,
ax.legend()
ax.set_title("Horas de estudo x Nota, com linha de tendência")
plt.show()

# Relação não-Linear -- correlação de Pearson pode enganar aqui
x_curva = np.linspace(-5, 5, 100)
y_curva = x_curva ** 2 + np.random.normal(0, 2, 100) # relação em U, não-linear

fig, ax = plt.subplots()
ax.scatter(x_curva, y_curva, alpha=0.6, color="steelblue")
ax.set_title("Relação não-linear (correlação de Pearson pode ser perto de 0)")
plt.show()
print("Correlação (Pearson):", np.corrcoef(x_curva, y_curva)[0, 1])
# o número pode ficar perto de 0, mesmo havendo uma relação clara (em U) -
# reforça a importância de visualizar, não só calcular

```

Erros comuns de quem está começando

- Calcular só a correlação numérica (Módulo 6) sem nunca visualizar os dados com scatter plot — relações não-lineares, agrupamentos ou outliers que distorcem o

coeficiente ficam invisíveis olhando só para o número.

- Usar muitos pontos sobrepostos sem alpha (transparência), fazendo a região mais densa parecer uma “mancha” sólida sem noção de quantos pontos realmente existem ali.
- Adicionar uma linha de tendência linear (np.polyfit com deg=1) em dados que claramente têm uma relação não-linear (como o exemplo em U) — isso sugere visualmente uma relação que não existe da forma representada.

Exercício prático

```
import numpy as np
np.random.seed(3)
tamanho_casa = np.random.uniform(50, 300, 60) # m²
preco_casa = tamanho_casa * 3000 + np.random.normal(0, 30000, 60) # reais
regiao = np.random.choice(["Centro", "Periferia"], size=60)
```

1. Crie um scatter plot de tamanho_casa (X) por preco_casa (Y).
2. Cora os pontos por regiao, com legenda.
3. Adicione uma linha de tendência linear ao gráfico do item 1 (sem a separação por região).
4. Calcule a correlação entre tamanho_casa e preco_casa e confirme que ela é coerente com o padrão visual do gráfico.

Solução comentada

► Clique para ver a solução

Checklist antes de avançar

- ☐ Sei criar um scatter plot e interpretar o padrão de pontos
- ☐ Sei colorir pontos por uma variável categórica adicional
- ☐ Sei adicionar uma linha de tendência linear
- ☐ Entendo por que relações não-lineares podem enganar a correlação de Pearson
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Customização de gráficos

— Módulo 7 — Visualização · Tópico 5 de 7

O que é e por que importa

Um gráfico tecnicamente correto ainda pode comunicar mal se estiver confuso, com cores aleatórias, sem contexto ou difícil de ler. Este tópico reúne os ajustes que separam um gráfico “de exploração rápida” (para você mesmo entender os dados) de um gráfico “pronto para apresentar” (para comunicar um achado a outra pessoa) — cores consistentes, anotações, grades e tamanho adequado.

Como funciona (com exemplo comentado)

```

import matplotlib.pyplot as plt
import numpy as np

meses = ["Jan", "Fev", "Mar", "Abr", "Mai", "Jun"]
faturamento = [12000, 13500, 11800, 15200, 16000, 21000]

# Tamanho da figura -- figsize=(largura, altura) em polegadas
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(meses, faturamento, marker="o", color="#2b6cb0", linewidth=2)

# Título e rótulos com controle de tamanho de fonte
ax.set_title("Faturamento mensal – 2024", fontsize=14, fontweight="bold")
ax.set_xlabel("Mês", fontsize=11)
ax.set_ylabel("Faturamento (R$)", fontsize=11)

# Grade (grid) -- ajuda a ler valores aproximados sem poluir o gráfico
ax.grid(True, alpha=0.3)

# Anotando um ponto específico -- útil para destacar um evento ou marco
ax.annotate(
    "Recorde!",
    xy=(5, 21000),          # ponto que está sendo anotado
    xytext=(3.5, 19500),    # posição do texto
    arrowprops=dict(arrowstyle="->", color="gray"),
)

# Formatando o eixo Y para mostrar como moeda (sem depender de biblioteca)
ax.set_yticks([10000, 12500, 15000, 17500, 20000])
ax.set_yticklabels([f"R$ {v:,.0f}".replace(",", ".") for v in ax.get_yticklabels()])

plt.show()

# Paletas de cores consistentes para categorias -- evita cores aleatórias
# entre gráficos diferentes do mesmo relatório
categorias = ["Papeleria", "Eletrônicos", "Roupas", "Casa"]
valores = [12000, 25000, 18000, 9000]
cores = ["#2b6cb0", "#dd6b20", "#38a169", "#805ad5"] # uma cor fixa por categoria

fig, ax = plt.subplots()
ax.bar(categorias, valores, color=cores)
ax.set_title("Faturamento por categoria")
plt.show()

# Removendo "bordas" desnecessárias (spines) -- comum em gráficos mais limpos
fig, ax = plt.subplots()
ax.plot(meses, faturamento, marker="o", color="#2b6cb0")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.set_title("Faturamento mensal (visual mais limpo)")
plt.show()

# Definindo um "estilo" global do Matplotlib -- muda a aparência de todos
# os gráficos seguintes de uma vez (fontes, cores de fundo, grade padrão, etc)
plt.style.use("seaborn-v0_8-whitegrid") # nome pode variar conforme a versão
fig, ax = plt.subplots()
ax.plot(meses, faturamento, marker="o")
ax.set_title("Com estilo pré-definido")
plt.show()
plt.style.use("default") # volta ao estilo padrão para os próximos gráficos

```

Erros comuns de quem está começando

- Usar cores diferentes para a mesma categoria em gráficos diferentes do mesmo relatório — isso confunde quem está lendo, que naturalmente espera que uma cor “signifique” sempre a mesma coisa ao longo de uma apresentação.
- Exagerar em anotações, grades e cores a ponto do gráfico ficar poluído — cada elemento visual deveria ajudar a entender o dado, não competir por atenção; menos

costuma ser mais em visualização de dados.

- Deixar o `figsize` padrão (pequeno) para gráficos com muitos rótulos ou categorias, gerando texto sobreposto e ilegível — ajustar `figsize` (e `plt.tight_layout()`) resolve a maior parte desses casos.

Exercício prático

```
produtos = ["Caneta", "Caderno", "Mochila", "Estojo", "Lapis"]
vendas = [320, 150, 45, 90, 280]
```

1. Crie um gráfico de barra com `figsize=(8, 5)`, título em negrito, e uma cor fixa diferente para cada barra (defina uma lista de 5 cores).
2. Adicione uma grade horizontal sutil (`alpha` baixo).
3. Anote com uma seta o produto de menor venda (*Mochila*), destacando que é o “menos vendido”.
4. Remova as bordas superior e direita do gráfico.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei ajustar `figsize`, título, rótulos e cores de um gráfico
- ☐ Sei adicionar grades e anotações sem poluir o gráfico
- ☐ Sei manter cores consistentes para as mesmas categorias entre gráficos
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Introdução ao Seaborn

| Módulo 7 — Visualização · Tópico 6 de 7

O que é e por que importa

Seaborn é uma biblioteca construída **em cima do Matplotlib**, focada em gráficos estatísticos: ela sabe trabalhar diretamente com DataFrames do Pandas (sem precisar extrair colunas manualmente), calcula automaticamente agregações comuns (como médias com intervalo de confiança) e já vem com um visual mais elegante por padrão. Para boa parte dos gráficos de EDA (retomando o Módulo 6), Seaborn exige bem menos código que Matplotlib puro.

Instalação (se ainda não tiver):

```
pip install seaborn
```

Convenção de importação:

```
import seaborn as sns
```

Como funciona (com exemplo comentado)

```

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

np.random.seed(0)
df = pd.DataFrame({
    "categoria": np.random.choice(["Papeleria", "Eletronicos", "Roupas"],
    "preco": np.concatenate([
        np.random.normal(15, 5, 70),
        np.random.normal(500, 100, 60),
        np.random.normal(80, 20, 70),
    ]),
    "avaliacao": np.random.uniform(3, 5, 200),
})

# Seaborn aplica um estilo visual mais agradável globalmente
sns.set_theme(style="whitegrid")

# histplot -- histograma, equivalente ao ax.hist do Matplotlib mas mais si
sns.histplot(data=df, x="preco", bins=30)
plt.title("Distribuição de preços")
plt.show()

# histplot com separação por categoria -- Seaborn cuida das cores automati
sns.histplot(data=df, x="preco", hue="categoria", bins=30)
plt.title("Distribuição de preços por categoria")
plt.show()

# boxplot -- muito mais direto que no Matplotlib puro (Tópico 3)
sns.boxplot(data=df, x="categoria", y="preco")
plt.title("Preço por categoria")
plt.show()

# scatterplot -- equivalente ao ax.scatter, mas aceita "hue" (cor por cate
# e "size" (tamanho por variável numérica) direto como parâmetros
sns.scatterplot(data=df, x="preco", y="avaliacao", hue="categoria", alpha=
plt.title("Preço x Avaliação")
plt.show()

# barplot -- desenha barras com a MÉDIA de uma coluna por categoria automa
# (não precisa fazer groupby manualmente, embora ainda seja útil saber faz
sns.barplot(data=df, x="categoria", y="preco", errorbar="sd")
# errorbar="sd" mostra o desvio padrão como barra de erro em cada barra --
# uma forma visual de já trazer a dispersão (Módulo 6, Tópico 2) junto com
plt.title("Preço médio por categoria (com desvio padrão)")
plt.show()

# heatmap -- ótimo para visualizar matrizes de correlação (Módulo 6, Tópic
df_numerico = pd.DataFrame({
    "preco": df["preco"],
    "avaliacao": df["avaliacao"],
    "vendas": np.random.uniform(10, 500, 200),
})
matriz_correlacao = df_numerico.corr()

sns.heatmap(matriz_correlacao, annot=True, cmap="coolwarm", vmin=-1, vmax=
# annot=True escreve o valor numérico em cada célula
plt.title("Matriz de correlação")
plt.show()

# pairplot -- gera uma grade de scatter plots entre TODAS as colunas numér
# de uma vez, ótimo para uma primeira olhada exploratória (Módulo 6, Tópic
sns.pairplot(df_numerico)
plt.show()

```

Erros comuns de quem está começando

- Continuar extraindo colunas manualmente (`df["coluna"].values`) como se estivesse usando Matplotlib puro, em vez de aproveitar que a maioria das funções do Seaborn aceita `data=df`, `x="coluna"`, `y="outra_coluna"` diretamente — isso é justamente o que torna o Seaborn mais rápido de usar com dados tabulares.
- Usar `sns.barplot()` esperando que ele desenhe a **soma** dos valores (como um `groupby().sum()` faria), quando na verdade o padrão do Seaborn é calcular a **média** — se a soma for o que interessa, é mais seguro calcular com `groupby` (Módulo 4, Tópico 8) e desenhar o resultado.
- Rodar `sns.pairplot()` em datasets com muitas colunas numéricas (10+) sem perceber que o número de gráficos gerados cresce ao quadrado — o gráfico fica lento de gerar e difícil de ler; nesses casos, vale selecionar um subconjunto de colunas antes.

Exercício prático

```
import numpy as np
np.random.seed(2)
df = pd.DataFrame({
    "regiao": np.random.choice(["Norte", "Sul", "Sudeste"], size=150),
    "temperatura": np.random.normal(25, 5, 150),
    "vendas_sorvete": np.random.uniform(50, 500, 150),
})
```

1. Crie um histograma de temperatura separado por região (use `hue`).
2. Crie um boxplot de temperatura por região.
3. Crie um scatterplot de temperatura (x) por vendas_sorvete (y), colorido por região.
4. Crie um heatmap da matriz de correlação entre temperatura e vendas_sorvete.

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei usar `data=df`, `x=...`, `y=...` diretamente com funções do Seaborn
- ☐ Sei usar `hue` para separar categorias por cor automaticamente
- ☐ Sei criar `histplot`, `boxplot`, `scatterplot`, `barplot` e `heatmap`
- ☐ Sei quando `pairplot` é útil e quando evitar (muitas colunas)
- ☐ Resolvi o exercício sem olhar a solução

ewpage

Escolhendo o gráfico certo

| Módulo 7 — Visualização · Tópico 7 de 7 (último do módulo)

O que é e por que importa

Com Matplotlib e Seaborn no repertório, a última habilidade a desenvolver não é técnica — é de **julgamento**: dado um conjunto de dados e uma pergunta, qual gráfico comunica a resposta com mais clareza? Este tópico fecha o módulo com um guia de decisão e um checklist de qualidade, para você não escolher um gráfico só porque “é o que sei fazer”, mas porque é o mais adequado à pergunta.

Como funciona (guia de decisão)

A escolha do gráfico depende principalmente de dois fatores: **quantas variáveis** você quer mostrar, e **de que tipo** elas são (numérica ou categórica).

Uma variável numérica	-> Histograma ou boxplot (Tópico 3)
Uma variável categórica	-> Gráfico de barra com contagens (Tópico 2)
Evolução ao longo do tempo	-> Gráfico de linha (Tópico 2)
Duas variáveis numéricas	-> Scatter plot (Tópico 4)
Numérica x categórica (Tópicos 3 e 6)	-> Boxplot por grupo, ou barra com médias
Duas variáveis categóricas	-> Barra empilhada/agrupada, ou heatmap de uma crosstab
Muitas variáveis numéricas (Tópico 6)	-> Heatmap de correlação, ou pairplot

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

np.random.seed(4)
df = pd.DataFrame({
    "categoria": np.random.choice(["Papeleria", "Eletronicos", "Roupas"],
    "regiao": np.random.choice(["Norte", "Sul"], size=200),
    "preco": np.random.uniform(10, 200, 200),
    "avaliacao": np.random.uniform(3, 5, 200),
})

# Exemplo de "duas variáveis categóricas" -- barra agrupada a partir de um
# (revisão do Módulo 6, Tópico 6)
tabela = pd.crosstab(df["categoria"], df["regiao"])
tabela.plot(kind="bar", figsize=(7, 4))
plt.title("Quantidade de produtos por categoria e região")
plt.ylabel("Quantidade")
plt.tight_layout()
plt.show()

# Checklist de qualidade -- aplicando num gráfico "final", pronto para apr
fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=df, x="categoria", y="preco", ax=ax)
ax.set_title("Distribuição de preço por categoria", fontsize=13, fontweigh
ax.set_xlabel("Categoria")
ax.set_ylabel("Preço (R$)")
ax.grid(True, axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# Perguntas para revisar antes de considerar um gráfico "pronto":
# 1. O tipo de gráfico é o mais adequado para a pergunta e o tipo de dado?
# 2. Título e eixos estão rotulados (com unidade, quando fizer sentido)?
# 3. As cores são consistentes com outros gráficos do mesmo relatório?
# 4. Dá para entender o gráfico sem precisar de explicação adicional em te
# 5. Não há elementos visuais desnecessários (grades excessivas, 3D sem
#     necessidade, cores demais) competindo por atenção?
```

Erros comuns de quem está começando

- Escolher gráfico de pizza para comparar muitas categorias — pizza fica ilegível com mais de 4-5 fatias, e comparar tamanhos de fatia é mais difícil visualmente do que comparar o comprimento de barras; para a maioria dos casos, barra é a escolha mais segura.
- Usar 3D “porque parece mais impressionante” quando duas dimensões já bastam — gráficos 3D costumam ser mais difíceis de ler com precisão do que a versão 2D equivalente, mesmo quando parecem visualmente interessantes.
- Gerar o gráfico e considerar a tarefa terminada sem revisar título, rótulos e clareza — um gráfico sem rótulos claros exige que quem o interpreta já saiba o contexto de antemão, o que raramente é o caso fora da sua própria análise.

Exercício prático

Para cada pergunta abaixo, escolha o tipo de gráfico mais adequado (justificando em uma frase) e implemente com dados fictícios de sua escolha:

1. “Como o faturamento da empresa evoluiu nos últimos 12 meses?”
2. “Qual a distribuição de idades dos clientes?”
3. “Existe relação entre gastos em marketing e vendas?”
4. “Como o preço médio varia entre as 5 categorias de produto da loja?”
5. “Quais variáveis numéricas do meu dataset parecem mais correlacionadas entre si?”

Solução comentada

► [Clique para ver a solução](#)

Checklist antes de avançar

- ☐ Sei escolher o gráfico adequado com base no tipo e quantidade de variáveis
- ☐ Sei revisar um gráfico usando o checklist de qualidade (título, rótulos, clareza)
- ☐ Sei explicar por que evitar gráfico de pizza com muitas categorias e gráficos 3D desnecessários
- ☐ Resolvi o exercício sem olhar a solução

Checklist do Módulo 7

Antes de seguir para o Projeto Final, confirme que você consegue, sem consultar a apostila:

- ☐ Criar um gráfico básico com `fig, ax = plt.subplots()`
- ☐ Escolher entre gráfico de linha e de barra de acordo com o tipo de dado
- ☐ Criar histogramas e boxplots, e interpretá-los
- ☐ Criar scatter plots e interpretar padrões (lineares e não-lineares)
- ☐ Customizar título, rótulos, cores e anotações de um gráfico
- ☐ Usar Seaborn para gráficos estatísticos direto de um DataFrame
- ☐ Escolher o tipo de gráfico certo para uma pergunta de análise específica

Pronto? Siga para o Projeto Final.

[ewpage](#)

Projeto Final — Projeto Integrador

| 3–5 dias

O que é este projeto

Você passou por sete módulos: fundamentos de Python, ambiente e ferramentas, NumPy, Pandas, limpeza de dados, estatística/EDA e visualização. Este projeto final não ensina nada novo — ele junta tudo em um fluxo de trabalho único, do jeito que aconteceria numa análise de dados real: carregar dados brutos, limpá-los, explorá-los estatisticamente, visualizar os achados e comunicar uma conclusão.

Não existe “a” solução certa aqui. Duas pessoas podem analisar o mesmo dataset e chegar a recortes, gráficos e conclusões diferentes — o que importa é que cada decisão seja justificada e que o processo seja sólido.

O dataset

Sugestão: **dados de qualidade do ar / poluição** ou **vendas de uma loja de e-commerce** — qualquer dataset público tabular com pelo menos:

- 500+ linhas
- Uma mistura de colunas numéricas e categóricas
- Pelo menos uma coluna de data
- Problemas reais de qualidade (valores ausentes, duplicatas, tipos inconsistentes) — datasets “perfeitos demais” não deixam você praticar a parte de limpeza

Algumas fontes conhecidas de datasets públicos: Kaggle, dados.gov.br, UCI Machine Learning Repository, Google Dataset Search. Escolha um tema que genuinamente desperte sua curiosidade — você vai passar alguns dias olhando para esses dados.

Backup caso não consiga baixar nada: monte você mesmo um dataset sintético parecido com o do exercício do Módulo 5, Tópico 7 (checklist de qualidade de dados), só que maior — por exemplo, gere 500-1000 linhas de vendas fictícias com `numpy.random`, introduzindo de propósito valores ausentes, duplicatas e tipos inconsistentes, para ter algo realista para praticar.

Etapas do projeto

1. Pergunta de negócio

Antes de escrever qualquer código, defina 2-3 perguntas que você quer responder com os dados (ex: “quais produtos têm a maior margem de lucro?”, “existe sazonalidade nas vendas ao longo do ano?”, “que fatores parecem associados a avaliações baixas?”). Elas guiam todas as decisões seguintes — sem uma pergunta, é fácil se perder explorando dados sem rumo.

2. Carregar e diagnosticar (Módulos 4 e 5)

- Carregue o dataset com Pandas.
- Rode o roteiro de diagnóstico do Módulo 5, Tópico 7: `.info()`, `.isnull().sum()`, `.duplicated().sum()`, tipos de cada coluna, `.unique()` de colunas de texto.
- Documente (em comentários ou markdown, se estiver num notebook) os problemas encontrados antes de corrigir qualquer coisa.

3. Limpar (Módulo 5)

- Trate valores ausentes com uma estratégia justificada (remover, preencher com estatística, preencher por grupo).
- Remova duplicatas.
- Corrija tipos de dados (números, datas).
- Padronize texto inconsistente.
- Trate outliers, decidindo caso a caso se são erros ou eventos legítimos.
- Valide o resultado final (confirme que os problemas foram resolvidos).

4. Explorar (Módulo 6)

- Faça a análise univariada das colunas mais relevantes para suas perguntas de negócio (tendência central, dispersão, forma da distribuição).
- Faça a análise bivariada relevante (`groupby`, `crosstab`, correlação).
- Se aplicável, monte uma amostra e reflita sobre o quanto seus achados generalizariam para um conjunto de dados maior (Módulo 6, Tópico 7).

5. Visualizar (Módulo 7)

Produza pelo menos 4 gráficos, cada um respondendo diretamente a uma das perguntas definidas no passo 1. Para cada gráfico:

- Escolha o tipo certo para a pergunta e o tipo de dado (Módulo 7, Tópico 7).

- Garanta título, rótulos de eixo e, se necessário, legenda.
- Use cores consistentes se houver mais de um gráfico com as mesmas categorias.

6. Comunicar

Escreva uma síntese final (um parágrafo por pergunta de negócio) com:

- A resposta encontrada, com base nos números e gráficos.
- O nível de confiança nessa resposta (os dados são suficientes? há limitações, como amostra pequena ou muitos valores ausentes numa coluna chave?).
- Uma recomendação prática, se fizer sentido para o contexto escolhido.

Checklist de conclusão

- ☐ Defini 2-3 perguntas de negócio antes de começar a analisar
- ☐ Diagnostiquei o dataset (ausentes, duplicatas, tipos, inconsistências de texto) antes de limpar
- ☐ Apliquei e justifiquei estratégias de limpeza para cada problema encontrado
- ☐ Validei que a limpeza funcionou (sem ausentes/duplicatas remanescentes não intencionais)
- ☐ Fiz análise univariada e bivariada relevante para as perguntas definidas
- ☐ Produzi pelo menos 4 gráficos, cada um respondendo a uma pergunta específica
- ☐ Todos os gráficos têm título, rótulos e (quando aplicável) legenda
- ☐ Escrevi uma síntese final com resposta, nível de confiança e recomendação para cada pergunta

E depois?

Terminar este projeto significa que você tem, na prática, o fluxo completo de um analista de dados júnior: carregar, limpar, explorar, visualizar e comunicar. A partir daqui, os próximos passos naturais (fora do escopo desta apostila) costumam ser: aprofundar SQL para buscar dados direto de bancos, aprender uma ferramenta de dashboard (Power BI, Tableau, ou Streamlit em Python), ou seguir para Machine Learning com scikit-learn, que usa diretamente os DataFrames que você já sabe preparar.

Parabéns por chegar até aqui — o caminho de “sabe o básico de Python” até “consegue tocar uma análise de dados do início ao fim” não é curto.